Work avoidance in sparse set algebra

Marcus D. R. Klarqvist[1]

[1]**[affiliation]**

ABSTRACT

Set algebra maps Boolean membership directly onto machine bits and underlies search indexes, graph engines, databases and telemetry analytics. Sparse systems nevertheless often spend more time visiting absent state than processing useful state. We use all-pairs set-intersection cardinality, the binary Gram matrix XX^{T} underlying similarity and ranking, to develop an exact two-scale work-avoidance architecture. Dense-bitmap execution evaluates every pair in $\Theta(m)$ work even when one row is nearly empty and the answer is usually zero. At row scale, an online selector chooses among ten bitmap, sorted-array, run and fill-compressed representation pairings. At tile scale, coarse positional occupancy is transposed into column postings, allowing one occupied column to generate all row pairs it can support while the remaining pairs are proved disjoint without reading row data. On 23 real corpora, one fixed per-tile policy was $1.48\times$–$52{,}448\times$ faster than all-bitmap execution, and a fixed online refinement policy outperformed a query-tuned CRoaring baseline on 22; selection remained below 1% of runtime. In the sparse all-pairs regime, amortised within-tile column postings were $7\times$–$5{,}700\times$ faster than unfiltered bitmap–array evaluation. With a Jaccard threshold, element-keyed prefix postings accelerated 48 of 63 corpus–threshold configurations by $1.67\times$–$1{,}089\times$, while the gate routed the remainder to exact scan. SIMD-throughput kernels won none of 25 asymmetric cell–shape comparisons. All routing can begin on the first traversal from constant-size row metadata and bounded online probes; offline optimization is optional. Set intersection thus provides an exact demonstration of a broader sparse-systems rule: use metadata to certify absent work, amortise each decision across many consumers and reserve throughput optimization for the work that survives.

Set algebra is a basic computational interface for binary data. For a universe of m possible elements, a set is represented exactly by an incidence vector in $\{0,1\}^m$: membership maps to one machine bit, while union, intersection, difference and complement map to bitwise OR, AND, AND-NOT and NOT. Population count then converts the resulting membership vector to a cardinality. This direct correspondence between set semantics and machine operations underlies compressed bitmap libraries[1–3], inverted indexes and Boolean search engines[4–6], relational graph processing[7], and bitmap-indexed real-time analytics over event streams[8]. Set algebra is therefore shared systems infrastructure rather than a primitive belonging to one application domain.

Among these operations, we focus on exact intersection cardinality, $|A \cap B|$. Intersection is both a direct filter and the common overlap term in binary similarity: Jaccard similarity divides it by $|A \cup B|$, cosine similarity divides it by $\sqrt{|A||B|}$, and containment divides it by the size of one operand. These scores drive document resemblance and near-duplicate detection[9–11], ranked retrieval over binary features, chemical fingerprint screening[12–14], graph-neighbourhood comparison and co-occurrence analysis. Intersection thus supplies a representative, exact set-algebra operation on which work avoidance can be proved and measured end to end.

For a collection of sets, computing every pairwise intersection cardinality produces the symmetric binary Gram matrix $Y = XX^{\mathsf{T}}$, with $Y_{ij} = |X_i \cap X_j|$, connecting the problem directly to sparse matrix multiplication and graph algebra[15,16]. Karp, Waarts and Zweig posed the all-pairs problem of finding sparse bit-vector pairs with at least t common bits[17]. We study exact cardinalities for the complete pair collection, with thresholded reporting as a separate optional contract. At hyperscale, rows may denote documents, users, items, graph neighbourhoods, telemetry streams, access-control cohorts, categorical attributes or software artefacts; columns denote the terms, identifiers, events, principals, features or symbols they contain. The same matrix then constructs co-occurrence graphs, finds related objects, correlates signatures and materialises exact overlap statistics. Internet-scale services, global observability systems and shared open-source data platforms intensify the same systems problem: both the number of possible pairs and the width of the shared universe are large, while the incidence matrix remains sparse and heterogeneous.

This exact operation exposes a broader systems problem: deciding which work should be admitted to an expensive sparse kernel. Zone maps and related summaries can certify that a region contributes nothing; representation metadata can identify a cheaper traversal; and transposed postings can reuse one sparse fact across many consumers. Their reusable structure is a cheap summary, a sound rejection predicate, an exact survivor path and a gate that invokes the summary only when the work it removes exceeds its cost. Sparse joins, graph analytics, block-sparse algebra, inverted-index evaluation and telemetry processing expose the same opportunity whenever absence or irrelevance can be certified from metadata before payload data are visited.

The conventional dense representation evaluates each entry as `popcount(A AND B)`. Its $\Theta(m)$ cost is independent of both operand cardinalities, so an all-pairs traversal pays $\Theta(N^2 m)$ even when most rows contain only a few elements. Wider SIMD instructions reduce the constant on the AND–popcount loop[18], and established SIMD intersection kernels likewise accelerate selected sorted-list traversals[19,20]; neither change removes the words or descriptors that the selected traversal visits. Under the $1/i$ cardinality spectra found in frequency, degree and allele-count data[21], most row pairs contain a sparse operand and the expected intersection is often below one. The fixed bitmap scan then spends most of its work proving an empty answer. The representation-independent headroom created by that fixed cost is developed in Supplementary Note 1.

Exact alternative representations expose different work variables. A sorted array scales with the number of stored elements, a run representation with the number of intervals, and a fill-compressed stream with its literals and fills. Pairing two representations therefore defines a matrix of algorithms rather than one universal intersection kernel. Prior work made sorted-set intersection adaptive by changing the comparison sequence within sorted lists[22], and developed preprocessed word-parallel structures for one- and multi-set intersections[23]. Culpepper and Moffat combined compressed lists with bitmap membership tests for conjunctive inverted-index queries[24]. Roaring instead selects array, bitset and run storage per fixed-width container at ingest[2,3], while EmptyHeaded selects dense or sparse set layouts and intersection code inside compiled graph joins[7,25]. We investigate a distinct decision target: pricing the representations of both operands for count-only execution at query time and hoisting that choice across an all-pairs row tile. The resulting executor selects online among ten representation cells.

Pairwise selection recovers work within a chosen row pair, but it leaves a second source of repeated work untouched. A row-oriented traversal rediscovers the same occupied positional bucket for every pair that shares it. Transposing the coarse row–bucket incidence relation produces a column posting of row identifiers for each positional bucket. Each posting contributes all candidate pairs sharing that bucket, and buckets occupied by fewer than two rows contribute none. Outer-product sparse matrix multiplication generates product entries from element-level columns[15,16]; wedge expansion similarly increments exact common-neighbour counts for the endpoint pairs induced by each graph vertex[26]. We instead transpose coarse occupancy buckets, accepting bucket collisions as candidates and resolving them through the row matrix. A tile-sized machine-word posting deduplicates candidate masks, while the online gate sends every survivor to the selected exact representation-pair verifier. Candidate generation and verification therefore follow occupied-column multiplicity rather than requiring a payload traversal for every row pair.

Here we combine these two scales into one exact all-pairs strategy. We evaluate ten row-oriented representation pairings, select among them online at tile granularity, and use column postings to reject empty pairs collectively before invoking a row kernel. Across 23 real corpora, the online row-cell selector is 1.48×–52,448× faster than all-bitmap execution, beats a query-tuned CRoaring baseline on 22 and consumes less than 1% of runtime. Across 25 asymmetric cell–shape comparisons, every winner removes work rather than increasing SIMD throughput. In the sparse all-pairs regime, where 95–100% of sampled intersections are empty, the amortised within-tile postings path is 7×–5,700× faster than unfiltered bitmap–array evaluation. For thresholded similarity queries, a second, element-keyed posting path generates only pairs that can clear the requested cutoff and reaches 1,089× speedup over an exact scan. The resulting method matches computation to heterogeneity along both axes of the binary matrix: representation choice controls the cost of surviving row pairs, and column postings control how many pairs reach a kernel at all. The online path requires no corpus-specific optimization pass: constant-size metadata and a bounded probe-and-commit step make the routing decision during the active traversal. Offline tournaments remain available as an optional specialization when a stable, extremely large workload will reuse their decision enough to amortise the one-time search.

The architecture factorizes all-pairs work into candidate generation, exact verification and amortised planning. Positional postings determine the survivor set C, representation pairing determines $K(i,j)$ for each $(i,j) \in C$, and the online gate selects the lower-cost route. Including routing overhead, the online system outperforms all-bitmap execution on all 23 corpora (Table 1) and query-tuned CRoaring on 22 (Table 2).

The contribution is therefore a general work-avoidance architecture, rigorously instantiated for exact and thresholded sparse binary Gram matrices: representation-paired row kernels, near-free online routing and column-oriented candidate generation, with each layer scaling in the descriptors of useful work exposed at its own resolution.

Results

We evaluated exact all-pairs set-intersection cardinality at three levels: the cost surface of the row representations, online selection among their pairings, and column-posting candidate generation across row tiles. Throughout, we use *cell* to denote one representation pairing (for example B×S), *variant* to denote one implementation of a cell, and *pairing matrix* to denote the full set of cells; unless stated otherwise, every external ratio is reported against CRoaring v4.7.2 after `run_optimize()` and the query-cost promotion defined in Methods. Figure 1 lays out the row matrix: the five representation families, the driving variable each pairing's cost is proportional to, and the path from precomputed row metadata through the selector to a chosen kernel.

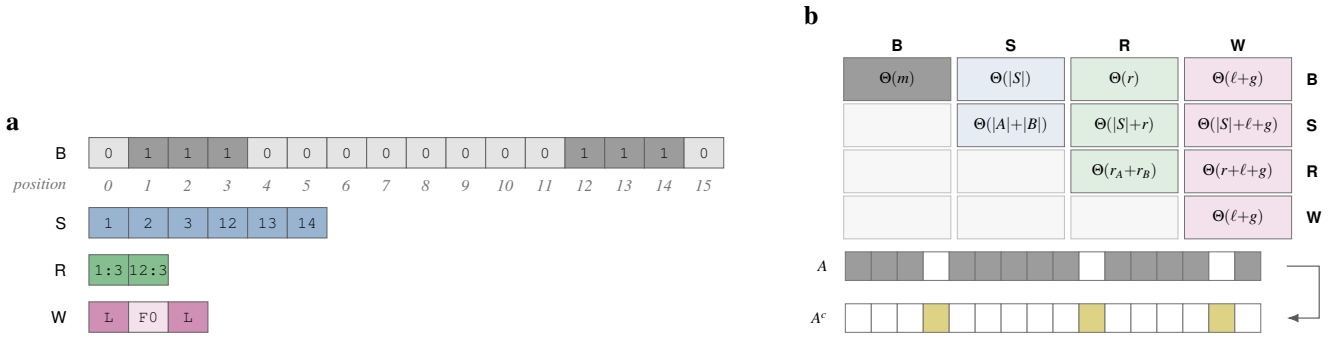

Figure 1. The object of study: equivalent encodings of one set, and the matrix of kernels that pairing them gives. a, One set of six elements over a 16-position universe, encoded four ways. One cell is one stored item, so a strip's length is the number of items that encoding must touch: 16 bits for B ($\Theta(m)$, whatever the set holds), six values for S ($\Theta(|X|)$), two (*start*:*length*) records for R ($\Theta(r)$, whatever the runs' lengths), and three words — two literal and one fill — for W ($\Theta(\ell+g)$). Note that a cell is a bit in B and a word in the others, so the strips compare work, not bytes. Strong fill marks set positions, and the position ruler applies to B alone, the only positional encoding. **b**, Pairing two representations gives a kernel with its own cost function. The matrix is symmetric, so ten cells are distinct, and *only* B×B is linear in the universe m; every other cell is sub-linear in it. Above density one half a side is handled through its complement (C), shown beneath: a dense operand becomes a sparse one and re-enters the same ten cells, with the answer recovered by inclusion–exclusion. Costs are derived in Methods. Schematic; no measured data.

All evaluated paths return the same exact cardinalities. Row cells reduce the work required for one selected pair; tile selection shares the decision across related pairs; and column postings invert coarse positional occupancy so that one column contributes to every row pair it can support. Results therefore progress from the row cost map, through online real-corpus dispatch, to the all-pairs tile algorithm. Detailed constructions and thresholds are given in Methods and Supplementary Note 5.

Total work separates into the number of surviving pairs $|C|$, their exact verification costs $K(i,j)$ and amortised planning. Positional postings control $|C|$, row representations control $K(i,j)$, and tile scheduling amortises selection and construction. We measured the complete online system, including selection, against all-bitmap execution (Table 1) and query-tuned CRoaring (Table 2), then resolved the row and column contributions by ablation.

The cost of intersection is U-shaped in effective sparsity

We first asked whether bitmap–bitmap intersection retains its fixed cost across density and where descriptor-driven cells overtake it. We measured all ten cells at 20 matched-density points from one set bit to the full universe. Every point used 96 uniformly structured rows over a fixed 65,536-bit universe on Apple M4, keeping the operand set L2-resident while density changed. The $1/i$ and clustered shapes used in the strategy comparison below exercise the corresponding heterogeneous regime.

The pure-SIMD B×B path remained between 115 and 150 ns per pair across five orders of magnitude of density. Against that fixed line, the fastest matched cell cost 1.6 ns at one set bit, a 70.1× improvement, and remained 13.2× faster at density 4.9×10^{-4}. The measured sparse-side crossover was approximately 0.4%: below it, descriptor work shrank with cardinality while bitmap work did not; through the uniform middle band, B×B was the selected cell.

The dense endpoint mirrored the sparse one when the complement was made an explicit strategy. At density 0.999, enumerating the complement cost 42.3 ns against 115.7 ns for B×B (2.7×); with one zero in 65,536 positions it cost 3.3 ns against 124.0 ns (37.2×). Thus the work variable is effective sparsity, $\min(d, 1-d)$, rather than density in one direction. Figure 2 derives the same U-shaped cost surface from descriptor lengths and shows how positional clustering expands the region in which summaries remove work.

Each mechanism above is a bet that the data deviates from randomness, and the size of the win at the sparse extreme, the location of the crossover, and the two-sided condition under which a zone map is worth consulting at all follow from a closed-form counting argument against the least favourable (independent-placement) arrangement of elements, given in full in Supplementary Note 4. Inside the uniform mid-band selected B×B because descriptor traversal admitted no less work than the bitmap scan; the return there comes from throughput rather than avoidance — the axis the vectorized `AND`-and-popcount literature has already carried a long way[18].

Online representation matching converts sparsity into end-to-end exact speedups

The density sweep establishes headroom under controlled inputs; we next tested whether an online selector recovers that headroom on data whose structure was not generated for these kernels. The evaluation contains 23 corpora spanning information-

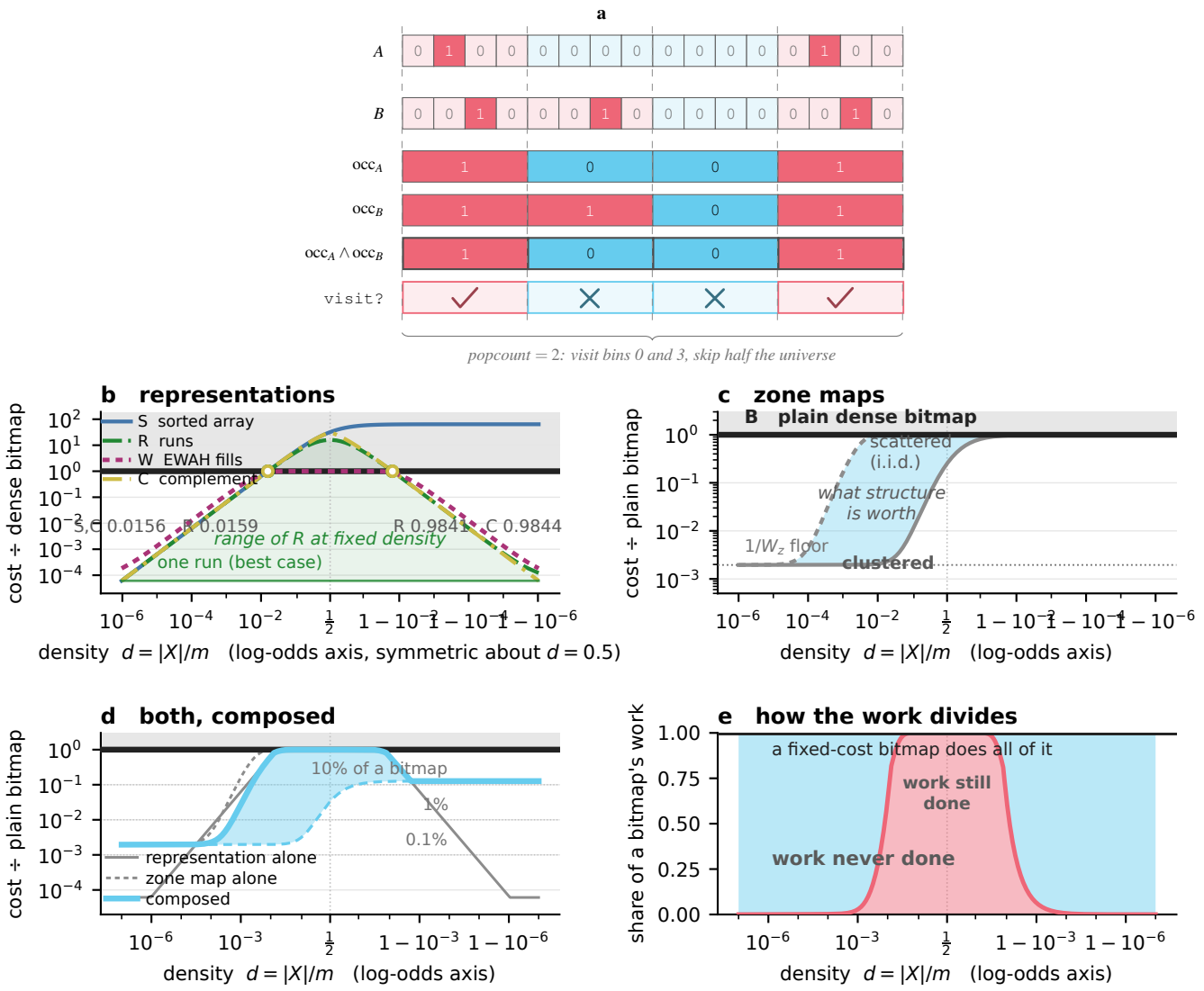

Figure 2. What a fixed cost leaves on the table, analytically. Derived, not measured; every panel models *work* rather than time. Reading key: hue names a representation, so mechanisms are grey and separated by line style; red marks work still done, cyan work avoided, and grey shading the region dearer than a plain bitmap. **a**, A zone map within one pair. Each bin carries one bit recording whether that operand holds anything in it, and $\mathrm{occ}_A \wedge \mathrm{occ}_B$ is the AND of the two summaries. Follow bin 1: B occupies it and A does not, so the pair skips the bin even though one side holds data there. The popcount of the AND is the *exact* number of bins to visit, and a popcount of zero proves the operands disjoint without reading either. Digits and glyphs repeat every verdict, so the panel reads without colour. **b**, Expected cost of each representation for a uniformly random set, as a multiple of the dense-bitmap cost, on a log-odds density axis. Every cost is proportional to the universe m, so this panel is scale-invariant and the crossovers are universal constants: open circles at $d = 1/w = 0.0156$ for S and C, 0.0159 for R, and their complements. The bitmap is the line at 1. Shading marks the range of R at fixed density — one run at best, the i.i.d. value at worst — which is the deviation the method exploits. **c**, The same for a zone map. Arrangement, not density, sets the share of bins an operand occupies: at $d = 0.01$ and $W_z = 512$ the scattered and clustered extremes differ by 482×, so the i.i.d. curve floors what a summary is worth. **d**, The two composed. Filtering concentrates density, so the representation used inside a surviving bin is the one for that bin's local density. Clustered, the composition costs 0.195% of a bitmap across the sparse range at $W_z = 512$, rising to 12.7% only in the dense half; the knee sits at $d = 1/8$ for every bin width. **e**, The same as a division of labour: the share of a bitmap's work never done against the share still done, taking the best of the four options at each density. 99.8% is never done at $d = 10^{-4}$; the waist about $d = \frac{1}{2}$ is where a bitmap is the right kernel. The measured sweep and all source records are retained in `results/density.jsonl`.

retrieval postings, web and social graphs, census and attribute data, sensor logs, and human and viral genomics. Twelve are the files used by CRoaring's own benchmark harness, and the remaining corpora extend the universe and density range[2,3]. Each pair was dispatched by one fixed online policy from row metadata. No corpus-level best cell was chosen after timing. The same per-tile policy was applied to every corpus, including the cost of selection.

Against unfiltered bitmap–bitmap execution, online representation matching won on all 23 corpora (Table 1). The end-to-end gain ranged from 1.48× on the dense `census-income` corpus to 52,448× on `enwiki-categorylinks`. The gradient is the operational consequence of the cost model: as the universe-to-cardinality ratio grows, the fixed bitmap scan grows while the selected sparse-side descriptor work does not. The result is also system-wide rather than a special sparse kernel. Dense sensor and census rows remain routed to bitmap cells, while sparse graph and retrieval rows move to array, run or constant-time rejection paths under the same online policy.

We then compared the same mechanism with a practical compressed-bitmap baseline. CRoaring was run-optimized and received the query-cost array-to-bitset promotion described in Methods, so the comparison prices cardinality execution rather than only serialized size. Each pair was dispatched by one fixed online refinement policy from row metadata; no corpus-level best cell or online policy was chosen after timing.

Online representation matching outperformed query-tuned CRoaring on 22 of 23 corpora. The winning ratios ranged from 1.16× to 6.34× across the evaluated domains (Table 2). Whole-process intervals remained above one for every reported win. The selector itself consumed 0.02–0.80% of runtime across the same corpus set, below the 2% design budget (Methods, "Online row-cell selection and offline tournaments").

The dispatch mixtures explain the result. Sparse graph and retrieval corpora route primarily to B×S, run-structured corpora activate B×R, and dense sensor and census corpora retain a substantial B×B share. Constant-time span rejection alone settles 99.5% of pairs in `dimension_003`. Thus the online system realizes the pairing matrix as a runtime algorithm: it selects sparse-side work, run-side work, dense bitmap work, or no kernel at all within the same implementation.

A storage-priced container rule leaves query cost unclaimed

The online comparison includes dense corpora whose work should approach bitmap popcount, so we asked why their container mixtures still differed. We used CRoaring's introspection API to census the array, bitset and run containers after `run_optimize()` (Methods, "Container census").

At 1.2×10^{-3} global density, `census1881` contains 1,332 array containers, no bitset containers and 132 run containers (Table 3). Even at 6.3% and 17.3% global density, `weather_sept_85` and `census-income` remain array-major because their mass is spread across many chunks. The 4,096 threshold therefore optimizes a chunk's storage size, whereas an intersection-cardinality query prices the work performed by pairs of containers.

We converted arrays with more than 64 values to bitsets after `run_optimize()`, the ordering that preserves the run decision and changes only the subsequent array–bitset choice. Every value and cardinality remained identical under differential checking. This query-tuned configuration is the CRoaring baseline in Table 2; online representation matching still won on 22 of 23 corpora. The experiment connects the external comparison to the pairing-matrix mechanism: storage and query execution have different crossovers, and a runtime selector can price the operation being performed.

Work avoidance controls every asymmetric representation pairing

The real-corpus mixtures establish that the selected cell changes with data shape. We next asked which implementation principle controls each cell: processing the same inputs at higher SIMD throughput, or arranging the computation so that fewer inputs are visited. We raced work-avoidance strategies against SIMD-throughput variants for the five asymmetric cells B×S, B×R, B×W, S×R and S×W across five corpus shapes. The shapes included uniform and $1/i$ cardinality spectra, clustered elements and long runs, with every variant evaluated on identical pairs.

Work avoidance won all 25 asymmetric cell–shape comparisons (Fig. 3a). Rank lookups answered whole runs, galloping and merge selection reduced sparse searches, occupancy maps skipped empty bitmap regions, and fill-aware traversal folded repeated words. A SIMD-throughput variant won none. This result was obtained after 13 optimization rounds covering approximately 130 variants and 2,658,633 differential checks against an independent scalar oracle. All ten cells met the stopping rule of five consecutive non-improving iterations, so the strategy map records a converged search around each measured optimum rather than a comparison with one first implementation. Prior SIMD list-intersection methods increase the throughput of a selected sorted-array traversal[19,20]. Building on these kernels, we compared throughput optimization with algorithms that change the number of descriptors visited. The latter was faster for every asymmetric cell and corpus shape tested.

Dense bitmap–bitmap intersection supplies the complementary result. Its generic path must read every word, so vector throughput remains the correct optimization target. On Sapphire Rapids, adding the AVX-512 `VPOPCNTQ` path[27] reduced measured dense B×B time from 742.1 to 73.3 ns per pair, a tenfold improvement. Across Apple M4, two Arm Neoverse systems and Sapphire Rapids, representation matching still beat query-tuned CRoaring at every tested synthetic density; the

Table 1. One online policy accelerates exact all-pairs cardinality on all 23 real corpora. Storm is the fixed per-tile selector, including its measured decision cost; all-bitmap is the unfiltered dense B×B traversal over the identical row pairs. The final column repeats the complete online measurement with the optional zone map disabled. The agreement between the two speedup columns shows that representation selection and constant-time span rejection now remove most sparse work before a zone map is considered, while the retained gated map serves the dense bitmap route. Times are nanoseconds per possible pair. Rows are generated from `results/vs_roaring.csv` and `results/vs_roaring_nozm.csv`.

Corpus	Domain	Universe m	All-bitmap (ns/pair)	Online (ns/pair)	Speedup (×)	No-map speedup (×)
enwiki-categorylinks	IR	1.30×10^8	644,587	**12.29**	**52,448**	**56,862**
uscensus2000	census	3.70×10^7	118,713	**5.21**	**22,786**	**25,499**
dbpedia-link	graph	1.83×10^7	56,479	**53.50**	**1,056**	**1,121**
wikipedia_link_en	graph	1.12×10^7	34,346	**63.18**	**544**	**645**
livejournal-groupmemberships	graph	7.49×10^6	21,389	**19.01**	**1,125**	**1,082**
com-Orkut	graph	3.07×10^6	8,888	**97.89**	**90.8**	**77.5**
com-LiveJournal	graph	4.04×10^6	11,188	**36.68**	**305**	**351**
soc-Pokec	graph	1.63×10^6	4,161	**40.47**	**103**	**103**
as-skitter	graph	1.70×10^6	4,437	**22.61**	**196**	**214**
wiki-Talk	graph	2.39×10^6	6,420	**17.83**	**360**	**379**
dimension_003	druid	3.87×10^6	10,624	**1.67**	**6,362**	**6,707**
dimension_008	druid	3.87×10^6	10,562	**7.81**	**1,352**	**1,508**
dimension_033	druid	3.87×10^6	10,837	**31.87**	**340**	**309**
census1881	census	4.28×10^6	11,560	**57.15**	**202**	**242**
census1881_srt	census	4.28×10^6	11,442	**15.39**	**743**	**529**
wikileaks-noquotes	text	1.35×10^6	3,356	**67.26**	**49.9**	**45.4**
weather_sept_85	sensor	1.02×10^6	2,554	**1,105**	**2.31**	**1.82**
census-income	census	2.00×10^5	375.0	**254.1**	**1.48**	**1.52**
usher_sarscov2	genomics	8.45×10^6	25,931	**527.6**	**49.1**	**49.5**
gnomad_chr21_exomes_af1e-3	genomics	1.46×10^6	3,783	**21.88**	**173**	**182**
msprime_1M	genomics-sim	2.00×10^6	5,622	**1,180**	**4.76**	**4.92**
msprime_100k	genomics-sim	2.00×10^5	396.2	**124.8**	**3.17**	**3.12**
msprime_10k	genomics-sim	2.00×10^4	47.70	**23.20**	**2.06**	**2.19**

Table 2. Online representation matching beats query-tuned CRoaring on 22 of 23 real corpora. Ratios are the median of three independent whole-process runs; brackets give the observed range. CRoaring v4.7.2 received both `run_optimize()` and the query-cost array-to-bitset promotion. Storm used the same online policy on every corpus, including its selection cost. The dominant disposition is the row cell or constant-time span rejection accounting for the largest share of pairs; other cells can remain active within the same corpus. Winning ratios are highlighted in bold. Values are generated from `results/vs_roaring.csv`.

Corpus	Domain	Universe m	Online vs. CRoaring (×)	Dominant disposition
enwiki-categorylinks	IR	1.30×10^8	**2.11 [2.06–2.24]**	B×S (89.1%)
uscensus2000	census	3.70×10^7	**2.00 [1.89–2.36]**	span reject (54.8%)
dbpedia-link	graph	1.83×10^7	**1.91 [1.86–1.92]**	B×S (92.8%)
wikipedia_link_en	graph	1.12×10^7	**1.71 [1.68–1.73]**	B×S (96.9%)
livejournal-groupmemberships	graph	7.49×10^6	**1.18 [1.13–1.37]**	B×S (54.9%)
com-Orkut	graph	3.07×10^6	**1.97 [1.88–1.99]**	B×S (99.5%)
com-LiveJournal	graph	4.04×10^6	**1.43 [1.37–1.44]**	B×S (81.6%)
soc-Pokec	graph	1.63×10^6	**2.33 [2.29–2.40]**	B×S (90.6%)
as-skitter	graph	1.70×10^6	**1.16 [1.13–1.29]**	B×S (73.9%)
wiki-Talk	graph	2.39×10^6	**2.15 [1.71–2.38]**	B×S (69.1%)
dimension_003	druid	3.87×10^6	**1.69 [1.68–1.69]**	span reject (99.5%)
dimension_008	druid	3.87×10^6	0.87 [0.83–0.92]	B×S (52.1%)
dimension_033	druid	3.87×10^6	**2.87 [2.85–3.05]**	B×R (70.5%)
census1881	census	4.28×10^6	**1.22 [1.19–1.27]**	span reject (63.3%)
census1881_srt	census	4.28×10^6	**3.94 [3.90–3.98]**	span reject (36.6%)
wikileaks-noquotes	text	1.35×10^6	**6.34 [6.13–6.50]**	B×R (39.0%)
weather_sept_85	sensor	1.02×10^6	**1.19 [1.04–1.22]**	B×S (53.4%)
census-income	census	2.00×10^5	**1.70 [1.68–1.72]**	B×S (53.8%)
usher_sarscov2	genomics	8.45×10^6	**1.76 [1.73–1.85]**	B×S (73.9%)
gnomad_chr21_exomes_af1e-3	genomics	1.46×10^6	**2.67 [2.67–2.76]**	B×S (70.2%)
msprime_1M	genomics-sim	2.00×10^6	**1.53 [1.05–1.70]**	B×S (83.7%)
msprime_100k	genomics-sim	2.00×10^5	**1.78 [1.73–1.81]**	B×S (83.7%)
msprime_10k	genomics-sim	2.00×10^4	**3.76 [3.70–4.36]**	B×S (72.4%)

Table 3. A storage-priced chunk threshold produces array-major layouts across the density range. Container composition of a `run_optimize()`-tuned CRoaring bitmap[2,3], counted after tuning (Methods, "Container census"), for four corpora spanning the density range of Table 2. CRoaring assigns a bitset container to a 2^{16}-bit chunk when that chunk's cardinality exceeds 4,096, the serialized-size crossover. Total chunks sums the three container counts. Bold marks the plurality container type; all four corpora are array-major.

Corpus	Global density	Array containers	Bitset containers	Run containers	Total chunks
`uscensus2000`	8.1e-7	**2,219**	0	2	2,221
`census1881`	1.2e-3	**1,332**	0	132	1,464
`weather_sept_85`	6.3e-2	**2,274**	561	21	2,856
`census-income`	1.7e-1	**553**	180	35	768

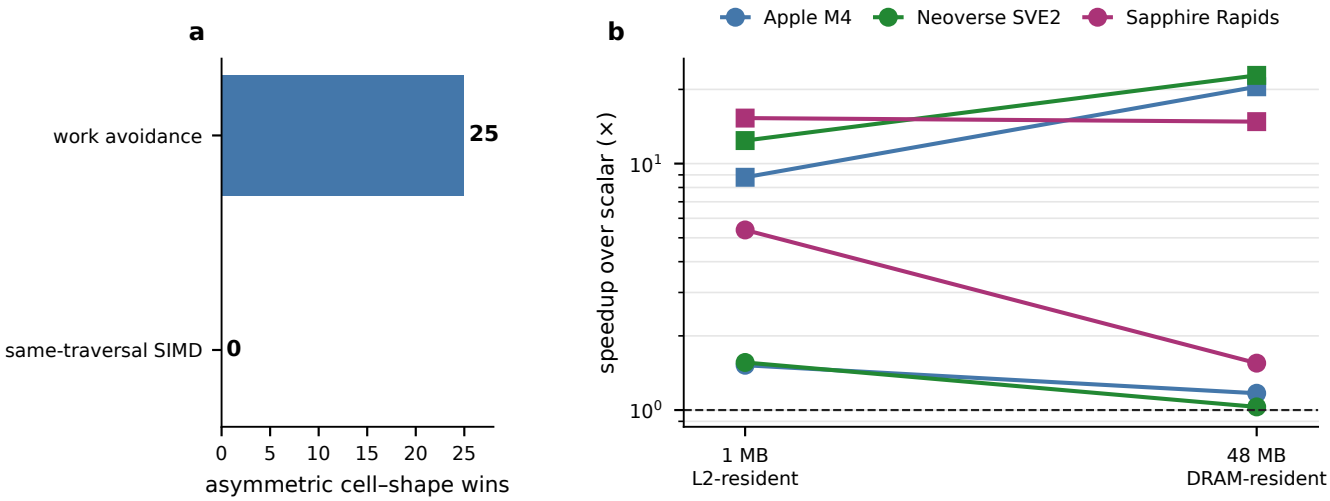

Figure 3. Work avoidance controls asymmetric cells across data shapes and memory tiers. a, Number of wins across five asymmetric representation cells and five corpus shapes. Work avoidance denotes rank lookup, occupancy skipping, fill folding or adaptive search; same-traversal SIMD retains the traversal and increases lane-level throughput. Work avoidance won all 25 comparisons. **b**, Speedup relative to the corresponding scalar kernel as the working set moves from 1-MB private-cache-resident inputs to 48-MB DRAM-resident inputs on Apple M4, Neoverse SVE2 and Sapphire Rapids. Circles denote SIMD and squares denote work avoidance; colour identifies the host. The horizontal rule marks equal time with the scalar kernel.

advantage reached 19.1× on Sapphire Rapids after the dense kernel had been strengthened. Improving the irreducible cell therefore sharpened the division of labour rather than removing the benefit of the pairing matrix.

Working-set growth reinforced the same separation. SIMD improves the rate at which resident bitmap words are processed, whereas a rank lookup, range rejection or occupancy skip removes memory accesses at every residency level. The work-avoidance advantage consequently persists as inputs cross from cache-resident to memory-resident execution (Fig. 3b). The transferable result is the strategy map: throughput governs work that must be performed; representation-aware planning governs how much work reaches that kernel.

Online tournaments make representation selection negligible

The cell map creates a runtime decision, and an N^2 traversal amplifies the cost of making it. We therefore compared a closed-form decision on every pair with two policies that share one decision across a tile: metadata-ranked per-tile selection and probe-and-commit, which races the two highest-ranked cells on representative pairs before committing. Each policy used cardinality, run count, occupancy and extent metadata computed with the row (Methods, "Online row-cell selection and offline tournaments").

Per-pair selection consumed 27.69–48.44% of runtime across three hosts. Hoisting the same ranking to tile granularity reduced that fraction to 0.10–0.29%, and probe-and-commit used 0.30–0.48% (Fig. 4a). The measurement-based tournament also corrected a model error on the Neoverse SVE2 host: model-only tile selection achieved 0.66× the all-bitmap baseline, whereas probe-and-commit reached 1.09× with 0.48% decision cost. At all-pairs scale, timing a few nominated kernels once is

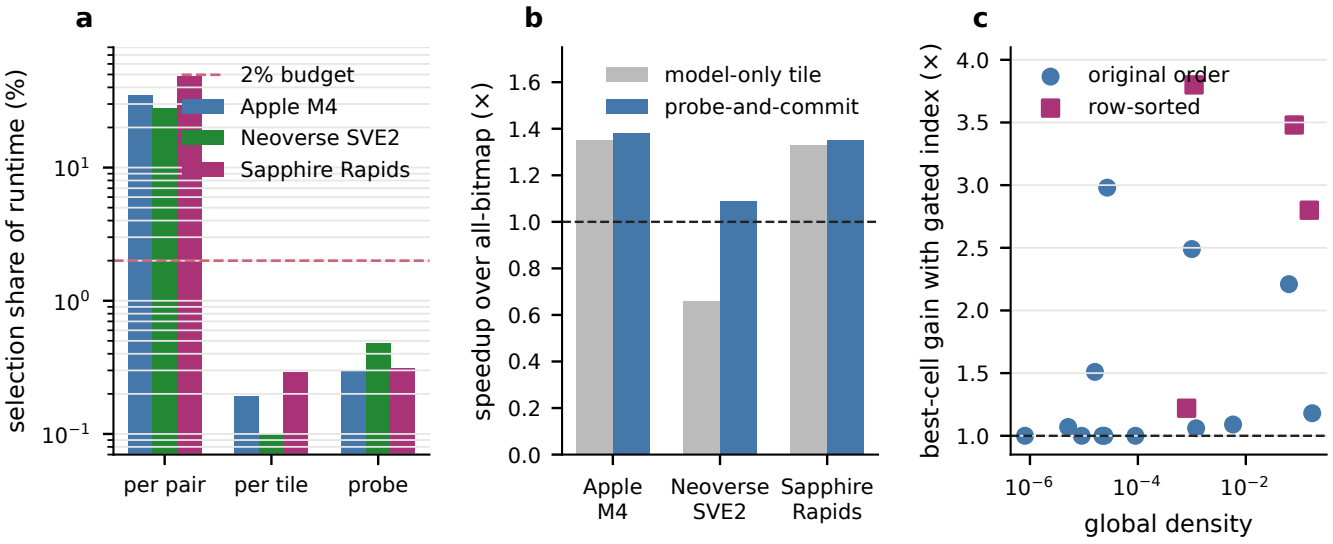

Figure 4. Hoisting and online measurement make representation selection negligible. a, Selection cost as a percentage of total runtime for per-pair, per-tile and probe-and-commit policies on three hosts. The dashed line marks the 2% design budget. **b**, End-to-end speedup over all-bitmap execution for model-only per-tile selection and probe-and-commit on the same hosts. **c**, Best-cell speedup with an optional gated zone map relative to the best index-free cell across 17 real corpora, plotted against global density. Circles denote the original row order and squares denote cardinality-sorted rows; 12 corpora improved and five were neutral.

cheaper than evaluating a detailed model on every pair.

The real-corpus policy adds a heterogeneity-gated refinement between the two nominated cells and uses the same setting for every dataset. Its measured selection share was 0.02–0.80% across all 23 corpora in Table 2. The selector is therefore part of the observed system result, not an oracle standing outside it. It requires no corpus-specific offline profile: a new or inline workload can route its first traversal from row metadata and the bounded measurements collected by probe-and-commit. Offline fixed-cell and bucket tournaments remain evaluation tools for mapping the design space and optional specializers for stable, repeatedly traversed corpora; the deployed policy uses row metadata and bounded online measurement. This extends established instance adaptivity for set intersection and database operators[22,28] from choosing progress within one operator to choosing the representation pair and row-versus-column route for an all-pairs tile.

The selector also gates optional work-avoidance indexes. A cardinality gate disables the universe-proportional zone map on sparse rows; representation routing and span rejection carry most of the system-level gain (Table 1). Optional zone-map consultation improved best-cell time on 12 of 17 corpora, was neutral on five and reached 3.80×. Metadata therefore activates the index only for row regimes that can exploit its work variable.

Column postings turn sparse all-pairs intersection into candidate generation

Row-cell selection minimizes the cost of a pair after that pair has been chosen. Sparse all-pairs data expose a larger opportunity because most possible pairs never intersect. On the pair sample used for timing, 95.1–100% of intersections were empty across eight sparse graph, census and attribute corpora. For `as-skitter`, the independent-placement estimate $|X_i||X_j|/m$ is 1.3×10^{-4}, matching the observed concentration at zero. We therefore recast the tile computation as candidate generation: identify the pairs that can share an element, and invoke an exact row cell only for those candidates.

The column orientation follows the outer-product lineage of sparse matrix multiplication[15,16] and exact wedge-based common-neighbour counting[26]. The evaluated path changes the resolution of that transpose: coarse positional buckets provide sound zero certificates, tile-word masks deduplicate their induced pairs, and the representation-pairing matrix exactly verifies the surviving bucket collisions.

The tile pipeline first orders rows by their minimum element and applies exact extent rejection. It then transposes the coarse row–bucket incidence relation into column postings, one word of tile-row identifiers per occupied positional bucket (Fig. 5a). Empty and singleton postings cannot produce a distinct pair and disappear during compaction. Each remaining posting is ORed into the candidate masks of the rows it contains, deduplicating pairs through the masks rather than by enumeration. A shared coarse bucket admits a pair but does not determine its cardinality, so every candidate is passed to the exact gated $\mathsf{B}\times\mathsf{S}$ verifier used by the postings benchmark. The online gate sends selective tiles through this postings route and sends the remaining tiles directly to the representation-pairing tournament (Fig. 5b; Methods, “Column postings for tile-level candidate generation”).

This traversal replaces exhaustive pairwise payload inspection with work on the occupied columns. Its candidate-generation work scales with the row–bucket incidences; its resolve work scales with the populations of multi-row postings; and its exact work scales with the unique candidates that survive. When every occupied column belongs to at most one row, the compact posting list is empty and the whole tile is settled without touching row data. At higher column multiplicity, candidate masks deduplicate the pair occurrences induced by multiple columns. Supplementary Note 5 gives the exact construction, soundness proof and scaling analysis.

In the amortised within-tile benchmark, column postings reduced resolve-and-verify time by 7×–5,700× relative to unfiltered $\mathsf{B}\times\mathsf{S}$ across the six sparse corpora routed through them (Table 4). The range predicate was the largest single improvement in the optimization sequence, and the multi-occupancy transpose supplied the next collective reduction. For the three denser corpora, the online route selected $\mathsf{B}\times\mathsf{B}$ with zone maps instead of forcing a sparse verifier, improving time by 4.1×–52.8×. Every successful iteration changed how much work was admitted; none made the admitted operation execute faster.

The row and column algorithms are therefore one contribution at two resolutions. Column postings control which pairs become work; the representation matrix controls the cost of the pairs that remain; and online tournaments select between the two without returning the decision to the innermost loop.

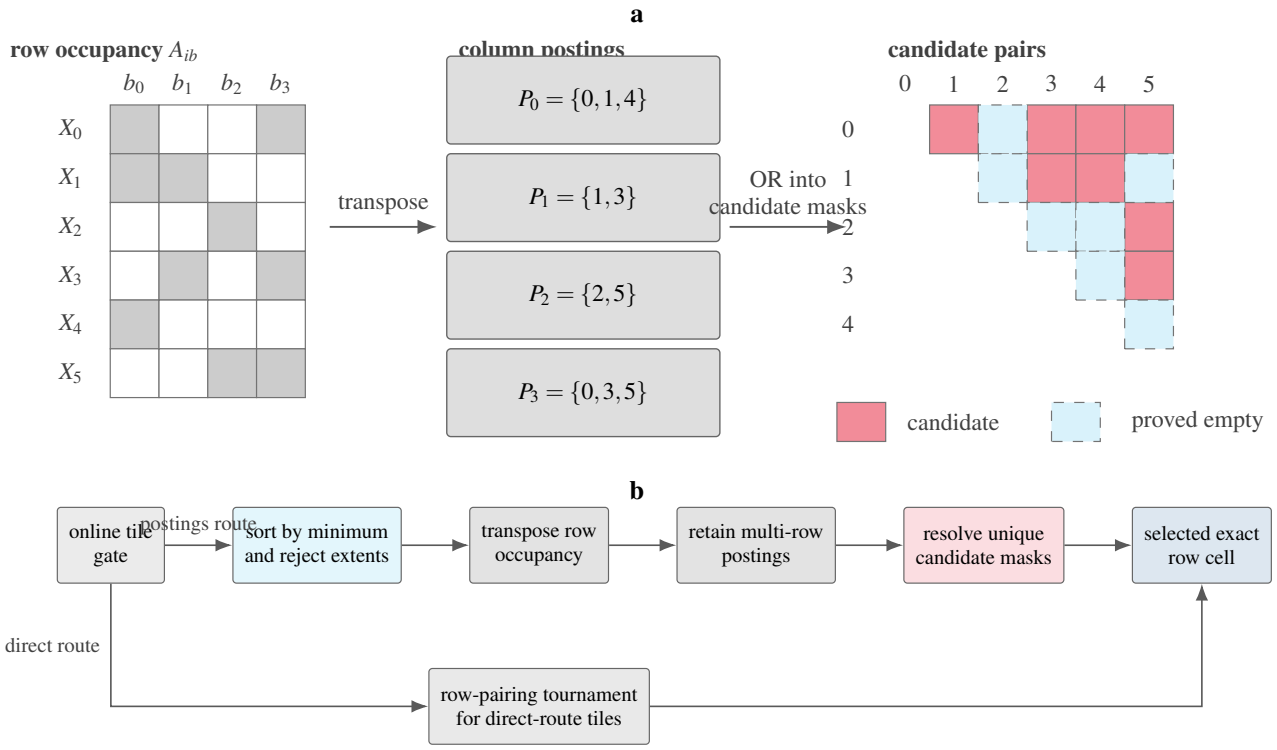

Figure 5. Column postings generate candidate row pairs once per occupied position. a, A coarse row–bucket incidence matrix is transposed into postings P_b of tile-local row identifiers. Multi-row postings contribute candidate pairs to the upper-triangular pair matrix; unshaded cells share no bucket and have exact cardinality zero. **b**, The tile pipeline applies extent rejection, compacts multi-row postings, resolves candidate masks and verifies candidates with the exact gated B×S cell. An online gate routes tiles directly to the row-cell tournament when column candidate generation is not selected. The diagram is schematic; benchmark values appear in Table 4.

Table 4. The all-pairs tile pipeline removes one to three orders of pairwise work. Times are measured nanoseconds per possible pair on Apple M4. Column-postings rows report amortised candidate resolution and exact verification after one posting construction per reused tile; the construction term and its amortisation are specified in Supplementary Eq. (S3). Zone-map rows are tiles routed to the row-oriented B×B cell. Both routes are compared with unfiltered B×S on the identical corpus. Selected times and speedups are highlighted in bold. Rows are generated from `results/ALLPAIRS.md`.

Corpus	Selected route	Unfiltered B×S (ns/pair)	Selected (ns/pair)	Speedup (×)
`dimension_003`	column postings	56.68	**0.010**	**5,700**
`soc-Pokec`	column postings	55.80	**0.098**	**570**
`com-LiveJournal`	column postings	42.60	**0.214**	**200**
`as-skitter`	column postings	42.82	**0.408**	**105**
`com-Orkut`	column postings	149.13	**2.571**	**58**
`wiki-Talk`	column postings	69.18	**9.762**	**7**
`dimension_033`	B×B zone map	2522.1	**47.8**	**52.8**
`census-income`	B×B zone map	2382.8	**323.9**	**7.4**
`weather_sept_85`	B×B zone map	4613.7	**1115.0**	**4.1**

Thresholded similarity queries turn element postings into an output-sensitive join

Many large-scale consumers request only pairs whose similarity clears a relevance, alerting or deduplication threshold. Supplying a Jaccard threshold changes which pairs belong to the requested output and exposes a stronger column algorithm. Exact set-similarity joins use inverted candidate generation, prefix and positional filtering, and exact verification[10,29–31]. Building on this filter-and-verify structure, we globally ordered elements by document frequency, placed rare elements first, indexed the threshold-determined prefix of every row in CSR postings and generated candidates only from shared prefix elements. Every candidate was verified by an exact row cell, so all reported cardinalities remained exact and the candidate result matched the exact scan at every measured point.

Mann et al. showed that the cost of similarity-join filters can exceed direct verification[32]. We therefore introduced a gate that prices prefix construction, posting multiplicity and exact row-cell verification, and bypasses the index when those terms are not favourable. Prior bitmap filtering and adaptive prefix selection provide complementary refinements within the same thresholded lineage[33,34].

The online gate activated prefix postings on 48 of 63 corpus–threshold configurations and routed the other 15 to the exact scan (Table 5). Every activated configuration improved: speedup ranged from 1.67× to 1,089×. At the largest scale, 20,000 rows expose 199,990,000 possible pairs, yet prefix postings operate on the element occurrences admitted to the prefix index and on the candidates those postings emit. Raising the threshold shortens prefixes and usually lowers column multiplicity, which explains the largest result: `com-LiveJournal` at $t = 0.1$ reaches 1,089× over enumerating and exactly testing all possible pairs.

Table 5. Element-keyed prefix postings accelerate thresholded all-pairs similarity by up to 1,089×. Each entry is gated speedup over an exact all-pairs scan on the identical rows; 1.00 denotes a gate decision to execute that exact scan. Prefix postings use a caller-supplied Jaccard threshold t, generate candidates from shared prefix elements and pass every candidate to exact verification. The CSR posting index is built once and reused by the timed query. All 63 runs returned a hit set identical to the exact baseline. Rows are generated from `results/threshold/all_corpora_full.csv`.

Corpus	Domain	Rows	Speedup over exact scan (×)		
			$t = 0.001$	$t = 0.01$	$t = 0.1$
`enwiki-categorylinks`	IR	20,000	**324**	**417**	**815**
`uscensus2000`	census	200	**40.6**	**15.3**	**28.9**
`dbpedia-link`	graph	20,000	**12.5**	**12.5**	**110**
`wikipedia_link_en`	graph	20,000	**6.38**	**14.4**	**103**
`livejournal-groupmemberships`	graph	20,000	**38.6**	**50.2**	**124**
`com-Orkut`	graph	20,000	**47.8**	**43.0**	**108**
`com-LiveJournal`	graph	20,000	**414**	**334**	**1,089**
`soc-Pokec`	graph	20,000	**225**	**247**	**362**
`as-skitter`	graph	20,000	**91.4**	**81.5**	**113**
`wiki-Talk`	graph	20,000	**3.20**	**4.72**	**12.7**
`dimension_003`	druid	15,482	**129**	**83.1**	**118**
`dimension_008`	druid	5,240	1.00	1.00	1.00
`dimension_033`	druid	173	**10.9**	**7.47**	**10.7**
`census1881`	census	200	**3.96**	**4.80**	**4.37**
`census1881_srt`	census	200	**1.67**	**1.83**	**4.03**
`wikileaks-noquotes`	text	200	**4.84**	**3.44**	**40.1**
`weather_sept_85`	sensor	200	1.00	1.00	1.00
`census-income`	census	200	1.00	1.00	1.00
`usher_sarscov2`	genomics	7,926	1.00	1.00	1.00
`gnomad_chr21_exomes_af1e-3`	genomics	20,000	**109**	**114**	**291**
`msprime_1M`	genomics-sim	917	1.00	1.00	1.00

Positional column postings and element-keyed prefix postings solve complementary contracts. The former preserves the full exact matrix and proves pairs disjoint through absent coarse buckets. The latter uses a caller's threshold to avoid generating pairs that cannot belong to the requested output. Both obtain their scale from the same systems principle: represent the column incidence once, then share its decision across all row pairs it governs.

Discussion

This study establishes an exact, two-resolution work-admission architecture for sparse set algebra, using all-pairs set-intersection cardinality as its end-to-end demonstration. At row resolution, a representation-pairing matrix matches the traversal to the effective sparsity and structure of both operands. At tile resolution, column postings transpose positional occupancy so that pairs with no shared bucket are answered as zero collectively, before an exact row verifier touches their data. These resolutions compose: column postings determine which pairs become work, and row matching determines the cost of the work that remains. The result is an exact algorithm whose dominant cost follows sparse-side descriptors, multi-row column occupancy and surviving candidates instead of following the full bitmap universe for every one of the $\binom{N}{2}$ possible pairs.

The method addresses a recurring hyperscale systems shape: large collections of heterogeneous rows over a shared identifier universe. In search and recommendation the rows can be documents, users, items or audiences; in observability they can be hosts, traces, alerts or event signatures; in graph systems they can be neighbourhoods; and in shared data and open-source infrastructure they can be records, packages, symbols or dependency sets. The downstream task may be similarity search, co-occurrence construction, correlation, deduplication or exact overlap materialisation, but the computational core is the same sparse binary Gram matrix. A method that removes fixed-universe scans and exhaustive pairwise payload inspection therefore changes capacity planning and query latency for the whole class, rather than accelerating one domain-specific pipeline.

The measured corpora exhibit exactly this regime. Sparse inputs contained 95.1–100% empty intersections, so a pairwise bitmap scan performs its fixed $\Theta(m)$ traversal mainly to rediscover zero. The fixed online row policy was 1.48×–52,448× faster than all-bitmap execution across all 23 corpora and beat query-tuned CRoaring on 22. Column postings then exploited the same structure across pairs, reducing amortised within-tile resolve-and-verify time by 7×–5,700× on the six sparse corpora routed through that path. Together, these findings identify sparse all-pairs intersection as a candidate-generation problem as much as a kernel-throughput problem.

Prior work made set intersection adaptive to its input instance through comparison algorithms and preprocessed in-memory structures[22,23]; SIMD list kernels accelerate a selected traversal[19,20]; Roaring, hybrid inverted indexes and EmptyHeaded adapt storage or operators among dense and sparse set layouts[2,24,25]; and outer-product sparse matrix multiplication or wedge expansion induces row pairs from columns[15,16,26]. Building on these methods, we developed an exact two-resolution system: query-priced selection across ten representation pairings for surviving row pairs, coupled to coarse positional postings that certify zeros collectively and resolve candidates with compact tile masks. The measurements connect that architecture to its scaling variables across 23 corpora and four instruction-set environments. The two resolutions minimize complementary terms of the execution cost: postings reduce the candidate set C, representation pairing reduces $K(i,j)$ for its survivors, and hoisting amortises planning across a tile. With routing cost included, one fixed online policy outperformed all-bitmap execution on all 23 corpora (Table 1) and query-tuned CRoaring on 22 (Table 2). Cell and posting ablations assigned these gains to their corresponding cost terms.

The row-cell results explain why the gains persist across representation families. Bitmap–bitmap work is fixed by the universe length, whereas bitmap–array, bitmap–run, array–run and related asymmetric cells can scale with the smaller cardinality, the number of runs, occupied regions or WAH fill groups. Work-avoidance strategies won all 25 asymmetric cell–shape comparisons; a same-traversal SIMD strategy won none. Dense bitmap–bitmap intersection gives the complementary case: when every word must be visited, vector popcount is the appropriate optimization and produced a tenfold improvement on Sapphire Rapids. The pairing matrix therefore identifies which resource matters in each regime rather than prescribing one microarchitectural technique everywhere. Its selection variables — cardinality, run count, extent and coarse occupancy — are descriptors of the work a cell must perform, which is why the resulting map transfers across the tested Arm and x86 hosts.

The broader technical contribution is a reusable work-admission rule. First expose metadata whose cost is smaller than the payload it summarizes. Next use that metadata either to certify a neutral contribution, to select a representation whose descriptors match the live work, or to generate only the consumers that can receive a contribution. Then hoist the decision to the largest safe scope and retain the ordinary exact kernel for survivors. Zone maps are one instance of the first step, not a set-intersection-specific trick; postings are the column-oriented, amortised form of the third. Equations (14) and (15) state the transfer condition independently of intersection syntax: the summary must soundly certify a neutral contribution, and the gate must price summary plus survivor work below the direct path. Sparse joins, graph and inverted-index traversal, block-sparse algebra and telemetry filtering all admit this construction wherever their metadata provides that certificate, even when intersection cardinality is not their final operation.

These results also separate storage adaptation from query adaptation. Roaring chooses containers to balance representation size and general operations[2,3], and EmptyHeaded selects dense and sparse layouts within compiled graph queries[7,25]; an all-pairs intersection workload additionally benefits from pricing the two representations and count-only operation presented together at query time. Applying `run_optimize()` and the measured array-to-bitset query-cost promotion made CRoaring a stronger comparator, but the online pairing policy retained its advantage on 22 of 23 corpora. The two crossover problems are consequently distinct. Online tournaments solve the query crossover with negligible measured overhead: sharing a decision

across a tile reduced selection to 0.10–0.29% of runtime, and probe-and-commit corrected a model-only misclassification on Neoverse. This makes adaptive representation matching a practical part of the all-pairs traversal rather than an oracle used only to interpret it.

Neither adaptive row matching nor column candidate generation requires an offline optimization phase. Constant-size metadata, fixed gates and bounded probe-and-commit measurements allow a short-lived or inline computation to begin immediately and adapt during its first traversal. Offline tournaments are an optional deployment specialization: for a stable workload issuing billions or trillions of comparisons, a one-time representation or bucket-width sweep can be amortised across the subsequent traversal and remove even the bounded online exploration. The distinction is operational, not algorithmic — both paths return the same exact answers and exercise the same work-avoidance mechanisms.

Thresholded similarity adds a second column-level result for systems that request only actionable pairs. Exact prefix filtering is an established set-similarity-join family[10,30–32]; here its rare-first prefixes and CSR postings changed the traversal from $\binom{N}{2}$ exact tests to the element occurrences admitted by the threshold and the candidates generated by their posting multiplicities. The online gate activated this path on 48 of 63 corpus–threshold configurations; every activation improved, by 1.67×–1,089×, and every reported pair retained its exact cardinality. Positional postings for the complete exact matrix and element postings for a thresholded result are thus two realizations of one systems principle: transpose incidence once and reuse a column decision across the row pairs it governs.

The natural next step is to carry the same two-resolution design through cross-tile panels: reuse a built posting panel across many opposing tiles, select bucket width from cardinality and clustering, and feed the resulting candidates into the full row-cell tournament. Multicore scheduling can then partition posting panels while retaining their reuse and keeping candidate masks local. These extensions deepen the same contribution rather than changing its premise. Exact sparse all-pairs intersection becomes fast when the implementation represents both axes of the incidence relation and admits work only at the resolution where that work is justified.

Methods

What makes work avoidable

This subsection is an existence argument depending on no measurement: a gap must exist between what a dense-bitmap kernel *spends* and what a cheaper but equally exact algorithm on the same operands *already achieves*, and representations whose cost is a function of structure rather than of the universe can enter it. Both compared quantities are achieved; neither lower-bounds the work the problem demands, and no such bound is claimed here. Every representation is exact and interconvertible — B, S, R, W and the complement view C encode the same set and every cell returns the same cardinality — so nothing is approximated and only the cost changes. Every quantity below is a count of stored items or of operations, never a time; the constants that convert one to the other are measured, not derived, and are given under "Benchmark protocol and cache-residency statement". Each result is stated here with its hypotheses; the proof, the worked numerical checks and the secondary propositions that support but do not themselves state a result are in Supplementary Note 8, for the results up to and including the size bound, and Supplementary Note 9, for the representation-choice optimisation problem that follows it.

Write $A, B \subseteq [0, m)$, $a = |A|$, $b = |B|$, densities $d_A = a/m$, $d_B = b/m$, word width w, vector width V words per operation. Complementation is an involution on density, $d \mapsto 1 - d$, so define the *effective sparsity* $s_X = \min(d_X, 1 - d_X) \in [0, \frac{1}{2}]$ and $s = \min(s_A, s_B)$.

What cardinality alone determines. Given only a and b — the cheapest metadata any selector holds — the answer is confined to

$$\max(0, a + b - m) \;\le\; |A \cap B| \;\le\; \min(a, b), \tag{1}$$

and every integer in the range is realised (Supplementary Note 8). Its width is the quantity the rest of this subsection turns on:

$$\min(a,b) - \max(0, a+b-m) \;=\; \min(a,\; b,\; m-a,\; m-b) \;=\; m \min(s_A, s_B) \;=\; ms. \tag{2}$$

The four terms are the sizes of the four lists one could enumerate — A, B, A^c, B^c — so the uncertainty is the size of the smallest, and one near-empty or near-full side collapses it alone. Above density one half the floor lifts off zero at $(d_A + d_B - 1)m$: at $d_A = d_B = 0.51$, two per cent of the universe must overlap.

The fixed cost, and therefore the headroom. A dense bitmap stores $\lceil m/w \rceil$ words whatever it contains, so B×B performs $\lceil m/(wV) \rceil$ vector AND-plus-popcount operations per pair, independently of a, b, of structure, and of the answer. Against that, suppose each operand X is available as a bitmap and as a sorted array of whichever of X, X^c is smaller — $\Theta(m s_X)$ of space, the

choice made from a, b, m in $O(1)$; no rank index is needed for this result. Then $|A \cap B|$ is answered in $O(ms)$ membership probes (four cases, one per term of Eq. (2); Supplementary Note 8). Counting descriptor items touched, and taking $w \mid m$,

$$\frac{\text{items touched by } \mathsf{B}\times\mathsf{B}}{\text{items touched by the cheapest list-based cell}} = \frac{\lceil m/w \rceil}{ms} = \frac{1}{ws}, \tag{3}$$

which exceeds one whenever $s < 1/w$ and grows without bound as either operand leaves density one half — the headroom, derived rather than observed, and a comparison of two *achieved* costs, not a lower bound on any algorithm's work. Two of the four cases need a complement view, which is why C is part of the design rather than an afterthought for the dense tail. Charging measured per-item costs — a random probe against a vector word — turns it into a crossover in s of **[measured crossover in effective sparsity]**.

From descriptor length to operation count. Everything else here counts *stored items*, a fact about representation size; the paper's claim is about work. One identity carries that step. Let A be a bitmap with an $O(1)$ rank index, $\mathrm{rank}_A(i) = |A \cap [0,i)|$, and B a run container with maximal runs $\{[u_i, v_i)\}_{i=1}^{r}$. The runs partition B, so

$$|A \cap B| = \sum_{i=1}^{r} \big(\mathrm{rank}_A(v_i) - \mathrm{rank}_A(u_i)\big), \tag{4}$$

in exactly $2r$ lookups — independent of m, of $|A|$, of $|B|$, and of the run lengths (Supplementary Note 8). A run of any length is priced as a run of length one, because the kernel never looks inside it; symmetrically $\mathsf{R}\times\mathsf{R}$ by merge costs $O(r_A + r_B)$ with no index. This is what makes a run count a cost parameter and not merely a storage parameter, and it is the only bridge between the two here: it reaches *operation count*, not time, since per-operation constants still differ across cells.

What randomness would predict. Let each position be set independently with probability d, and write $\kappa(\rho)$ for the number of stored items representation ρ must touch. The first two entries below are exact; the last two are the leading-order forms of exact Bernoulli expectations (Supplementary Note 8), written in $s = \min(d, 1-d)$,

$$\mathbb{E}\,\kappa(\mathsf{B}) = \left\lceil \tfrac{m}{w} \right\rceil, \quad \mathbb{E}\,\kappa(\mathsf{S}) = md, \quad \mathbb{E}\,\kappa(\mathsf{R}) = md(1-d) = ms(1-s), \quad \mathbb{E}\,\kappa(\mathsf{W}) = \tfrac{m}{w}\big[1 - (1-s)^{2w} - s^{2w}\big], \tag{5}$$

where $\kappa(\mathsf{R})$ counts maximal runs and $\kappa(\mathsf{W})$ literal words plus fill groups. None of these expectations is new. The W row is the word-aligned expected-size model of Wu, Otoo and Shoshani[1], whose bracket is identical to the one above under their payload width of $w-1$ bits per word rather than w; the symmetry about $d = \frac{1}{2}$ and the resulting incompressibility at that density are theirs as well. The R row is the classical expected run count for a uniformly random arrangement[35], and the interval bound used below is the Fréchet–Boole inequality. The density–clustering parameterisation is also due to[1]; the design space it spans has since been mapped empirically for Roaring, WAH and tree-encoded bitmaps by Lang et al.[36], who report that an uncompressed bitmap reads faster than any of the three compressed formats over $16 \le f \le 128$ and $0.01 \le d < 1$, and that they expect "a performance-optimized implementation to dominate an even larger space" than their unoptimized baseline did. That is the headroom this paper measures, stated from the compressed side. They are restated here in one notation because the pairing matrix needs them as inputs, not because they are results of this paper. With the complement view making the best of S and $\mathsf{C}\circ\mathsf{S}$ cost ms, every entry is constant in s or strictly increasing on $[0, \frac{1}{2}]$ and symmetric about $d = \frac{1}{2}$: under this null model the U-shaped cost curve is a theorem — about the model and the bitmap's flatness, not about any method. Randomness gives run containers nothing either, since $\frac{1}{2}ms \le \mathbb{E}\,\kappa(\mathsf{R}) \le ms + 1$, so choosing *among* $\{\mathsf{S}, \mathsf{C}\circ\mathsf{S}, \mathsf{R}, \mathsf{W}\}$ buys at most a constant on structureless data. B is the exception and the exception is the point: $\kappa(\mathsf{B})$ is $\lceil m/w \rceil$ at every density, so the spread across the full matrix is Eq. (3) and unbounded. It is the envelope, not each cell, that follows $\Theta(\min(m/w, ms))$.

The deviation, which is the contribution. Real data is not random, and the representations part company with Eq. (5) in a way density cannot express. At fixed density $d \le \frac{1}{2}$ with $k = dm$, $\kappa(\mathsf{B})$ and $\kappa(\mathsf{S})$ are constants, while $\kappa(\mathsf{R})$ ranges over every integer in $[1, k]$ and $\kappa(\mathsf{W})$ from $O(1)$ to $\Theta(\min(k, m/w))$ (Theorem B, proved by explicit families in Supplementary Note 8). Writing $\sigma(\rho, X)$ for predicted descriptor length at X's density over actual, σ is identically one for B and S, at least $\max(d, 1-d+\frac{1}{m}) \ge \frac{1}{2}$ for R, and unbounded above: real rows can be arbitrarily cheaper than random rows of the same density and at most about twice as expensive.

One impossibility follows, exactly. A selector reading density — or cardinality, or any function of them — evaluates one fixed function on every row of that density, so by the range above it mis-predicts $\kappa(\mathsf{R})$ by an unbounded factor on some rows at every density; a fixed representation cannot exploit σ either, having committed before σ is knowable. That rules out a family of selectors without choosing among the survivors, and two further readings do not follow: whether a mis-predicting model costs more than selecting nothing depends on which cell it picks and that cell's actual cost, which is measured (Fig. 4b), not derived; and a run or occupancy count is $O(1)$ metadata that sees σ perfectly and could feed a model. What is proved is that selection must be gated on a statistic that sees structure rather than on cardinality — structural summary or direct kernel measurement is settled below, by measurement.

The null model is a floor, and it is the same floor three times. Independent placement at fixed density is the *extremal* arrangement for every mechanism used here, always in the same direction (proofs in Supplementary Note 8). For a representation, Eq. (5) sits within a factor of two of the worst achievable descriptor. For a zone map of bin width W_z, scattering maximises the occupied-bin fraction, so the summary's null cost is an upper bound on its cost and a lower bound on its value. For the prefix filter of the narrowed-contract mode, the entire mechanism depends on the operands through one dimensionless quantity, and any ordering that concentrates rare elements into prefixes makes real prefixes collide less often than scattered ones of the same length. In each case the null is what the mechanism is worth on data with no structure to find, real data deviates from it only in the mechanism's favour, and that deviation is exactly the quantity the mechanism converts into avoided work.

The mid-density regime selects bitmap throughput. At $s = \frac{1}{2}$ the interval has width $m/2$ and excludes nothing, the cheapest list-based cell spends $m/2$ probes against $\lceil m/w \rceil$ words, and Eq. (5) offers neither R nor W an advantage: Eq. (3) falls below one. In the generic case the mid-density band belongs to B×B, and a selector choosing it there is correct rather than defeated. This bounds what cardinality determines absent structure, not what any kernel must spend — call it a *metadata* or *identifiability floor*, never an information-theoretic one — and a structured row at exactly this density is a counterexample to the unqualified form (Supplementary Note 8). It is also why the zone map of the next paragraph matters in the one band where cardinality is useless.

Why the two mechanisms are selected between rather than stacked. Composing a zone map with a representation is not the product of the two, because filtering *concentrates* what it passes on: conditioned on a bin of width W_z being occupied its expected local density is d/p, $p = 1-(1-d)^{W_z}$, not d — no summary hands a kernel anything sparser than one bit per bin (Supplementary Note 8). Charging the summary pass c/W_z of a bitmap scan and taking the operands independent, the composed cost relative to a plain bitmap is

$$\frac{c}{W_z} + p_A\, p_B\, \kappa\!\left(\frac{d}{p}\right), \tag{6}$$

$\kappa \le 1$ being the envelope of Eq. (5) normalised against the bitmap. The composition never costs more than the summary alone, yet is floored at c/W_z, below which a sparse representation is cheaper outright — three regimes, the middle one exactly $xe^{-x} > c/w$ in $x = W_z d$ bits per bin, opening where the summary undercuts enumeration and closing where bins stop being empty. This is the analytic form of the gating result of Fig. 4c, and it models *work*: it bounds where a filter has work to remove and says nothing about what one costs when it removes nothing.

Each mechanism's liability is two-sided, for unrelated reasons at the two ends. A summary is indexed by the universe while every compressed representation is indexed by the data, so their costs must cross. Charging the summary c/W_z of a bitmap scan as in Eq. (6), and a one-word-per-element representation wd, the low crossing is

$$d_{\text{low}} = \frac{c}{wW_z} = 3.1\times10^{-5} \qquad (c=1,\ w=64,\ W_z=512), \tag{7}$$

below which reading the summary costs more than enumerating the operands outright (Supplementary Note 5). At the other end $p_A p_B \to 1$: no bin is empty, the summary proves nothing, and only its own c/W_z remains. A run- or fill-based representation crosses at its own density, wherever its descriptor length falls below m/W_z, but that a crossing exists at all is generic. Prefix filtering has the same shape in $y = p^2/m$: for $y \ll 1$ the surviving fraction is y itself, so pruning improves quadratically as prefixes shorten, while for $y > 1$ prefixes collide by default and the index earns nothing. This is why no mechanism here is applied unconditionally, and why each gate is a two-sided test rather than a threshold.

A narrowed contract, and what two integers then settle. Every statement above returns the cardinality itself. A caller may instead ask a narrower question — report only the pairs with $J(A,B) \ge t$, or only those with $|A\cap B| \ge \tau$, or restrict attention to operands whose size falls in a band — and the narrowing is itself exploitable, because Eq. (1) already says what sizes permit:

$$J(A,B) = \frac{|A\cap B|}{|A\cup B|} \le \frac{\min(a,b)}{\max(a,b)}, \tag{8}$$

so $J(A,B) \ge t$ requires $\min(a,b) \ge t\,\max(a,b)$, and $|A\cap B| \ge \tau$ requires $\min(a,b) \ge \tau$ (proof in Supplementary Note 8). The hypotheses are that $a,b>0$ and $t \in (0,1]$; nothing is assumed about arrangement, about m, or about how either operand is stored. Both conditions are necessary and neither is sufficient, so the test discards pairs and never reports one — the survivors still go to an exact kernel, at the cost of one comparison between two integers already held as metadata, no element of either set read. This is the Swamidass–Baldi pruning bound[12] together with the size (length) filter of[30], in the two-sided form standardised by[32] and applied jointly with prefix filtering to Tanimoto over binary vectors by[37], restated here to place it against the mechanisms above rather than as a result of this paper. The operational narrowing and the complete thresholded construction are given in Supplementary Notes 2 and 3.

What that removes, and why a wider corpus helps it. Modelling a corpus by drawing cardinalities independently with $\log a$ uniform on $[0, \ln K]$ — the density $\Pr(a) \propto 1/a$, exactly the $1/i$ cardinality spectrum this paper's benchmark protocol mandates ("Corpora, provenance and loaders"), not an approximation to it — $\min(a,b)/\max(a,b) \geq t$ becomes $|\log a - \log b| \leq \gamma$, $\gamma = \ln(1/t)$, and for two independent uniforms on an interval of length $L = \ln K$,

$$\Pr\left[\text{pair can qualify}\right] \;=\; 1 - \left(1 - \frac{\ln(1/t)}{\ln K}\right)^2, \qquad \ln(1/t) \leq \ln K, \tag{9}$$

with the absolute form $\left(1 - \ln\tau/\ln K\right)^2$ (proof and worked values in Supplementary Note 5, which agree with a 10^5-sample simulation to three decimals). Expanding for small γ gives $2\gamma\int\phi^2$ for any density ϕ of $\log a$, so the governing quantity is the *effective log-spread* $1/\int\phi^2$ of the size distribution, of which Eq. (9) is the log-uniform case. Two hypotheses carry real weight. Cardinalities are integers, and $a = b$ passes at every t, so the continuous form is optimistic where sizes are small. And the favourable scaling in K — Eq. (9) decreases monotonically in K — is a property of the log-uniform family and not of heavy tails generally: under the alternative reading in which *ranked* sizes scale as $1/i$, the surviving fraction is exactly $1 - t$, independent of K, since the effective log-spread saturates at two nats however far the sizes are spread. The favourable scaling in K is therefore claimed for the spectrum this paper benchmarks, not for every skewed corpus.

The size bound does not multiply with the mechanisms below it. Acting at a coarser granularity is not sufficient for savings to compose. The size test decides whether a pair is formed at all, while representation choice and the zone map act inside a pair that is formed, so the granularity argument of Eq. (6) predicts a product — and the product is not achieved, because both mechanisms read the same statistic. The survivors of Eq. (8) are the pairs with $a \approx b$, and every mechanism below is cheapest exactly when a and b are *dissimilar*, since its cost tracks $\min(a,b)$ against a fixed $\lceil m/w \rceil$. Conditioning raises the mean cost of a survivor by

$$\frac{\mathbb{E}\left[\min(a,b) \;\middle|\; \min/\max \geq t\right]}{\mathbb{E}\,\min(a,b)} \;=\; \frac{L^2\big(K(1-t) - \gamma\big)}{\gamma(2L-\gamma)\,(K-L-1)} \;\xrightarrow[t\to 1]{}\; \frac{\ln K}{2}, \tag{10}$$

and the composed work exceeds the product of the two savings by that same factor; the derivation, the numerical verification and the decorrelation control that isolates the shared statistic as the cause are in Supplementary Note 8. The composition is still strongly worth taking. What is refuted is only the product; the general rule is that mechanisms compose multiplicatively when they act on different waste *and* are driven by different operand statistics, and sharing a statistic costs the composition Eq. (10)'s factor even across granularities. Values here are analytic, computed over the model above at $m = 10^6$, and count work rather than time.

Choosing a representation is a different problem from choosing a size, and only one of them is separable. Everything above prices representations; a deployed library must *choose* between them, and the choice has to be stated as an optimisation before it can be called optimal. Partition the universe into chunks of M bits, each stored in one representation, $n = M/w$ words to a chunk and e bits to a stored array element. Given a corpus, a distribution π over the chunk pairs an application actually evaluates, and an operation ω, the problem is to choose an assignment ρ of representations to chunks minimising $\sum_{(x,y)\sim\pi} C_\omega(\rho(x), \mu_x; \rho(y), \mu_y)$ subject to a byte budget, where μ is the metadata of Eq. (5) and C_ω counts items touched at measured per-item constants. Storage is a *separable* objective — minimised one container at a time — whereas the query cost is quadratic in the assignment, since there is no such thing as the cost of a container, only the cost of a pair (Supplementary Note 9). And the incumbent's three container decisions — array against bitset, array against run, bitset against run — are one rule, $\arg\min$ over serialized size, so its array-to-bitset threshold is $\tau_{\text{stor}} = M/e$, a property of the format carrying no machine constant and no property of the workload: the exact optimum of a separable objective, and why it can be a compile-time constant. What follows is what changes when the objective is not separable.

The crossover is a formula, and the two constants in the field are its two limits. Write c_w for a vectorised word AND-plus-popcount, c_p for a membership probe into a bitmap and c_m for a merge step, and $\theta_{pw} = c_p/c_w$, $\theta_{mw} = c_m/c_w$. Promotion of a chunk of cardinality k from array to bitset is favourable against a bitmap partner exactly above

$$\tau_\cap \;=\; \frac{c_w}{c_p}\cdot\frac{M}{w} \;=\; \frac{n}{\theta_{pw}}, \qquad \text{whence} \qquad \frac{\tau_{\text{stor}}}{\tau_\cap} \;=\; \frac{w}{e}\,\theta_{pw}, \tag{11}$$

and against an array partner above $(\theta_{pw}/\theta_{mw} - 1)\,k_Y$ for a partner of size k_Y (proof in Supplementary Note 9). Pricing a byte at $\lambda \geq 0$ and minimising query cost plus λ bytes gives a threshold that runs from $\tau_\cap$ at $\lambda = 0$ to τ_{stor} as $\lambda \to \infty$: every value between the two is optimal at some price of a byte, and $\tau_{\text{stor}} = M/e$ is the unique threshold at which promotion is never paid for in bytes. The threshold also carries the pair count, so under any byte budget the optimum is not a function of the container at all.

The measured input is the symmetric crossover **[cardinality at which an array–array merge stops beating a bitmap–bitmap popcount]**, which fixes θ_{mw}; substituted into Eq. (11) at $M = 2^{16}$, $w = 64$, $e = 16$ together with **[measured ratio of a bitmap probe to a merge step]** it gives **[the ratio of the two thresholds]**, recovering the divergence between the two objectives from measurements of the constants and no free parameters. Both crossovers share c_w, so measuring them together over-determines the model and the probe-to-merge ratio becomes a checkable prediction rather than an input.

A fixed threshold is optimal, under five hypotheses, and each of them fails. The loose form is false, so state the hypotheses first: if the candidates are only S and B, the operation is a count-only intersection, the per-item constants do not depend on the resident footprint, there is no byte budget, and every chunk faces the same partner mix, then the optimal assignment *is* the fixed threshold $\tau_\cap$ of Eq. (11) — a machine constant, independent of the corpus. Dropping any one hypothesis breaks it, with an exact witness for each (Supplementary Note 9): admitting R reintroduces the impossibility above, specialised to the container decision, since R×B costs a strictly increasing function of run count while S×B does not depend on it; letting the partner mix vary makes the optimal threshold

$$\tau^*(\beta,\bar{k}) \;=\; \frac{\beta\, c_w n + (1-\beta)(c_p - c_m)\,\bar{k}}{\beta\, c_p + (1-\beta)\, c_m}, \tag{12}$$

where β is the pair-weighted fraction of a chunk's partners that are bitmaps and $\bar{k}$ the mean cardinality of the array partners: a statistic of the workload rather than of the machine, and self-referential — an optimal fixed threshold is a fixed point $\tau = \tau^*(\beta(\tau),\bar{k}(\tau))$, which exists but need not be unique; admitting a byte budget makes the threshold its dual variable, sweeping the whole interval above; changing the operation flips the matrix's sign, since cardinality queries have no operation dimension at all (union, difference and symmetric difference are inclusion–exclusion over the intersection) but a *materialising* union with a bitmap operand costs n words whatever the other side is, so no single stored assignment serves a mixed workload. The fifth hypothesis — that the per-item constants do not depend on footprint — is the one that is not analytic and the one the measurements say dominates: c_w is not a constant, footprint re-enters through it, and a work model that prices the choice cannot price its consequence, the same divergence recorded under Eq. (6) and treated as a measurement question, not an analytic one.

Metadata settles the decision wherever the decision matters. Two identifiability questions must not be merged. Eq. (2) says what cardinality determines about the *answer*, pinned only at the density extremes. What cardinality determines about the *decision* is different and more favourable: every cost in this paragraph and the two before it is a closed form in $(k, r, \ell + g)$ and the constants, so a selector holding those three integers per operand evaluates the ordering exactly, in $O(1)$, touching no operand data — which is what "selection must be near-free" means, and a consequence of the cost model rather than an engineering aspiration (proof in Supplementary Note 9). Holding only k, the ordering is not determined once R is a candidate; one extra integer restores it and more cardinality precision does not. Suppose the constants are known within a factor $\eta \geq 1$ — the uncertainty of the modelled costs, distinct from the γ of the size bound above — and let Λ be the ratio of the two cheapest modelled costs: if $\Lambda > \eta$ the ordering is determined; if $\Lambda \leq \eta$ it is not, and choosing the second-best costs at most η. So for every pair, either the metadata settles the decision or the decision does not matter to within the precision of the constants, and the undetermined set is an η-neighbourhood of the boundaries rather than a region of unbounded error. The caveat that makes η interesting: it is the uncertainty of the *model*, and where the model omits the memory system it is not small — the derived reason to measure a candidate rather than only to model it.

Why an advantage survives repairing the kernel beneath it. A pairwise batch costs C_{meta} plus the cell cost summed over the pairs a summary does not discard and the regions it does not exclude; those sets are functions of cardinality, bin occupancy and the data, and of nothing else, so no change of representation moves either set (Supplementary Note 9). Consequently, if a better kernel divides every cell cost by α, the advantage over the same batch without avoidance is invariant in α when the summary is free and erodes only through the summary's share of the surviving cost: improving a kernel cannot subsume avoidance, because avoidance changes which work is asked for and a kernel changes only what asked-for work costs. Which savings multiply is settled by the same rule as Eq. (10): a zone map reads bin occupancy while representation choice reads cardinality and run count, so those multiply; the size bound reads cardinality, which representation choice also reads, so those do not, and a container-threshold gain must not be multiplied into a size-bound gain. Quantitatively, against an incumbent already holding the compute-optimal threshold of Eq. (11), the modelled advantage of adding a zone map of bin width W_z peaks at

$$G_{\max} \;=\; \frac{\theta_{pw}\, w}{2\sqrt{c\, W_z}} \qquad \text{at} \qquad d^* = \frac{\sqrt{c}}{W_z^{3/2}}, \tag{13}$$

valid on $(\theta_{pw} w \sqrt{c})^{2/3} \leq W_z \leq \theta_{pw} w$ and saturating at $\sqrt{\theta_{pw} w / c}\,/2$ above it, and falling below one outside a window whose lower edge is Eq. (7) with the per-item constants carried, $c/(\theta_{pw} w W_z)$ (proof in Supplementary Note 6). Eq. (13) is a *null-model*

ceiling on structureless data, and by the one-sidedness of σ above, real corpora can only fall below the null, so a measured advantage exceeding it is evidence of structure and must be reported as such rather than as agreement. And Eq. (13) excludes the narrowed contract entirely: Eq. (9) supplies its own factor, and by the rule above the two are not multiplied.

The irreducible regime selects throughput. The mid-band above is not only where these results stop applying; it is where a different optimization applies, and the two are complementary rather than competing. Where Eq. (3) falls below one and $p_A p_B \to 1$, the work $\mathsf{B}\times\mathsf{B}$ performs is, in the generic case, work that is going to be performed, and what remains to optimize is the per-cycle throughput of its $\lceil m/(wV)\rceil$ vector `AND`-plus-popcount operations — a quantity none of the results above speaks to, and one the popcount literature has already driven close to the hardware limit[18]. Eq. (3) and Eq. (6) locate the boundary between the two domains; they do not rank them.

Representations and the per-cell cost model

Each row — one set X_i over a shared universe of m bits — is stored in one of several representations, each with a distinct asymptotic cost for computing an intersection *cardinality*, $|X_i \cap X_j|$, rather than the intersection itself. We define each representation below, together with the cost of the cell it forms when paired with itself or with another representation; every cost-model symbol used elsewhere in this paper is defined here and only here.

Four representations form the symmetric pairing matrix. A dense bitmap (B) stores m bits directly, one per universe position, and costs $\Theta(m)$ to intersect against another dense bitmap by AND and popcount, independent of either side's cardinality. A sorted array (S) stores the set's $|X|$ set positions as sorted integers and costs $\Theta(|X|)$ against a bitmap, since each element is resolved by a single random probe. A run container (R) stores the set as r maximal runs (a,b) of contiguous set bits and, against a bitmap fitted with a rank index (below), costs $\Theta(r)$ — independent of run length. A fill-compressed word stream (W, an EWAH-style encoding of literal words interleaved with run-length-coded fill words) costs $\Theta(\ell+g)$ against a bitmap, where ℓ is the literal-word count and g the fill-group count, and its fills are skipped in constant time using the same rank index used for R. A fifth view, the complement (C), does not add a representation to the symmetric matrix; it is a strategy, described in the next subsection, for reusing whichever of B, S, Ror $\mathsf{W}$is cheapest on a row's complement when that row sits above density one-half.

The cost of each symmetric cell is, in the notation above: $\mathsf{B}\times\mathsf{B} = \Theta(m)$ (naive; made sub-linear by the zone-map filter below); $\mathsf{B}\times\mathsf{S} = \Theta(|S|)$; $\mathsf{B}\times\mathsf{R} = \Theta(r)$ with a rank index; $\mathsf{B}\times\mathsf{W} = \Theta(\ell+g)$; $\mathsf{S}\times\mathsf{S} = \Theta(|A|+|B|)$ by merge or $\Theta(|A|\log(|B|/|A|))$ by galloping search when $|A| \ll |B|$; $\mathsf{S}\times\mathsf{R} = \Theta(|S|+r)$ by merge or $\Theta(|S|\log r)$ by binary search into runs; $\mathsf{S}\times\mathsf{W} = \Theta(|S|+\ell+g)$; $\mathsf{R}\times\mathsf{R} = \Theta(r_A+r_B)$ by run merge; $\mathsf{R}\times\mathsf{W} = \Theta(r_A+(\ell+g)_B)$; and $\mathsf{W}\times\mathsf{W} = \Theta((\ell+g)_A+(\ell+g)_B)$. Every cell except $\mathsf{B}\times\mathsf{B}$ is sub-linear in the universe size m whenever their descriptor counts are $o(m)$. More generally, every non-bitmap cell follows one or more stored descriptors rather than paying a fixed universe scan. The formal statement of what these costs imply — that the null model is a floor rather than a prediction, and the exact conditions under which no representation beats a bitmap — is given with full proofs in Supplementary Note 8.

Prior work established adaptive merge, search and galloping algorithms for sorted sets and inverted indexes[22–24], together with SIMD implementations for selected list traversals[19,20]. Building on these kernels, the pairing matrix schedules each row pair to the exact cell with the lowest predicted descriptor cost.

The pairing matrix and its cells

The paper reports the ten cells of the symmetric representation matrix formed from {B, S, R, W}: $\mathsf{B}\times\mathsf{B}$, $\mathsf{B}\times\mathsf{S}$, $\mathsf{B}\times\mathsf{R}$, $\mathsf{B}\times\mathsf{W}$, $\mathsf{S}\times\mathsf{S}$, $\mathsf{S}\times\mathsf{R}$, $\mathsf{S}\times\mathsf{W}$, $\mathsf{R}\times\mathsf{R}$, $\mathsf{R}\times\mathsf{W}$, $\mathsf{W}\times\mathsf{W}$. The complement strategy, $\mathsf{C}\times\mathsf{B}$, is an eleventh, asymmetric strategy rather than a twelfth symmetric cell: when a row's density places it above one-half, its complement is computed and intersected using whichever of the ten cells above is cheapest for that complement, and the requested cardinality is recovered by inclusion–exclusion, $|A\cap B| = |B| - |A^c\cap B|$, where A^c is A's complement against the shared universe. This convention is used consistently throughout: any count of "ten" cells in this paper refers to the symmetric matrix over {B, S, R, W}, and $\mathsf{C}\times\mathsf{B}$ is reported separately as a strategy rather than folded into that count. Roaring is never constructed as a cell of this matrix; it is used throughout as an external baseline only (below).

Each cell is implemented independently against a shared representation layer providing one view per format (bitmap, sorted list, run, EWAH-fill and complement). Several of the asymmetric cells have more than one implemented strategy — for example $\mathsf{B}\times\mathsf{R}$ has a rank-indexed variant and a scalar per-run-scan variant — and each variant was raced against the independent oracle described below before being admitted to any comparison. The variant reported for a given cell in Results is the one selected for the corresponding density regime or corpus shape by the stated tournament. The complete variant and optimization record is retained with the benchmark results.

Zone-map and rank-index construction

Two mechanisms let a kernel decide how much of a row it needs to visit, rather than visiting all of it faster. To decide how much of a dense row a pairing needs to scan, we used a *zone map*, a bitmap of coarser granularity in which bit k records whether bin k

of the underlying row — a contiguous block of 512 bits — contains any set bit. To answer a run's or a fill's contribution to a bitmap without touching the bits it spans, we used a *rank (prefix-popcount) index*, following the rank/select lineage of Jacobson, Clark and Vigna's `rank9`[38–40].

The zone map is built by a single forward pass over the row, or is free at query time when the underlying storage already carries block-occupancy metadata, as columnar formats commonly do; at one bit per 512-bit bin its storage cost is 0.195% of the bitmap it summarizes. For a pair, the bitwise AND of the two rows' zone maps, $\mathrm{occ}_A \wedge \mathrm{occ}_B$, is itself a bitmap of $m/512$ bits, and its popcount is the *exact* number of bins the pairing needs to visit, not an estimate: a popcount of zero proves the two rows disjoint without visiting either. The rank index is built by a single forward prefix-popcount pass and stores two 64-bit words per 512-bit block, a 25% overhead relative to the bitmap; for a run or fill spanning positions $[a,b)$ on the indexed side, its contribution to the intersection is answered in constant time as $\mathrm{rank}_B(b) - \mathrm{rank}_B(a)$, the number of set bits in B up to b minus the number up to a — this formula is used, unchanged, by every cell that pairs a run- or fill-based representation against an indexed bitmap. The absolute prefix occupies the first word of each block; seven nine-bit within-block prefixes are packed into the second.

Auxiliary construction is cardinality-gated. The row builder constructs occupancy only when $512|X| > m$, the point at which sparse-side work can amortise an $m/512$ summary, and constructs rank only when the mean run length is at least 512 bits. Rows that do not satisfy a gate carry a null index pointer and execute the corresponding exact index-free cell. Supplementary Note 7 quantifies the resulting footprint and gating rationale.

Neither mechanism is applied unconditionally. Within the row-wise pairing matrix, the fixed-width zone map is consulted ahead of cells that may otherwise scan a dense representation in full ($\mathsf{B}\times\mathsf{B}$ and $\mathsf{B}\times\mathsf{W}$). A separately gated, coarser occupancy map is also evaluated ahead of $\mathsf{B}\times\mathsf{S}$: it cannot avoid traversing the sparse list, but it can reject a list element before the corresponding scattered bitmap probe. The same coarse map is the input to the column-posting algorithm below. No occupancy map is put unconditionally in front of $\mathsf{B}\times\mathsf{S}$ or $\mathsf{B}\times\mathsf{R}$; when most bins are occupied, the summary removes no probes and its own lookup is pure overhead. The online and offline policies used to decide whether to construct and consult it are described below and in Supplementary Note 5.

Relation to block-skipping indexes. Summarising a block so that a query can skip it is long established, and the zone map used here is a re-parameterisation of that idea rather than a new mechanism. Small materialised aggregates keep one aggregate — typically min and max — per bucket[41]; column imprints keep a bit vector per cache line over a sampled value-domain histogram[42], vectorised in later work[43]; PostgreSQL's block range index keeps an opclass-defined summary per page range[44]; adaptive range filters keep one occupancy bit per leaf of a trie over the key domain[45]; and Hippo keeps a bitmap over histogram buckets per dynamically sized page range[46]. Closer to this setting, hierarchical bitmap indexes summarise groups of positions for set-valued attributes[47], BitFunnel's higher-rank rows OR together fixed powers-of-two blocks of a signature[5], WAH and EWAH fill words already encode that a region is uniformly empty[48], and Lucene's sparse-block bitset stores one occupancy bit per word within a block[6]. Roaring itself summarises at container granularity: an absent key certifies an empty 2^{16}-bit chunk, and intersection already skips keys present on only one side[49].

Two things differ here, and neither is the summary itself. First, every mechanism above is *one-sided*: one stored summary is matched against a query predicate. The test used here is *two-sided* — the summaries of both operands are combined, $\mathrm{occ}_A \wedge \mathrm{occ}_B$ — and its popcount is the exact number of bins that can contribute to this pair. A bin absent from either operand is rejected exactly; a bin occupied on both sides is visited by the cardinality kernel. Second, the bins partition *bit position* rather than a value domain, so no histogram is sampled, no bin boundary is learned, and nothing is rebalanced when the data changes — the machinery those designs spend on placing bin boundaries has no counterpart here. What is specific to this setting is therefore the pairwise composition and its exactness, not the occupancy summary, which we take as established.

The cost model for the pairwise filter, the crossings that bound where it pays, and the derivation of its bin width are given in Supplementary Note 4.

A reusable work-avoidance construction

The implementation separates work avoidance into four roles that do not depend on set-intersection syntax: a summary σ cheaper than the payload it describes, a sound exclusion predicate E over summaries, an exact survivor kernel K, and a gate that chooses between the summarized and direct routes. For an operation F with neutral contribution e, correctness requires

$$E(\sigma(x),\sigma(y)) = 1 \quad \Longrightarrow \quad F(x,y) = e. \tag{14}$$

The exclusion predicate may admit false-positive work — those cases continue to K — but it cannot reject a non-neutral contribution. If p is the surviving fraction, the summarized route is selected when its estimated work satisfies

$$C_{\text{summary}} + pC_K < C_{\text{direct}}. \tag{15}$$

Construction or selection shared by q consumers enters C_{summary} at its amortised cost per consumer. Equations (14) and (15) are the application-independent contract: define what metadata can prove, preserve an exact path for survivors, and route online from the quantities that price both sides.

The mechanisms evaluated here are concrete instances of that contract. A zone-map AND certifies that a positional region contributes zero; the row kernel evaluates occupied regions. A rank index replaces traversal of every position spanned by a run with two prefix queries. Positional postings hoist the same occupancy certificate across a tile, while element postings hoist a threshold-dependent candidate decision across every row containing that element. In sparse joins, block-sparse algebra, graph traversal, inverted-index evaluation or telemetry filtering, σ, E and K change with the operator, but the construction and its scaling criterion do not. Supplementary Note 5 gives the instantiation procedure and shows how the row and column mechanisms map onto it.

Column postings for tile-level candidate generation

The row-wise cells above answer one pair at a time. The column-posting path changes the traversal order for a tile of T rows while leaving both the query and the exact row kernels unchanged. Let $\Delta = 2^q$ be the positional bucket width and $f = \lceil m/\Delta \rceil$ the number of buckets, and define

$$A_{ib} = \mathbf{1}\{X_i \cap [b\Delta, (b+1)\Delta) \neq \varnothing\}, \qquad 0 \leq b < f, \tag{16}$$

so that a bucket is obtained by a shift. The row-oriented occupancy maps are the rows of A. Transposition gives one *column posting* per bucket, $P_b = \{i : A_{ib} = 1\}$. With the evaluated tile width $T = 64$, each P_b is one 64-bit word whose set bits are tile-local row identifiers. This is a temporary column representation of the coarse occupancy relation, not another representation of the original set elements and not the prefix index used for thresholded queries.

This column view is an outer-product evaluation of the Boolean pattern of AA^{T}, a classical sparse-matrix multiplication organization[15,16]. Element-level wedge expansion similarly counts common neighbours for all vertex pairs without performing pairwise set intersections[26]. Building on this column view, we combine coarse positional buckets, one-word tile postings and bit-mask deduplication with an exact representation-pair verifier. Coarse columns certify zeros collectively, while surviving collisions are resolved by the row cell selected for their operand structure.

Rows were sorted within each tile by their first set position. For row i, candidate rows were first restricted to those whose exact extent $[\min X_j, \max X_j]$ overlapped that of i; because the minima are ordered, the scan stops at the first j with $\min X_j > \max X_i$. An early adaptive cascade built the posting stage only when this range pass left more than one quarter of the tile's pairs. The later amortised variant separated construction from resolve and prebuilt each tile's postings once. Construction walked the set bits of each row occupancy map and set bit i in the corresponding P_b. Postings with $|P_b| < 2$ were discarded because they cannot produce a distinct row pair. The surviving postings were retained as a compact list, and their union supplied a touched-row mask so that candidate state was cleared and scanned only for rows participating in at least one shared bucket.

For every retained posting, the implementation ORed P_b into the candidate mask of every row $i \in P_b$. Thus

$$C_i = \bigcup_{b:\, i \in P_b} P_b \tag{17}$$

Equation (17) is formed without enumerating each bucket-induced pair separately. Self pairs and the lower triangle were masked out, and the resulting extent-overlap condition was applied to C_i. Every surviving (i, j) was finally evaluated with the exact gated $\mathsf{B}\times\mathsf{S}$ verifier used by the postings benchmark; tiles rejected by the postings gate use the full row-cell tournament. Exact verification is required because two rows can occupy the same coarse bucket through different elements. Conversely, the filter has no false negatives. If $x \in X_i \cap X_j$, then both rows occur in $P_{\lfloor x/\Delta \rfloor}$, so $j \in C_i$ before the triangular mask. The postings therefore generate a superset of the non-empty pairs; they never replace the exact cardinality kernel.

An adaptive cascade selected the tile route from a strided sample. It estimated (i) *survival*, the fraction of sparse-side element probes whose coarse bucket was occupied on the other side, and (ii) *touch*, the number of such probes divided by the number of 128-byte cache lines in the probed bitmap rows. The research harness entered the filtered path when survival was below 0.15 and touch below 0.25 and enabled the range pass only when it rejected at least 5% of pairs in the first tile. The conditional-build cascade used a 25% range-survival threshold; the amortised multi-posting variant instead prebuilt the compact lists. These values are study-tuned settings; the touch statistic uses the target's measured 128-byte cache line. A gate rejection routes the tile to the row-wise tournament below.

The postings benchmark measures the $T(T-1)/2 = 2{,}016$ within-tile pairs of each 64-row tile. The transpose and compact posting list are constructed once and reused by the timed candidate-resolution and exact-verification phase, matching the amortised block-traversal schedule. Bypass tiles are timed under the row-wise tournament. Table 4 reports these two routes separately. Construction, candidate resolution and verification, including their scaling regimes, are derived in Supplementary Note 5.

Element-keyed prefix postings for thresholded similarity

The thresholded mode accepts a caller-supplied Jaccard cutoff $t \in (0,1]$ and returns exact cardinalities only for pairs satisfying $J(X_i,X_j) \geq t$. It uses the standard prefix-filtering condition developed for exact set-similarity joins[10,11,29–31]: after applying one global total order to the elements, row X_i contributes a prefix of length

$$p_i = |X_i| - \lceil t|X_i| \rceil + 1. \tag{18}$$

Any pair meeting the threshold shares at least one element between these prefixes. The implementation orders elements by increasing document frequency, breaking ties by element identifier. This rarest-first order shortens the postings traversed during candidate generation; correctness requires only that every row use the same total order.

Wang et al. selected prefix lengths adaptively per object[34], Sandes et al. introduced a bitmap candidate filter[33], and Mann et al. showed that filter cost can exceed direct verification[32]. We couple the prefix index to the exact representation-pair verifiers used by the complete-matrix path and admit it only when Eq. (19) prices construction, posting multiplicity and verification below an exhaustive traversal.

The prefix index is a compressed sparse row (CSR) representation keyed by dense element rank. A first pass counts the rows containing each element in their prefix, a prefix sum converts counts into offsets, and a second pass writes row identifiers into one contiguous posting array. Rows are visited in increasing cardinality order. For row i, the implementation traverses the postings of its first p_i elements, accepts only rows appearing earlier in that cardinality order and deduplicates them with a byte mark array. It then applies the exact size condition $\min(|X_i|,|X_j|)/\max(|X_i|,|X_j|) \geq t$ before calling the cardinality kernel. Sparse-only runs use the adaptive $\mathsf{S}\times\mathsf{S}$ cell; bitmap-bearing runs orient the pair so that $\mathsf{B}\times\mathsf{S}$ probes the smaller list. The computed cardinality is finally tested against t. Thus shared prefix elements generate a superset of qualifying pairs, while the same exact row cells used elsewhere determine both the cardinality and final membership in the result.

The gate prices the two terms that control a prefix join. Let $P = \sum_i p_i$, let L_e be the length of element e's prefix posting, let $Q = \sum_e \binom{L_e}{2}$ be the candidate occurrences before deduplication, and let $\bar{s} = N^{-1}\sum_i |X_i|$. The implementation compares

$$W_{\text{prefix}} = 2P + Q\bar{s} \quad \text{with} \quad W_{\text{exact}} = \binom{N}{2}\bar{s}, \tag{19}$$

and activates the CSR path when $W_{\text{prefix}}/W_{\text{exact}} < 5$. The factor of two charges one pass to build and one to traverse the prefix occurrences; $Q\bar{s}$ prices exact verification; and the fixed threshold lies in the measured gap between activated and bypassed corpora. This decision is made once per corpus from the same prefix counts used to build the index.

For Table 5, rows were stored in sparse-only form so that the evaluated collection size was determined by encoded elements rather than by Nm bitmap storage. The maximum was 20,000 rows, or 199,990,000 possible pairs. Global ranks and the CSR posting index were constructed once before the repeated-query timer and reused; the timed prefix path includes posting traversal, candidate deduplication, size filtering and exact verification. The comparator enumerates every pair over the identical rows with the exact $\mathsf{S}\times\mathsf{S}$ cell and applies the same Jaccard test. Three repeats were run at each of $t \in \{0.001, 0.01, 0.1\}$ and the minimum was retained. Before a timing was accepted, the sorted set of qualifying pair identifiers from the posting path was required to equal that of the exhaustive scan. Supplementary Note 3 gives the exactness argument, output-sensitive cost and relation to the positional tile postings.

Online row-cell selection and offline tournaments

Selection — choosing which cell to run for a given pair — is distinct from the zone-map and rank-index and posting mechanisms above, which decide how many pairs or row items an already-chosen cell must visit. Four online row-cell policies were evaluated over the same eligible cells. Their runtime inputs are the constant-size row metadata already carried by the representations and, for the probe policies, timings collected during the active traversal. None requires a corpus-specific offline tournament before computation can begin.

Under *per-pair* selection, precomputed $O(1)$ metadata for each row — cardinality, run count, non-zero-word count, index-availability flags and exact extent — is consulted for every pair (i,j), and all calibrated cells are ranked by predicted cost. Under *per-tile* selection, rows are stably ordered by cardinality and partitioned into 64-row panels. Cardinality, run count and non-zero-word count are averaged over each panel, while the panel extent is the union of its row extents and an index is marked available only if every row carries it. One model decision is then shared by all 4,096 pairs of an off-diagonal 64×64 block or all 2,016 pairs of a diagonal block. An exact two-comparison extent test remains per pair: hoisting the expensive cell ranking does not discard a constant-time disjointness proof.

Under *probe-and-commit*, the $\varepsilon \to 0$ special case of Micro Adaptivity in Vectorwise[28], the model ranks the cells once per block and nominates its two cheapest candidates. Both candidates are timed on up to three representative pairs from the block, and the faster measured candidate is committed for the remaining pairs. Restricting the race to the model's top two avoids

timing a dense cell that the model places orders of magnitude behind on a large sparse universe. The companion *refine* policy also ranks candidates once per block but retains the top two: when the maximum-to-mean cardinality ratio of either panel is at least 8 and the runner-up's predicted cost is within a factor of 3 of the winner, each live pair chooses between only those two predictions. Otherwise refine collapses exactly to per-tile selection. The reported dynamic CRoaring comparison used this heterogeneity-gated refine policy as one fixed online policy; per-tile, probe-and-commit and alternative refinement gates were retained as ablations rather than selected per corpus with hindsight.

These online policies are distinct from the *offline tournaments* used to characterize them. The fixed-cell tournament times every eligible cell over the same corpus and chooses one winner with full hindsight; the bucket oracle repeats that comparison within metadata buckets. Both are evaluation ceilings and are charged no selection cost, so neither is presented as a deployable selector. For a stable corpus that will be traversed repeatedly at very large scale, an operator may optionally run the same tournament before service and reuse its verdict; this changes routing cost, not the exact result or the availability of the online path. Short-lived and inline workloads omit that step and execute the online policy immediately. An opt-in corpus calibration also timed every cell on a strided sample of real, span-overlapping pairs and fitted one time-per-work-unit slope per cell. It improved some rankings but was a net loss across the corpus sweep because a single slope absorbed corpus-size and memory-regime effects that the work function did not represent; it is therefore disabled by default. The analogous online gate and offline width tournament for coarse occupancy maps are specified in Supplementary Note 5. Every online policy is evaluated against the design budget of 2% of total runtime; a policy exceeding that budget is reported as failing it.

Selection cost was measured by instrumenting the decision path itself separately from the kernel calls it dispatches to, so that the reported cost of choosing is not conflated with the cost of computing. *Regret* against a bucket oracle is reported alongside selection cost for every online policy: the oracle is the best cell for a given bucket of pairs chosen with full hindsight after every candidate cell has been timed on it, and a policy's regret is its runtime on that bucket expressed relative to the oracle's runtime on the same bucket. The same bucket oracle is used to report regret for an always-B baseline and for a closed-form per-pair cost model evaluated from the metadata above but without online timing, so that measured selection can be compared directly against both a no-selection baseline and a modelled-selection baseline on the same metric.

Two offline references are never interchanged. The *bucket oracle* operates at bucket granularity and is the denominator for the regret experiment. The *best fixed cell* is coarser: for a whole corpus it takes the single cell whose measured end-to-end runtime is lowest. Because it may not vary within a corpus, it is not a lower bound on the bucket oracle's runtime. A third diagnostic in the implementation precomputes the model's per-pair choices before starting the clock; that measures free *model* selection, not a perfect measured oracle. The optimisation problem these policies approximate, and the conditions under which metadata alone determines the optimum, are given in Supplementary Note 9.

Corpora, provenance and loaders

Three corpus families were used. The first is the set of corpora CRoaring's own benchmark harness runs by default (the `real-roaring-datasets` collection), including `census1881`, `census-income`, `weather_sept_85`, `wikileaks-noquotes`, `uscensus2000` and the `dimension_0`*xx* family, together with the row-sorted (`_srt`) clustering variant of each that the original Roaring papers also report. The second is a set of larger modern graph corpora added beyond CRoaring's default set, drawn from public network-graph collections (including `com-LiveJournal`, `com-Orkut` and `as-skitter`). The third is genomic: two population-scale variant call sets, UShER SARS-CoV-2 and gnomAD chromosome 21, both stored variant-major (one row per variant site, one bit per sample), and one haplotype-major encoding of 1000 Genomes Project phase 3 chromosome 20 (one row per haplotype, one bit per site) used specifically to demonstrate that storage layout, not the underlying data, determines whether a corpus lands in the sparse or the mid-density regime. Universe size and density are reported per corpus in Table 2 and are not duplicated in prose here; every corpus is drawn against the same competitively tuned CRoaring baseline used throughout Results.

As a loader-correctness check, the `census1881` loader was validated by reproducing the published corpus statistics reported for it (universe size and mean row cardinality) against the values reported in the Roaring bitmap literature[49], rather than trusting the loader's own output uninspected.

Container census

The container counts in Table 3 are read from CRoaring's own introspection API, `roaring_bitmap_statistics()`, and not inferred from our measurements. The census is taken after `run_optimize()` and `shrink_to_fit()`, so that it reports the containers the library settles on once it has had the opportunity to choose run containers, matching the tuned baseline every ratio in this paper is computed against. Four fields are recorded per corpus: `n_array_containers`, `n_bitset_containers`, `n_run_containers`, and the total container count. The threshold governing that choice is a vendor fact rather than a measurement of ours: CRoaring stores a 2^{16}-bit chunk as an array container unless the chunk's own cardinality exceeds the compile-time constant `DEFAULT_MAX_SIZE`, whose value is 4096 (declared as `enum { DEFAULT_MAX_SIZE = 4096 }` at `roaring.h:2486`, and applied at insertion by `array_container_try_add(ac,`

`val, DEFAULT_MAX_SIZE)` in `container_add`, `roaring.h:5690`, in the pinned version below). The constant is not a tuning knob: it is an `enum` with no override, it is enforced as a structural invariant by `array_container_validate` and `bitset_container_validate` (`roaring.c:7133`, `roaring.c:8263`), and the portable serialization format writes no array-versus-bitset type tag at all, reconstructing the type from cardinality against this same constant on read (`roaring.c:14421`, `roaring.c:14553`). The same fixed commitment prices a run container against a bitset by iterating every run and charging for its length (`run_bitset_container_intersection_cardinality`, `roaring.c:10901`).

A compute-priced CRoaring baseline, and why it must be a second pass

Because `DEFAULT_MAX_SIZE` prices storage rather than query cost, comparing against stock CRoaring would compare against a library configured for a different objective. Every CRoaring number in this paper is therefore reported against a baseline we first re-priced ourselves, so that the comparison is against Roaring's structure under *our* objective and not against its storage objective. The re-pricing is a runtime, in-memory pass that promotes any container still held as an array past a compute threshold into a bitset (`roaring_bitmap_storm_promote_arrays`). It is a pure representation change — the set of values encoded is unchanged, and every cardinality query returns an identical answer before and after, cross-checked against the independent oracle on every pair before any timing is taken. Nothing is serialized: the promoted state exists only for the duration of the run, so the portable format and its cardinality-inferred container types are untouched.

The threshold is not `DEFAULT_MAX_SIZE`. The measured crossover at which an array–array scalar merge loses to a bitset–bitset popcount for a count-only AND lies between cardinalities 64 and 128 on Apple M4. The query-tuned baseline uses 64, selected by the subsequent real-corpus sweep.

The pass must run after `run_optimize()`, not before, and lowering the insertion threshold instead is worse than doing nothing. We first tried the obvious thing — lowering the constant that `container_add` tests — and measured a 46 per cent regression on `census1881` (1496 ns to 2186 ns per pair). The cause is that CRoaring's run conversion is not confluent with container type. Deciding whether a container should become a run compares the run encoding against the size of the encoding it currently has: for an array container that is $2c + 2$ bytes and scales with cardinality, but for a bitset container it is a fixed 8192 bytes. Promoting a borderline container to a bitset *before* run conversion therefore changes which comparison that container faces, and flips containers into runs that would otherwise have stayed arrays — a worse choice for AND-cardinality cost on the weakly clustered data this paper is concerned with. Running the promotion afterwards removes the interaction: run conversion has already made its choice on its own terms, so any container still an array has already been judged not worth converting, and the only decision left is a pure array-versus-bitset compute call. This ordering dependence is a property of the library's conversion rules rather than of our pass, and it is the reason a compute-priced Roaring cannot be obtained by retuning a single constant.

Benchmark protocol and cache-residency statement

Every reported cell variant, including CRoaring, was compiled with the same compiler and flags, allocated with the same alignment, and given the same warm-up treatment before timing, so that no comparison advantages one implementation by construction. CRoaring was tuned in good faith before every comparison: `run_optimize()` and `shrink_to_fit()` were called before timing began, matching what a competent user of the library would do, and every ratio against CRoaring reported in this paper is computed against that tuned baseline, not against CRoaring's untuned default construction.

Per-cell density and strategy sweeps (Figs. 2 and 3) raced every candidate variant over an identical, pre-generated pair list in an identical order, so that the cache state each variant encountered at call time was comparable across variants; each variant's differential correctness against the oracle (below) was checked before any of its timings were accepted. One untimed pass over the complete pair list preceded timed repeats. For the online real-corpus comparison, each contestant then repeated until it accumulated at least 100 ms of wall time, and the minimum repeat was retained within the process. Wall-clock time was measured with a monotonic per-run timer; where a hardware cycle count is reported, it is derived from wall time and a frequency measured on the host at run time rather than assumed from a nominal clock speed, and is labelled as a derived quantity rather than a hardware performance-counter reading. On Apple silicon hosts, benchmark threads were requested onto the performance core cluster to avoid the heterogeneous core migration that otherwise mixes two different microarchitectures' timings into one nominally single-threaded measurement.

The working-set-size sweep behind the residency comparison (Fig. 3b) varied bitmap-corpus footprint from 1 MB to 48 MB on Apple M4, Neoverse SVE2 and Sapphire Rapids, and cache-residency boundaries (L1, L2 and shared/last-level cache) were identified from each host's own reported cache geometry rather than assumed to be uniform across hosts (below). Every throughput number reported anywhere in this paper is qualified by the cache-residency regime it was measured in, named once here by regime (L1-resident, L2-resident, DRAM-resident) so that Results and Discussion can use the regime name as shorthand rather than restate cache sizes at each mention.

Statistics and aggregation

Two different aggregation conventions were used, for two different kinds of comparison, and neither is substituted for the other anywhere in this paper. For controlled synthetic sweeps in which every variant races the identical pair list under identical conditions (the density sweep, the asymmetric-cell strategy comparison, the working-set-size sweep), the reported cost is the *minimum* over five round-robin repeats for the density and run-length sweeps and seven repeats for the real-corpus cell and tile sweeps, on the standard microbenchmarking rationale that the minimum estimates the underlying cost with scheduler and system-noise inflation removed, while the mean instead reports how noisy the run happened to be. For the real-corpus comparison (Table 2), where repeats necessarily interleave across corpora and share a host with other system activity over a longer measurement window, the reported ratio is instead the *median* over three independent whole-process repeats, reported together with the observed range; no per-corpus ratio in this paper is quoted to two significant figures without that range being available alongside it.

All 23 completed corpus measurements are included in Table 2; the observed minimum–maximum interval across the three processes is retained beside every median.

Correctness oracle and differential testing

Every cell variant is checked against an independent scalar oracle that computes intersection cardinality by direct bit-by-bit or element-by-element counting, sharing no code path with any of the variants under test, before that variant is admitted to any timed comparison; a variant that returns a wrong cardinality is not reported as fast regardless of its measured speed. Correctness is tracked at three levels: an ABI-level regression suite with 1,279 oracle checks and zero failures, compiled and linked as C; a cell-level differential suite with 2,113,377 checks and zero failures across densities, structures and alignments; and cross-architecture runs with 2,668,209 checks on the Arm hosts and 2,428,809 on x86, again with zero failures. The public C ABI surface was verified independently with `nm`: 42 unmangled `STORM_` exports and zero defined mangled symbols.

The cell-level suite is driven by a fuzz grid over {row count, word count, density, within-row structure (uniform, clustered, run-based), alignment}, following the same axes used for every kernel change in this project. The regression suite's own ability to fail was itself verified rather than assumed: each of a set of known defects fixed earlier in the project was reverted individually and the suite was confirmed to fail before being re-fixed, establishing that a red result from the suite is informative and not merely a suite that always passes.

Hardware and toolchain

Measurements were taken on four microarchitectures spanning two independently hand-written SIMD kernel paths — one using Arm NEON, the other using x86-64 AVX-512 with the `VPOPCNTDQ` extension — plus a portable scalar fallback used where neither hand-written path applies. The hosts were a 10-core Apple M4, an AWS Graviton3-class Neoverse system, an AWS Graviton4-class Neoverse system and an Intel Xeon Platinum 8488C (Sapphire Rapids). The two Neoverse hosts execute the shared NEON pairing kernels; their SVE and SVE2 labels identify the machines, not distinct kernel implementations. Sapphire Rapids executes the AVX-512 $\mathsf{B}\times\mathsf{B}$ path gated on AVX512F and AVX512VPOPCNTDQ; the intrinsic-to-instruction mapping and feature gates follow the Intel Intrinsics Guide[27].

The Apple host reports a 64-kB L1 data cache, a 4-MB performance-cluster L2 and a 16-MB system-level cache. Cache-residency labels on every host were assigned from the cache geometry recorded with its run. Apple measurements used Apple clang 21 with `-O3 -std=c++17 -mcpu=native`; the three Linux hosts used GCC 11.5 with `-O3 -std=c++17 -march=native`. The CRoaring baseline used throughout is pinned at version v4.7.2, vendored as a source amalgamation within the project repository, so that the exact baseline is reproducible rather than referred to generically as "CRoaring."

Data and code availability

The benchmark harness, pairing-matrix kernels, online selectors, tile-postings implementation and per-corpus loader scripts are available in the project repository under the Apache License, Version 2.0. Public real-world corpora were obtained from their original sources and reproduced locally with the loader scripts referenced above. Raw and collated measurements, including the whole-process repeats used for Table 2 and the optimization record used for Table 4, are deposited with the source tree.

Supplementary Information

Supplementary Note 1: The fixed-cost headroom argument

Relocated from the Introduction, where it followed the statement that the $\Theta(m)$ bitmap kernel pays the same fixed cost regardless of skew, spending most of that cost combining zeros against zeros (main text).

That waste is not an anecdote about one kind of data; it is a property of the fixed cost itself, and its size can be settled before any kernel is written. Every alternative encoding of the same set — a sorted array of its elements, a list of its maximal runs, a fill-compressed word stream, or any of these applied to its complement — is exact and interconvertible with the bitmap, and each is sized by the set rather than by the universe. The cheapest of them is smaller than the bitmap by a factor that grows without bound as either operand moves away from density one half, so a flat cost provably leaves headroom at every density but the middle of the range, for a single operation as much as for a workload of them. Assuming nothing about the data beyond its density gives a floor on that headroom rather than an estimate of it: independently placed elements are the worst case for every encoding whose cost depends on how a set is arranged, so real structure moves those encodings further below the bitmap and never above it. What this paper optimizes is therefore not how fast a kernel runs but how much of the computation is never entered. The same analysis says where that is not possible. Around density one half no encoding is smaller than the bitmap and nothing can be skipped; there the fixed-cost kernel is the right one, and the return comes from executing it as fast as the hardware allows — which is what two decades of vectorized `AND`-and-popcount engineering already deliver[18]. Avoid the work where it can be avoided, accelerate it where it cannot: this paper contributes the first axis, and a cheap way of knowing which of the two applies.

Supplementary Note 2: Narrowing the query

All of that leaves the caller's question untouched, and a caller willing to narrow it can be given more. If only pairs above a similarity threshold are wanted, or only operands inside a band of set sizes, then the two cardinalities alone cap the similarity a pair can reach, so most pairs are discarded before either operand is read[10,12] — knowing what is being looked for is itself a way of not doing work, and it is the cheapest one available.

Supplementary Note 3: Thresholded-mode filtering — the size bound and prefix filtering

Full derivation for the opt-in thresholded mode named third in the mechanism ladder that opens Results, and stated with its hypotheses in Methods, "What makes work avoidable" (Eq. (8)–(10)): only pairs whose Jaccard similarity or intersection size clears a threshold are reported. Two mechanisms serve that mode, and the cheaper of them touches no data whatsoever. Because $|A \cap B| \le \min(a,b)$ while $|A \cup B| \ge \max(a,b)$, a pair's similarity is capped by the ratio of its two sizes, $J(A,B) \le \min(a,b)/\max(a,b)$, so a pair whose sizes differ by more than a factor $1/t$ cannot reach the threshold and is discarded from two integers, before either operand is read. An absolute threshold behaves the same way: $|A \cap B| \ge \tau$ requires only $\min(a,b) \ge \tau$. Restricting the *inputs* to a band of sizes — ignoring the extremely rare, or keeping only those — is that same comparison applied before a pair is formed rather than after. The bound is the Swamidass–Baldi pruning result[12] together with the size (length) filter of[30], shipped over popcount-sorted fingerprint bitmaps at all-pairs scale as BitBound in chemfp[14], and layered as a popcount-based bound onto set-similarity joins by[33]; what is reported here is not the bound but where it sits relative to the rest and how it composes with them.

How much it removes is fixed by one property of the corpus, and that property is what makes it worth separating from the mechanisms above. Under the $1/i$ cardinality spectrum this paper benchmarks — cardinalities log-uniform on $[1,K]$, which is the density $\Pr(a) \propto 1/a$ exactly, so this is the mandated spectrum and not a stand-in for it — the fraction of pairs that can possibly qualify is $1-(1-\ln(1/t)/\ln K)^2$ (Methods, Eq. (9)), which agrees with a 10^5-sample simulation to three decimals. At $K = 10^4$ that leaves 14.5% of pairs at $t = \frac{1}{2}$, 4.8% at $t = 0.8$, 2.3% at $t = 0.9$ and 1.1% at $t = 0.95$, so examining every pair does 6.9× to 90.0× the work of examining only the survivors. The expression falls as K rises: the wider the spread of set sizes, the more pairs are ruled out by the ratio alone. That is the reverse of the dependence every other mechanism in this paper has. The null models of Fig. 2 are all stated at a *fixed* density and bound what a mechanism is worth against within-row arrangement; the size bound is the only one whose value is a function of the *dispersion of cardinalities across the corpus*, and it strengthens monotonically as that dispersion widens — 2.3% surviving at $K = 10^4$ against 1.1% at $K = 10^8$, both at $t = 0.9$. Under the alternative model in which *ranked* sizes scale as $1/i$, the surviving fraction is exactly $1-t$ for every K. The spectrum therefore determines whether the benefit grows with corpus breadth or stabilizes at a threshold-controlled fraction (Methods, "What makes work avoidable").

It does not, however, multiply with the mechanisms below it, and the reason refines the composition rule this section has otherwise used. Deciding whether a pair exists at all is the coarsest granularity in the paper, so the rule as stated predicts a clean product with anything acting inside a pair. The product is not achieved. The survivors of the size test are the pairs with $a \approx b$, and representation choice, a zone map and the prefix filter are each cheapest precisely when the two sizes are *dissimilar* — so the test passes on the pairs the rest serve worst, and the composed cost exceeds the product of the two savings by the

factor that conditioning raises a survivor's mean cost, $\ln K/2$ in the limit of a high threshold (Methods, Eq. (10)). At $K = 10^4$, $t = 0.9$ and $m = 10^6$ the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition costs 0.151%. Replaying the same composition with the size test decorrelated from cardinality, at an identical survival rate, restores the product to within 0.6%, which identifies the shared statistic rather than the shared granularity as the cause. The composition remains firmly worth taking — it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does 660× its work — so what this refutes is the product, not the composition. Mechanisms compose multiplicatively when they act on different waste *and* read different operand statistics; acting at different granularities is necessary and not sufficient. These are analytic values over the model above, and like the rest of this note they count work rather than time.

The thresholded mode changes the contract rather than the kernel: only pairs above a similarity threshold are reported, and the second mechanism that exploits that is prefix filtering rather than any representation choice. It admits the same analysis, and it reduces further than the others do. Because a qualifying pair must share an element between its prefixes, the method belongs to the established filter-and-verify lineage for exact set-similarity joins[10,29–32]. The entire behaviour of the filter is governed by one dimensionless quantity — the expected number of shared prefix elements per pair, $y = p^2/m$ — and the fraction of pairs surviving is $1 - e^{-y}$ regardless of density, threshold or universe size separately (Supplementary Fig. S1). Below $y = 1$ pruning improves quadratically as prefixes shorten; above it prefixes collide by default. The implemented gate directly prices prefix traffic and posting multiplicity through Eq. (19), retaining the same output-sensitive dependence without assuming independent placement.

Measured realization and scaling. Let E be the number of globally ranked elements, $P = \sum_i p_i$ the number of indexed prefix occurrences and L_e the length of element e's posting. The two-pass CSR build costs $\Theta(E + P)$ time and storage. During a query, traversing every row prefix and its postings costs $\Theta(P + \sum_e L_e^2)$ before deduplication; if C_t is the set of unique candidates, exact verification adds $\sum_{(i,j)\in C_t} K(i,j)$. Rare-first ordering reduces both L_e^2 and $|C_t|$, while raising t shortens every p_i. This is why the algorithm scales with prefix occurrences and posting multiplicity rather than with $\binom{N}{2}$. In the 63-point real-corpus sweep, the online gate activated the CSR path 48 times and every activation improved over exhaustive exact scan, from 1.67× to 1,089× (Table 5).

Supplementary Note 4: Derivation of the zone-map cost model and its crossings

This note gives the full derivation behind Fig. 2b–d: the null-model floor, the four counting properties of a zone map, the crossing densities and the work-versus-time distinction that determines the selector's route.

Every panel of Fig. 2 is the same argument applied to a different mechanism, and it is worth stating that argument once. Each mechanism — choosing a representation, consulting a zone map, composing the two, and in the thresholded mode the prefix filter of Supplementary Fig. S1 — is a bet that the data deviates from randomness, and each comes with a closed-form null model saying what the bet is worth if it does not. Fixing the density and placing elements independently is the most pessimistic assumption available about *arrangement*, and it is pessimistic in the same direction for all of them: independent placement maximises the number of maximal runs a set has, maximises the fraction of bins it occupies, and maximises the chance that two prefixes collide. The null model is therefore a floor and not a prediction. Real structure can only move each mechanism further into profit, and the size of that movement is exactly what the mechanism converts into speed.

Two things a fixed design cannot do follow immediately, and they are the reason the selector is built the way it is. A representation committed to at ingest cannot exploit the deviation, because it commits before the deviation is knowable. A selector reading density alone cannot exploit it either, because density is the very statistic the null model is built from: at a fixed density d with $k = dm$ elements the expected number of maximal runs is $k(m-k+1)/m$, but the actual number ranges over every integer from one to $\min(k, m-k+1)$, so no single function of d can track it. The deviation is also *one-sided* — a real row can be arbitrarily cheaper than a random row of the same density and at most about twice as expensive (Methods, "What makes work avoidable") — which is what makes it a resource rather than a source of variance. Both readings point the same way: the statistic a selector consults has to see arrangement, or be a direct measurement of the kernels themselves.

Four properties of the summary in Fig. 2b–c follow from counting alone, and they are worth stating plainly because together they say what a summary can and cannot buy.

Why there is a floor, and where it is. A zone map is a bitmap of bitmaps: one bit per bin of W_z positions, so m/W_z bits against the row's m. Reading it therefore costs $1/W_z$ of reading the row, and no pairing that consults a summary can cost less than reading that summary — 0.195% of a bitmap at $W_z = 512$. That is the floor, and it is a property of the bin width alone, not of the data.

Why the leverage is so large. The exchange rate is the same W_z: one summary bit decides the fate of W_z data bits, so a bin proved empty returns W_z bits of work for the one bit spent asking. This is why a coarser bin buys a deeper floor — and, symmetrically, why it stops paying at a lower density, since a wider bin is harder to leave empty.

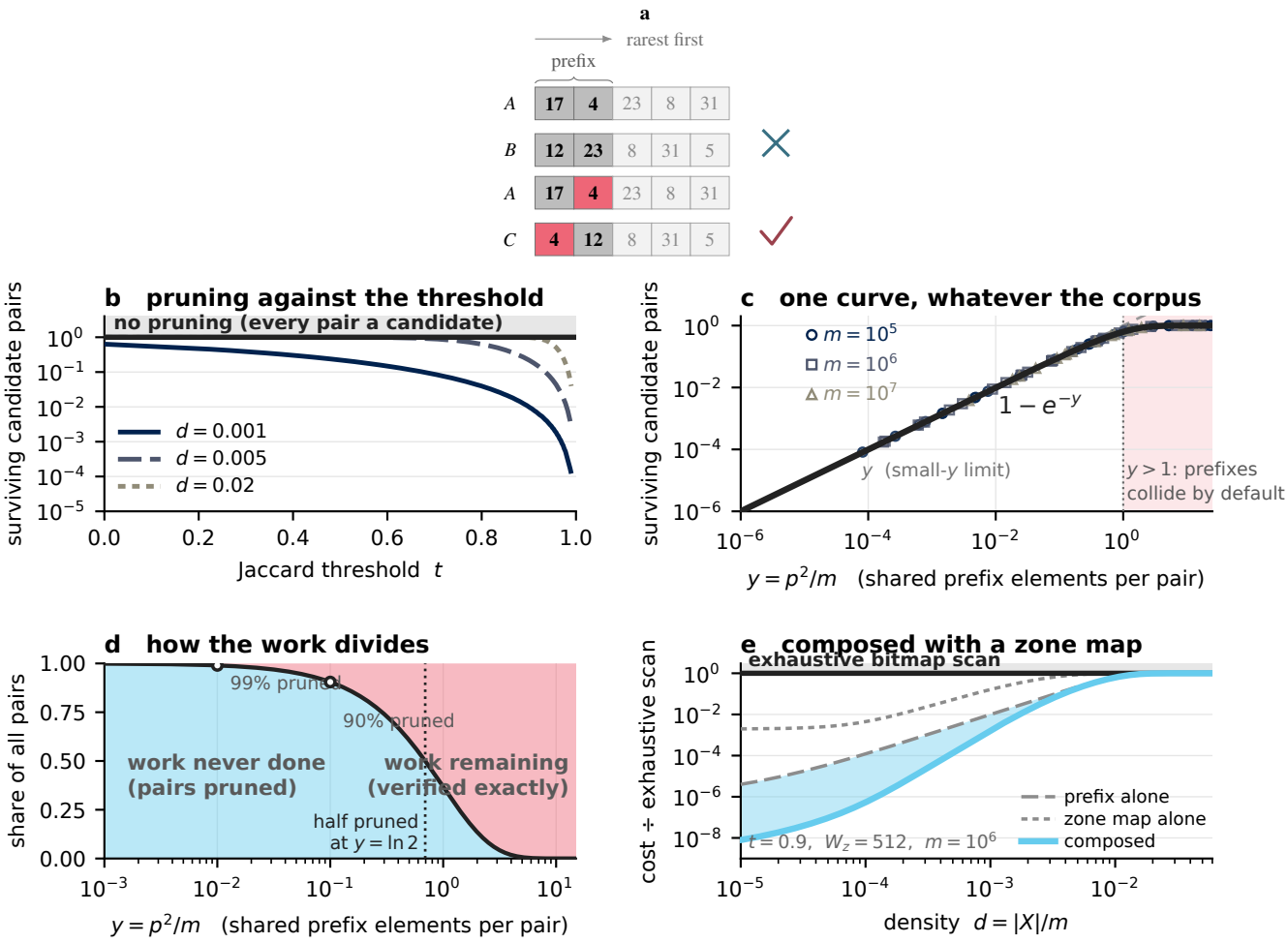

Figure S1. Prefix filtering, analysed on the same terms — and reducible to a single variable. Prefix filtering serves the opt-in thresholded mode, in which only pairs of Jaccard similarity at least t are reported. With elements in a global order and prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$[11,31], following the prefix-filtering principle of[50], any qualifying pair must share an element between its prefixes, so indexing prefixes alone answers the thresholded question exactly — and unlike the mechanisms of Fig. 2c–d it is candidate *generation*: the quadratic pair set is never enumerated. Reading key as in Fig. 2; an ordered series takes a sequential ramp with its own marker or dash pattern. **a**, How the rule works. Elements are laid out rarest first, hence the non-monotonic labels, and each set's prefix (saturated) is its rarest $p(X)$ elements. Top: the prefixes share no element, so the pair is discarded without computing any intersection. Bottom: one element appears in both, marked in red because it is what commits the pair to exact verification. **b**, Fraction of pairs surviving as candidates against the threshold, at universe 10^6. Prefix length falls as t rises, so the filter that removes almost everything at a high threshold removes almost nothing at a low one, and denser operands need a higher threshold to prune as hard. **c**, All of that collapses onto one curve. For prefixes of length p over a universe of m the expected number of shared prefix elements per pair is $y = p^2/m$, and the surviving fraction is exactly $1 - e^{-y}$ for scattered operands; markers are (d, t, m) combinations drawn at random, and lie on the curve by construction rather than by fit. For $y \ll 1$ the surviving fraction is y itself; for $y > 1$ prefixes collide by default and the selector routes to exact scan. So y — not t, and not density — is the governing variable. **d**, The same as a division of labour: pairs never examined against pairs surviving to exact verification. The two are equal at $y = \ln 2$. **e**, Prefix filtering composed with a zone map. The mechanisms act at different resolutions: prefix postings control which pairs are generated and the zone map controls which regions an exact verification visits. The panel reports their analytic combined work against exhaustive bitmap scan. Analytic throughout; like Fig. 2, it models work rather than time.

Why the advantage is flat across the sparse range and then erodes quadratically. A bin survives only if *both* operands occupy it, so the surviving fraction goes as the product of the two occupancies. For clustered operands, where a run longer than a bin occupies exactly a d fraction of bins, that product is d^2, and the cost is $1/W_z + d^2\kappa$ with κ the cost of a bin's contents. The two terms are equal when $d^2\kappa = 1/W_z$; with κ the cost of one run in a bin, $\kappa = w/W_z$, this reduces to $d = 1/\sqrt{w} = 1/8$

— independent of the bin width. Below that density the summary term dominates and the advantage is constant; above it the surviving-bin term dominates and the advantage decays as d^2. Changing W_z moves the floor but not the knee.

Why a summary can be a liability, and exactly when. The floor cuts both ways, and the reason is structural rather than a property of any one representation: a summary is indexed by the universe, while every compressed representation is indexed by the data. The zone map costs $1/W_z$ of a bitmap whatever the operands contain, because its size is set by m. The cost of S, R and W is set instead by a quantity that shrinks with the set — cardinality, run count, or literal-and-fill count — and each of those falls without limit as the operands sparsify. A fixed cost and a vanishing one must cross. Below that crossing, consulting a summary costs more than the entire computation it was meant to shorten, and a zone map is worse than not having one.

Where the crossing falls depends on which representation is being displaced, and one case fixes the scale. A representation costing one word per element costs wd of a bitmap, so it meets the summary's $1/W_z$ at $d = 1/(wW_z) = 3.1 \times 10^{-5}$ for $w = 64$ and $W_z = 512$ (Methods, Eq. (7)). At $d = 10^{-6}$ that representation costs 6.4×10^{-5} of a bitmap while the summary alone costs 1.95×10^{-3}, so consulting it does $31\times$ the work of simply enumerating the operands. A run- or fill-based representation crosses at its own density, wherever its run or word count falls below m/W_z, but that a crossing exists at all is generic.

This makes the gating two-sided, and for two unrelated reasons. A summary must be declined when the operands are *too dense*, because every bin is occupied and it proves nothing; and equally when they are *too sparse*, because it is then the largest object in the computation. It pays only between those two bounds. The same fact appears in the storage measurements as universe-proportional selector metadata dwarfing the data it describes; the time and space statements of it are one phenomenon, and a guard on one is a guard on the other.

The practical reading is that the sparse regime is not a narrow operating point. In the clustered idealization — the favourable edge of the band, where a run spans a whole bin — the advantage is the same at $d = 10^{-6}$ as at $d = 10^{-2}$: five orders of magnitude of density over which a fixed-cost kernel does $512\times$ the work of one that consults a summary first. It is only above $d = 1/8$ that the margin narrows at all, and even at $d \to 1$ a bitmap still does $7.9\times$ the work.

All of the quantities above are counts of 64-bit words touched, not times, and the distinction is load-bearing rather than pedantic. A work model prices a summary that removes nothing at its own size, $1/W_z$ of a bitmap, and that is a correct statement about words. It is not a statement about runtime: a filtered scan gives up sequential access and hardware prefetch and pays an indirection and a branch per bin, and the measured penalty for consulting a summary on operands it cannot help is far larger than these panels charge — **[worst-case speedup, filter applied unconditionally, which the work model does not predict]** against the $1.002\times$ overhead the work model allows, matching the always-on-versus-gated contrast reported directly in Results, "Online tournaments make representation selection negligible" (Fig. 4c). That divergence is not a failure of the derivation; it is the reason the gate is decided by measurement rather than by the model, and the per-bin and per-word constants needed to convert these curves into time are given in Methods.

Supplementary Note 5: Column postings over row tiles: construction, exactness and scaling

The column view is a different algorithmic family. The ten pairing-matrix cells are row-oriented: choose (i, j), then traverse representations of X_i and X_j. Column postings invert a coarse row–bucket incidence relation within a tile. For the incidence matrix A of Eq. (16), the row view stores $O_i = \{b : A_{ib} = 1\}$ while the column view stores $P_b = \{i : A_{ib} = 1\}$. Walking O_i is useful when one pair has already been chosen. Walking P_b is useful when the task is to discover, among many rows, which pairs could possibly intersect. Writing p_b for the indicator vector of P_b, the candidate pattern is the Boolean lower triangle of AA^{T} and each bucket contributes the outer product $p_b p_b^{\mathsf{T}}$. This transposition is the source of the batch saving: a bucket is read once for all row pairs that share it, instead of rediscovered by pairwise ANDs of row summaries. Outer-product sparse multiplication and transposed accumulation are classical[15,16]; element-level wedge expansion applies the same column-induced-pair view to exact all-pairs common-neighbour counts[26]. The positional posting path specializes that lineage at a coarser resolution and composes it with the row matrix: bucket postings certify zeros, one-word tile masks deduplicate candidate occurrences, and a representation-pair cell computes every surviving exact cardinality.

These postings must not be confused with the element-keyed prefix index of Supplementary Fig. S1. Prefix filtering changes a thresholded similarity join by indexing selected elements and generating only pairs that can still clear the threshold. The column postings here are keyed by *coarse positional buckets*; they serve the unchanged exact-all-pairs contract and can discard a pair only after proving that the rows share no bucket. A shared bucket is merely a candidate, because its rows may contain different elements within that bucket.

Construction and resolve. For each tile, the evaluated $T = 64$ implementation performed the following steps.

1. It sorted the tile's rows by $\min X_i$ and stored each exact extent $[\min X_i, \max X_i]$. In the range-first cascade, the ordering permits the scan for i to stop at the first j with $\min X_j > \max X_i$. In the later amortised-posting variant, the same extent test is applied only to pairs already present in the sparse candidate masks.

2. It traversed every set bit b of every row occupancy map O_i and executed $P_b \leftarrow P_b \cup \{i\}$. Because $T = 64$, the posting was a single machine word and the update was one shift and OR.

3. It defined the multi-occupancy bucket set $H = \{b : |P_b| \geq 2\}$ and compacted the corresponding posting words into a dense list. Empty postings encode no row; singleton postings encode no distinct pair. After compaction, the bucket identifier is unnecessary for candidate generation and only the row word is retained. Identical row words from different buckets remain separate list entries.

4. It formed a touched-row word $D = \bigcup_{b \in H} P_b$. Candidate masks were cleared only for $i \in D$. For each $b \in H$, it iterated over the set row bits $i \in P_b$ and executed $C_i \leftarrow C_i \cup P_b$. Repeating the same pair through several buckets is harmless because OR is idempotent; a pair reaches an exact row-wise cell at most once.

5. It applied the extent-overlap condition to C_i, removed i and all $j \leq i$, and evaluated each remaining pair with the exact gated $\mathsf{B}\times\mathsf{S}$ verifier used by the postings benchmark. The row-cell tournament supplies the direct route selected by the tile gate.

The amortised variant constructs the postings once, retains only H and reuses that compact transpose during candidate resolution. Table 4 reports candidate resolution and exact verification over diagonal within-tile blocks under this reuse schedule. The construction term is reported separately in Eq. (S3), which also gives its amortisation under a blocked traversal that reuses a built posting panel.

Exactness. Range rejection is exact because disjoint closed extents imply disjoint sets. Posting rejection is also exact in the one direction used. If $X_i \cap X_j$ contains x, then $b = \lfloor x/\Delta \rfloor$ belongs to both O_i and O_j; hence $i, j \in P_b$, the update for P_b sets j in C_i, and the pair survives before triangular masking. Therefore

$$X_i \cap X_j \neq \varnothing \quad \Longrightarrow \quad j \in C_i, \tag{S1}$$

Equation (S1) means that a pair absent from the candidate masks has cardinality zero. The converse fails whenever two different elements collide in one bucket. This is why postings can settle rejected pairs but only a row-wise cell can compute the cardinality of an accepted pair.

Work and storage. Let w be the machine-word width, $W = \lceil T/w \rceil$, $q_b = |P_b|$, $z = \sum_b q_b = \sum_i |O_i|$, $h = |H|$, and let C be the set of unique candidate pairs after applying the extent-overlap condition. The current dense builder zeros fW posting words, inserts z row–bucket incidences, and scans the f postings during compaction. Its construction cost is therefore $\Theta(fW + z)$ and its peak temporary storage is $\Theta(fW)$. The retained multi-occupancy list costs $\Theta(hW)$ words; at $T = 64$, this is one word per retained posting. Candidate masks require $\Theta(TW)$ words.

The logical number of bucket-induced pair occurrences is

$$G = \sum_{b:q_b \geq 2} \binom{q_b}{2}, \qquad |C| \leq \min\left\{ \binom{T}{2}, G \right\}. \tag{S2}$$

Equation (S2) counts repeated pair occurrences before bit-mask deduplication. A scalar inverted-list expansion would pay $\Theta(G)$. The bit-mask resolve instead ORs W words for each row present in each retained posting, for $\Theta\big(W \sum_{b:q_b \geq 2} q_b\big)$ word operations, and deduplicates the G logical occurrences implicitly. Exact verification then costs $\sum_{(i,j) \in C} K(i,j)$, where $K(i,j)$ is the cost of the selected row cell for that pair. The complete cost is consequently

$$\underbrace{\Theta(fW + z)}_{\text{construction}} + \underbrace{\Theta\left(W \sum_{b:q_b \geq 2} q_b + TW \right)}_{\text{resolve}} + \sum_{(i,j) \in C} K(i,j), \tag{S3}$$

In Eq. (S3), the construction term is divided by the number of block visits only when the block traversal reuses the same built postings. By comparison, direct row-wise execution pays $\sum_{i<j} K(i,j)$ and has no posting build.

When postings win. The favourable limit is not merely low row density; it is low *column multiplicity*. If every occupied bucket belongs to at most one row, then $h = 0$, $C = \varnothing$, and the whole tile is proved disjoint after construction without touching any original row data. More generally, postings win when most q_b are one or two, so the compact list and resolve are small and $|C| \ll \binom{T}{2}$. This occurs when rows occupy few buckets relative to f, and it is strengthened when their positional extents are separated: the cheaper range pass then eliminates pairs before a posting resolve is needed. Long runs or high per-row density do not by themselves help the posting path; they raise q_b, increase collisions, and ultimately make the candidate matrix dense.

The dense-column route applies when even one coarse bucket contains most rows. If $q_b = T$, that posting alone makes C the complete pair set. The method then pays construction and resolve before running the same row kernels as the direct path. This is the column analogue of applying a zone map whose every bin is occupied, and it is why the postings path must be gated. Between the limits, bucket width trades memory for precision. Increasing f reduces unrelated rows colliding in one bucket and therefore reduces G and $|C|$, but grows the per-row occupancy maps and the transient fW table linearly. The best width can consequently move when the posting table crosses a cache boundary.

Tile width makes a second trade. At fixed per-row occupancy, direct pair work grows as T^2 while the incidence count z grows initially as T, so amortised planning work per possible pair can fall as T grows. Once $T > w$, however, every posting and candidate row occupies $W > 1$ words, multiplying the resolve and its working set. The measured implementation retained $T = 64$ because it makes a posting one word; a $T = 256$ variant reduced the arithmetic cost per potential pair but regressed on two of five corpora while increasing the posting table from the measured L1-capacity scale into L2. The association does not isolate cache residency from scheduling or other host effects, so $T = 64$ is an empirical choice for this implementation, not a universal optimum.

Online gate and offline configuration tournaments. The online postings gate is intended for data whose best path is not known in advance. On a strided sample it estimates

$$s = \frac{\text{element probes whose bucket is occupied on the other row}}{\text{sampled sparse-side elements}}, \qquad t = \frac{\text{sampled sparse-side elements}}{\text{128-byte cache lines in the sampled dense rows}}. \tag{S4}$$

For the two statistics in Eq. (S4), the tile harness sampled every seventh partner in up to the first four tiles and entered the filter path at $s < 0.15$ and $t < 0.25$. It separately disabled range when the first tile rejected less than 5% of pairs. An early cascade variant built postings only when range left more than 25% of a tile; the later amortised variant prebuilt compact postings once and used them whenever the corpus-level gate accepted the filter path. The thresholds are plateau-centre settings from the research harness rather than constants derived from the model, and the touch statistic uses the target's measured 128-byte cache-line size.

No offline width search is required for correctness or for use of the postings path. Every legal power-of-two bucket width Δ preserves Eq. (S1): width changes the number of collisions and the size of the summary, but never removes a true intersection from the candidate set. An online-only execution therefore starts with a configured width, measures the two gate statistics during the active traversal and either commits to postings or routes directly to the row tournament. This is the appropriate path for one-shot, short-lived and inline computations. When a stable corpus will drive billions or trillions of comparisons, the same exact algorithm can instead pay a one-time width tournament and reuse its choice; only the performance specialization is offline.

Two offline tournaments answer different questions and are not substitutes for that gate. The coarse-filter corpus optimizer is analogous to a build-time `run_optimize()`: with the full corpus available, it times bypass and filter widths of 4,096, 8,192, 16,384, 32,768, 65,536, 131,072 and 262,144 bits, then selects the measured winner. It estimates the ceiling available to a system allowed an offline build pass. The tile-postings width sweep extended the search through 1,048,576 and 4,194,304 buckets. The reported tile-pipeline measurements use the winner of this per-corpus tournament. The universe-only rule evaluated in the same sweep separated fewer configurations than the tournament, showing that cardinality and clustering are required alongside universe size for automatic width selection. Timing candidate filters on a small sample and committing the fastest was also tested. It over-selected wide filters because the sample kept them resident while the full traversal evicted them, so the measured sample was biased with respect to the workload it was supposed to predict. That probe-and-commit variant was rejected for filter-width selection even though probe-and-commit remains useful for the row-cell tournament.

Rejected structural variants. Ordering the stages matters. Building the transpose before applying range was slower when range had already emptied most of the tile; the cheap exact test must precede the expensive structure. A second hierarchy over positional buckets was also rejected: after compaction has removed every empty and singleton posting, the coarse level has no such entries left to prune and adds a dense scan. It was 2.4 to 100 times slower than the flat multi-occupancy list across six corpora. Deduplicating identical posting words, unrolling the resolve, sorting postings by population and specializing two-row postings changed time only within the run-to-run noise band. Prefetching candidate rows was counterproductive because it requested the row data before the filter had decided whether that data was needed. These failures all have the same boundary: once candidate generation is output-sensitive, optimizing the mechanism is less valuable than keeping non-candidates out of it.

What is amortised. The exact-answer contract is unchanged, but the batch regime exposes quantities that can be paid for once. Cardinality, run count, extent and occupancy maps are built with the row and reused across its operations. A cell decision is shared by a tile or panel. A probe tournament times a handful of pairs and reuses its verdict. Column postings invert a tile

once and reuse each bucket across the row pairs it induces. In a complete blocked traversal, loaded operand panels can likewise be reused before eviction. Only the exact verification of the surviving pairs necessarily remains per operation. This is a property of any workload with enough related operations to share preparation; it does not require that a caller materialize an N^2 output. The complete dataflow is shown in Fig. 5.

Transfer conditions beyond set intersection. The row and column algorithms instantiate a general work-admission procedure. First partition the payload into regions whose contribution can be summarized more cheaply than it can be evaluated. Second define an exclusion implication of the form in Eq. (14); the neutral result may be zero, no emitted tuple, no graph edge, no accumulator update or a predicate known to be false. Third retain the ordinary exact kernel for every survivor. Fourth estimate both summary cost and survivor fraction and apply Eq. (15), using online samples when no workload profile exists. Finally hoist the summary or decision to the largest collection of operations that can reuse it without changing its meaning.

This procedure explains the three different forms of leverage measured here. A row representation makes one operation scale with cardinality, run count or fill count. A zone map exchanges one summary bit for a decision over an entire region. A posting transpose reuses one occupied column across all rows it contains and changes scaling from the complete consumer space to incidences, posting multiplicities and survivors. The operator-specific proof establishes which contribution is neutral; the systems pattern — summarize, certify, gate, hoist and verify — is unchanged. It can therefore be applied to sparse joins, block-sparse matrix operations, graph and inverted-index traversal, and telemetry predicates whenever their metadata supplies the required sound certificate.

Supplementary Note 6: Corpus provenance and inventory

Supplementary Table S1 gives full provenance and per-corpus statistics for every corpus referenced anywhere in this paper, including the corpora scored only against an all-bitmap baseline and never against CRoaring, which Table 2 summarises in a single row. It also records, for the two genomic corpora, which axis is treated as the universe under each storage layout, exposing the representation change induced by transposing the same incidence data.

Table S1. Corpora, provenance and statistics. Every corpus referenced anywhere in this paper, in four provenance groups used to assemble the benchmark set: **G1**, the twelve `real-roaring-datasets` files CRoaring's own harness runs by default[2,3]; **G2**, five SNAP social/web graphs (Stanford Large Network Dataset Collection) scored by neighbourhood-intersection density; **G3**, two genomic corpora in paired row/column layouts used to expose incidence-orientation effects; **G4**, KONECT graphs, UShER SARS-CoV-2, gnomAD and the `msprime` coalescent-simulation series, added after the original survey. Density and mean $|X_i|$ are corpus properties from the data loaders, not timing results. For the two G3 corpora, *universe* and *rows* swap roles between layouts: `chr20.bin` is variant-major (universe = 5,008 haplotypes, one row per variant site); `chr20_hap.bin` is haplotype-major (universe ≈ 1,048,576 sites, one row per haplotype) — the layout dependence discussed in the main text. —, not reported by the source. [All values provisional, pending the final measurement campaign.]

Corpus	Group	Rows/sets	Universe m	Mean $\|X_i\|$	Density	Origin / notes
`census1881`	G1	200	4,277,806	5,019	1.2e-3	1881 Canadian census, categorical attributes
`census1881_srt`	G1	200	4,277,735	3,404	8.0e-4	as above, rows sorted
`census-income`	G1	200	199,523	34,610	1.7e-1	US census income (UCI "Adult"-family), attributes
`census-income_srt`	G1	200	199,523	30,464	1.5e-1	as above, rows sorted
`weather_sept_85`	G1	200	1,015,367	64,353	6.3e-2	September 1985 weather observations, binned attributes
`weather_sept_85_srt`	G1	200	1,015,367	80,540	7.9e-2	as above, rows sorted
`wikileaks-noquotes`	G1	200	1,353,179	1,377	1.0e-3	WikiLeaks cable corpus, term/attribute postings
`wikileaks-noquotes_srt`	G1	200	1,353,133	1,440	1.1e-3	as above, rows sorted
`uscensus2000`	G1	200	36,974,578	30	8.1e-7	US Census 2000; sparsest and largest-universe G1 corpus
`dimension_003`	G1	15,482	3,866,847	91	2.4e-5	Druid-derived dimension column
`dimension_008`	G1	5,240	3,866,845	347	9.0e-5	Druid-derived dimension column
`dimension_033`	G1	173	3,866,847	22,352	5.8e-3	Druid-derived dimension column
`as-skitter`	G2	1,696,415	1,696,415	[n]	9.1e-6	CAIDA skitter internet AS topology, 2005; undirected, symmetrized
`wiki-Talk`	G2	2,394,385	2,394,385	[n]	2.2e-5	Wikipedia talk-page edits; directed; only 67,492 vertices have out-degree ≥ 2
`soc-Pokec`	G2	1,632,803	1,632,804	[n]	1.6e-5	Pokec social network (Slovakia); directed; lowest skew of the graphs – top 1% of sets hold only 8.8% of elements
`com-LiveJournal`	G2	3,997,962	4,036,538	[n]	5.1e-6	LiveJournal friendship, ground-truth communities; undirected, symmetrized
`com-Orkut`	G2	3,072,441	3,072,627	[n]	2.7e-5	Orkut social network; undirected, symmetrized; largest input in this set
`chr20.bin`	G3 (layout)	[n]	5,008	[n]	3.5e-2	1/i site-frequency spectrum, 43.6% singletons; variant-major, universe 626 bytes, L1-resident
`chr20_hap.bin`	G3 (layout)	[n]	≈1,048,576	[n]	3.1e-2	haplotype-major; large-universe orientation selects the all-bitmap route (0.93× relative comparison)
`dbpedia-link`	G4	4,384,102	18,268,993	29.8	1.63e-6	KONECT
`wikipedia_link_en`	G4	5,978,043	11,206,012	46.4	4.15e-6	KONECT
`livejournal-groupmemberships`	G4	2,197,915	7,489,074	48.7	6.50e-6	KONECT, bipartite; universe inflated ∼2.3× (single id space for both columns; true member domain is 3,201,203)
`usher_sarscov2`	G4	29,411	8,451,771	15,769.3	1.87e-3	UCSC UShER SARS-CoV-2; mean dominated by near-universal variants (max 99.7% carriers), median only 404
`enwiki-categorylinks`	G4	2,165,205	129,698,523	102.3	7.89e-7	Wikimedia dumps; largest universe, lowest density in the full set
`gnomad_chr21_exomes_af1e-3`	G4	625,656	1,461,894	28.2	1.93e-5	gnomAD v4.1
`msprime_10k`	G4 (sim)	≈42k	2e4	1,887	9.4e-2	msprime dense-spectrum control
`msprime_100k`	G4 (sim)	≈51k	2e5	15,720	7.9e-2	as above
`msprime_1M`	G4 (sim)	≈15.6k	2e6	136,288	6.8e-2	as above

Supplementary Note 7: Selector-metadata overhead and storage cost

Storage is a reported trade-off rather than an objective this paper optimizes for. The two tables below separate the two costs that the main text keeps apart. Supplementary Table S2 gives the ungated reference footprint of selector metadata per row and against the data it describes. It quantifies why the implementation constructs universe-proportional rank and occupancy

Table S2. Cardinality-gated construction bounds universe-proportional selector metadata. The table reports the ungated reference footprint of the rank index and occupancy map, both of which scale with universe rather than cardinality. Meta B/row and data B/row are bytes of selector metadata and bytes of the chosen data representation, per row; ratio is meta:data. On `enwiki-categorylinks` that is 4 MB of selector metadata attached to 155 bytes of data – a 26,312× overhead. The complement representation already carries a guard, built only when $2n >$ universe; the implementation constructs occupancy only when $512n >$ universe and gates rank construction on mean run length. Rows for which the ungated reference footprint exceeds data by more than $10^3\times$ are bold. [All values provisional, pending the final measurement campaign.]

Corpus	Universe m	Meta B/row	Data B/row	Ratio (meta:data, ×)
`census-income`	199,523	6,335	13,344	0.5
`weather_sept_85`	1,015,367	32,031	54,497	0.6
`census1881`	4,277,806	134,784	18,774	7.2
`as-skitter`	1,696,415	53,479	49	**1,083**
`dbpedia-link`	18,268,993	575,415	113	**5,051**
`enwiki-categorylinks`	129,698,523	4,084,799	155	**26,312**

Table S3. Storage cost, bits per value, data and metadata separated. Reported the way the Roaring papers report it[49], so corpora of different cardinality are comparable. Metadata is excluded from every column here: the zone map and rank index are what the selector needs to *choose* a representation, not a representation themselves, and folding them into a data column would hide the cost of the mechanism this paper adds (Supplementary Table S2). **Best** is the minimum of the four representation columns, per corpus, and is set in bold in every row for that reason. Bitmap costs are given in scientific notation throughout, one notation down the column. On dense corpora the numbers are Roaring-comparable (3–7 bits/value); on sparse corpora a dense bitmap costs 10^3–10^6 bits/value, the storage statement of the same fact the timing tables make: a bitmap over a large sparse universe is not a compression scheme, it is a liability. [All values provisional, pending the final measurement campaign.]

Corpus	Universe m	Bits per value				
		Bitmap	Array	Runs	EWAH	**Best**
`census-income`	199,523	5.77e0	32.00	20.73	3.86	**3.08**
`msprime_100k`	200,000	1.26e1	32.00	31.88	5.29	**4.44**
`weather_sept_85`	1,015,367	1.58e1	32.00	30.06	7.87	**6.77**
`usher_sarscov2`	8,451,771	1.08e3	32.00	2.04	4.06	**2.02**
`wikileaks-noquotes`	1,353,179	9.83e2	32.00	11.36	19.46	**10.64**
`census1881`	4,277,806	8.52e2	32.00	58.86	43.79	**29.92**
`as-skitter`	1,696,415	1.26e5	32.00	47.13	77.72	**29.26**
`wikipedia_link_en`	11,206,012	2.68e5	32.00	46.21	72.90	**28.27**
`dbpedia-link`	18,268,993	6.38e5	32.00	56.38	101.63	**31.84**
`uscensus2000`	36,974,578	1.24e6	32.00	57.78	91.89	**31.98**
`enwiki-categorylinks`	129,698,523	3.33e6	32.00	62.72	113.23	**31.85**

structures only when row cardinality and run structure can amortise them. Supplementary Table S3 gives the cost of the chosen data representation alone, with that metadata excluded, so the two are never conflated.

Supplementary Note 8: Formal treatment of work avoidance — full proofs

Methods ("What makes work avoidable") states each result of this development as an equation with its hypotheses and a pointer here. This note supplies the proof, the worked numerical checks and the secondary results that support but do not themselves state a numbered result in Methods. Notation follows Methods throughout: $A, B \subseteq [0,m)$, $a = |A|$, $b = |B|$, $d_A = a/m$, $d_B = b/m$, word width w, effective sparsity $s_X = \min(d_X, 1-d_X)$, $s = \min(s_A, s_B)$. For a representation ρ and a set X, $\kappa_\rho(X)$ is the number of stored items a kernel must touch to traverse X in that representation — $\lceil m/w \rceil$ words for B, $|X|$ for S, the run count $r(X)$ for R, and literal-word count plus fill-group count $\ell(X) + g(X)$ for W. All costs below are counts of stored items or of elementary operations, never a time; the constants that convert one to the other are measured, not derived, and are given in Methods under "Benchmark protocol and cache-residency statement".

Proof of Eq. (1) (the Fréchet interval) and Eq. (2) (its width). Upper bound: $A \cap B \subseteq A$ and $A \cap B \subseteq B$, so $|A \cap B| \le \min(a,b)$. Lower bound: $|A \cup B| = a + b - |A \cap B| \le m$, so $|A \cap B| \ge a + b - m$; together with $|A \cap B| \ge 0$ this gives the stated interval. Every integer t in the range is attained: take $A = [0,a)$ and $B = [a-t, a-t+b)$, so that $A \cap B = [a-t, a)$ has size t. The construction is legal because $t \le a$ gives $a - t \ge 0$, $t \ge a + b - m$ gives $a - t + b \le m$, and $t \le b$ makes $[a-t, a) \subseteq B$.

For the width, $\min(a,b) - \max(0, a+b-m) = \min(a,b) + \min(0, m-a-b) = \min\big(\min(a,b), \min(a,b) + m - a - b\big)$,

and $\min(a,b)+m-a-b = m-\max(a,b)$, so the width is $\min\big(\min(a,b),\, m-\max(a,b)\big) = \min(a,b,m-a,m-b)$. Grouping the four terms in pairs, $\min\big(\min(a,m-a),\, \min(b,m-b)\big) = \min(ms_A, ms_B) = ms$. The number of feasible answers is $ms+1$, so a cardinality-only view of a pair leaves $\log_2(ms+1)$ bits of the answer unresolved. The four terms $a,b,m-a,m-b$ are exactly the sizes of the four lists A, B, A^c, B^c one could enumerate, so the residual uncertainty is the size of the smallest of them — not a coincidence, since Eq. (3)'s proof below shows the smallest of those four lists is precisely what an exact algorithm enumerates to answer the query.

The pigeonhole floor. The lower bound of Eq. (1) lifts off zero exactly when $d_A + d_B > 1$, giving $|A \cap B| \geq (d_A + d_B - 1)m$; equivalently, by inclusion–exclusion on complements, $|A \cap B| = a + b - m + |A^c \cap B^c|$, so the floor is the case $A^c \cap B^c = \varnothing$: the complements can be at most disjoint. At $d_A = d_B = 0.51$ the floor is $(2 \cdot 0.51 - 1)m = 0.02m$ — the factor is two, so 51% density forces 2% of the universe to overlap, not 1%; as a fraction of either operand it is $2 - 1/d \approx 3.92\%$. Above $d = \frac{1}{2}$ the floor rises at $2m$ per unit density while the ceiling $\min(a,b) = dm$ rises at m, so the width $m(1-d)$ shrinks at the constant rate m per unit density — consistent with $m(1-d) = ms$ for $d \geq \frac{1}{2}$.

Proof of Eq. (3) (the headroom). $\kappa(\mathsf{B}) = \lceil m/w \rceil$ for every X by definition: a dense bitmap stores m bits whatever they contain, so $\mathsf{B}\times\mathsf{B}$ performs $\lceil m/w \rceil$ word AND-plus-popcount operations for every pair independently of a, b, of structure and of the answer. This is not a defect of any implementation; B is the only member of the matrix whose descriptor length is a function of the universe alone.

Suppose each operand X is available as a bitmap *and* as a sorted array of whichever of X, X^c is the smaller — $\Theta(ms_X)$ of extra space, decided from a,b,m in $O(1)$. No rank index is required for what follows; that enters separately, below. Without a stored complement array, enumerating X^c from the bitmap costs $\Theta(m/w + (m-a))$, whose first term is already $\kappa(\mathsf{B})$ — so the dense arm of the argument genuinely requires the complement to be materialised, not derived on the fly. Then $|A \cap B|$ is computed in $O(ms)$ constant-time membership probes, by four cases, one per term of $\min(a,b,m-a,m-b)$: if $U = a$, enumerate A's a positions and probe each into B's bitmap (a probes); if $U = b$, symmetric; if $U = m-a$, enumerate A^c and recover $|A \cap B| = b - |A^c \cap B|$ ($m-a$ probes); if $U = m-b$, symmetric via $|A \cap B| = a - |A \cap B^c|$. Each case is $O(1)$ arithmetic plus U probes. Two of the four cases require the complement view, which is why C is part of the design and not an afterthought for the dense tail: without it the achievable cost is $\min(a,b)$, which is $\Theta(m)$ at high density, and the dense arm of the U does not exist.

Comparing descriptor items touched, $\kappa(\mathsf{B})/U = \lceil m/w \rceil/(ms)$. Taking $w \mid m$ (true of the implementation, which pads to a whole number of words) this is exactly $1/(ws)$; otherwise read it as $\geq$ with an additive w/m term (relative error at $m = 1000, w = 64$ is 2.4%, at $m = 65$ it is 97%; nothing downstream changes). The ratio exceeds one whenever $s < 1/w$ and is unbounded as $s \to 0$. What this does and does not assert: it asserts that a specific, exact, correct algorithm exists whose cost is ms items where $\mathsf{B}\times\mathsf{B}$ spends m/w items, and that the ratio between the two achieved costs diverges. It does *not* assert that ms is a lower bound on the work any algorithm must do ("Why the interval width is not a lower bound on work", below); both quantities compared are achieved, neither is a floor.

The crossover heuristic. Converting to time needs two measured constants: the cost of a random probe into a bitmap, c_p, and the cost of a vectorised AND-plus-popcount word, c_w/V. Equating the two costs gives the crossover $s^* = c_w/(c_p w V)$ — an explicitly constant-factor, not asymptotic, argument, and the only quantitative prediction of this kind in this note.

Proof of Eq. (4) (the rank-index bridge). This is the only result here that converts a descriptor *length* into an *operation count*; every other result in this note is a fact about storage. Let A be a bitmap carrying an $O(1)$ rank index, $\mathrm{rank}_A(i) = |A \cap [0,i)|$, and B a run container with maximal runs $\{[u_i, v_i)\}$, $i = 1, \ldots, r$. The runs partition B, so $A \cap B$ is the disjoint union of $A \cap [u_i, v_i)$, and $|A \cap [u_i, v_i)| = \mathrm{rank}_A(v_i) - \mathrm{rank}_A(u_i)$ by the definition of rank. Summing over the r runs gives Eq. (4), computed in exactly $2r$ rank lookups and $2r-1$ additions — independent of m, of $|A|$, of $|B|$ and of the run lengths $v_i - u_i$: a run of any length is priced as a run of length one, because the kernel never looks inside it. Symmetrically, $\mathsf{R}\times\mathsf{R}$ by simultaneous merge of two sorted run lists costs $O(r_A + r_B)$ with no rank index needed, since the merge never inspects a position inside a run. This is what makes a run count a cost parameter and not merely a storage parameter; it converts descriptor length into operation count, not into time, since per-operation constants still differ across cells.

Proofs for Eq. (5) (expected descriptor lengths under a null model). Model: each position of $[0,m)$ is set independently with probability d (Bernoulli), and $w \mid m$ so every word carries w live bits. The B and S rows are exact and not asymptotic statements at all: $\mathbb{E}\,\kappa(\mathsf{B}) = \lceil m/w \rceil$ trivially, and $\mathbb{E}\,\kappa(\mathsf{S}) = md$ by linearity of expectation over the m indicator variables. For $\mathsf{C}\circ\rho$, substitute $1-d$ for d, since complementation is an involution on density.

Run count. A maximal run of ones begins at position i iff $x_i = 1$ and ($i = 0$ or $x_{i-1} = 0$). Summing the indicator expectations over $i = 0, \ldots, m-1$ gives the exact Bernoulli $\mathbb{E}[r] = d + (m-1)d(1-d)$; under the fixed-cardinality model (uniform random k-subset) the same argument gives the exact $\mathbb{E}[r] = k(m-k+1)/m$. Both equal $md(1-d)(1+O(1/m))$, whose leading-order form in $s = \min(d, 1-d)$ is $ms(1-s)$, the form used in Eq. (5). The sandwich is exact, not merely leading-order: writing

$p_0 = (1-d)^w$ is not needed here, but the two exact forms bound each other by

$$\tfrac{1}{2}ms \;\le\; \mathbb{E}\,\kappa(\mathsf{R}) \;\le\; ms+1, \tag{S5}$$

since fixed-cardinality $\mathbb{E}[r] = ms(1-s)+d$ and Bernoulli $\mathbb{E}[r] = ms(1-s)+d^2$ both exceed ms marginally at the dense corner ($m = 64, k = 57$ gives $\mathbb{E}[r] = 7.125$ against $ms = 7$; the worst ratio $\mathbb{E}[r]/(ms)$ tends to 2). This corrects an earlier form of the sandwich that omitted the $+1$ and is violated by 38 cases near $d \to 1$ without it.

Fill-compressed stream. Let $p_0 = (1-d)^w$, $p_1 = d^w$ be the probabilities that a w-bit word is all-zero or all-one, and $n = m/w$. Words are independent because they cover disjoint bit ranges (this is where $w \mid m$ is used). Dirty (literal) words: $\mathbb{E}[\ell] = n(1-p_0-p_1)$. Fill groups: a maximal block of consecutive all-v words begins at word j iff word j is all-v and ($j = 0$ or word $j-1$ is not all-v), so $\mathbb{E}[g_v] = p_v + (n-1)p_v(1-p_v)$ exactly. Summing over $v \in \{0,1\}$,

$$\mathbb{E}[\ell+g] \;=\; n(1-p_0-p_1)+p_0+(n-1)p_0(1-p_0)+p_1+(n-1)p_1(1-p_1) \;=\; n-(n-1)(p_0^2+p_1^2), \tag{S6}$$

which to leading order in n is $n[1-p_0^2-p_1^2]$, matching Eq. (5)'s form $(m/w)[1-(1-s)^{2w}-s^{2w}]$ (the exact form exceeds the leading-order one by at most $p_0^2+p_1^2 \le 2$ words, immaterial at any n of interest). The encoding must compress both all-zero *and* all-one fills for the symmetry claims below to hold; an encoder compressing only zero fills is not complement-symmetric. Sparse limit. $(1/w)[1-(1-s)^{2w}-s^{2w}] \to 2s$ as $s \to 0$, so under the null model W costs about *twice* S in the sparse tail, not the same — the reason W never appears on the cost envelope in either tail.

Complement symmetry (Proposition 9, supporting the paragraph "the deviation, which is the contribution"). $\kappa(\mathsf{S})(X^c) = m - \kappa(\mathsf{S})(X)$: S is *not* complement-symmetric, which is why the C strategy exists. $\kappa(\mathsf{B})(X^c) = \kappa(\mathsf{B})(X)$ trivially. R is complement-symmetric by construction, because runs of ones and runs of zeros alternate along $[0,m)$, so $|r(X^c) - r(X)| \le 1$; the sign is not a two-sided uncertainty for a given set, it is determined by boundary membership, exactly $r(X^c) = r(X) - 1 + \mathbb{1}[0 \notin X] + \mathbb{1}[m-1 \notin X]$:

$0 \in X$	$m-1 \in X$	$r(X^c) - r(X)$
no	no	$+1$
no	yes	0
yes	no	0
yes	yes	-1

Degenerate cases obey it: $X = \varnothing$ gives $r = 0$, $r(X^c) = 1$; $X = [0,m)$ gives $r = 1$, $r(X^c) = 0$. This is why $\mathsf{R}\times\mathsf{R}$ wins the dense tail with no complement machinery: R's descriptor length is nearly invariant under complementation, while S's is not, which is exactly why the array-based cells need an explicit complement view and the run-based ones do not. $\kappa(\mathsf{W})(X^c) = \kappa(\mathsf{W})(X)$ *exactly, provided* $w \mid m$: complementing every word maps all-zero words to all-one words and back and maps dirty words to dirty words, leaving the literal/fill partition untouched. This fails when $w \nmid m$: a zero-padded tail word that is all-ones over the live bits is not all-ones over the padded word, so complementation moves it between the literal and fill classes.

Proof of the U-shape as a theorem of the null model (supporting "what randomness would predict"). Each leading-order entry of Eq. (5) is, as a function of s, either constant (B) or strictly increasing on $[0, \frac{1}{2}]$: best-of-$\mathsf{S}/\mathsf{C}\circ\mathsf{S}$ is ms, increasing; R is $ms(1-s)$, and $\mathrm{d}/\mathrm{d}s[s(1-s)] = 1-2s > 0$ for $s < \frac{1}{2}$; W is $(m/w)[1-(1-s)^{2w}-s^{2w}]$, with derivative $(m/w)\cdot 2w[(1-s)^{2w-1}-s^{2w-1}] > 0$ for $s < \frac{1}{2}$. Each is symmetric about $d = \frac{1}{2}$ by construction. This is a statement about the leading-order forms, not the exact ones: the exact Bernoulli $\mathbb{E}[r] = d + (m-1)d(1-d)$ is not itself a function of s (at $m = 1000$ it is 90.01 at $d = 0.1$ and 90.81 at $d = 0.9$, a gap of $1-2d$, and its maximum sits at $d = \frac{1}{2} + 1/(2(m-1))$ rather than at $d = \frac{1}{2}$), while $\mathbb{E}\,\kappa(\mathsf{W})$ *is* exactly a function of s, since $(1-d)^{2w} + d^{2w}$ is symmetric under $d \mapsto 1-d$. Nothing downstream depends on the choice.

By Eq. (S5), the four compressed representations agree to within a constant factor under the null model: S-with-complement costs ms, R costs between $\frac{1}{2}ms$ and $ms+1$, and W costs about $2ms$ in the sparse tail while saturating at m/w. Selection *among* $\{\mathsf{S}, \mathsf{C}\circ\mathsf{S}, \mathsf{R}, \mathsf{W}\}$ therefore buys at most a constant under randomness — randomness gives run containers nothing over sorted arrays. B is the exception and the exception is the entire point: it is a member of the matrix and its expected cost is m/w at every density, so the spread across the full matrix is $(m/w)/(ms) = 1/(ws)$ (Eq. (3)), which is unbounded. At $s = 10^{-6}$ the ratio of the most to the least expensive representation in the matrix exceeds 10^4. What collapses under randomness is the choice among the compressed representations; what does not collapse is the choice between the bitmap and the rest. Correspondingly it is the *envelope*, not every cell, that is $\Theta(\min(m/w, ms))$ — $\kappa(\mathsf{B})$ is m/w regardless. The crossover of that envelope sits at

$ws \approx 1$, i.e. $s^* \approx 1/w$, about 1.6% for $w = 64$ before per-item constants. The U-shape is a genuine theorem of the null model and of the bitmap's flatness. It is not a property of the method, and the method is not in the business of reproducing it — on real data the representations deviate below the null without limit, and that deviation, not the U itself, is the contribution (next).

Proof that density is not a sufficient statistic for cost (the deviation, supporting Eq. (5) and the impossibility argument). Fix m and $d \le \frac{1}{2}$, $k = dm$. Over the sets of density exactly d: $\kappa(\mathsf{B})$ and $\kappa(\mathsf{S})$ are constants, functions of (m,k) alone; $\kappa(\mathsf{R})$ ranges over *every* integer in $[1, \min(k, m-k+1)]$; $\kappa(\mathsf{W})$ ranges from $O(1)$ to $\Theta(\min(k, m/w))$. Two explicit families witness the extremes. $X_{\text{block}} = [0,k)$ has one run, $r = 1$, and its words are one all-one fill flanked by all-zero fills, so $\kappa(\mathsf{W}) = O(1)$. For the spread extreme, take

$$X_{\text{spread}} := \{\, i\lfloor m/k \rfloor \,:\, 0 \le i < k \,\}, \tag{S7}$$

which has $|X_{\text{spread}}| = k$ *exactly* (the largest element is $(k-1)\lfloor m/k \rfloor < m$) — unlike the naive candidate "every $\lceil 1/d \rceil$-th position", which has density $k = dm$ only when $1/d$ is an integer and is not used here for that reason. The stride $\lfloor m/k \rfloor \ge 2$ whenever $k \le m/2$, so every element of X_{spread} is isolated and $r = k$. For $\kappa(\mathsf{W})$: every w-bit word contains at least one element iff the stride is $\le w$, i.e. iff $d \gtrsim 1/w$, in which case every word is dirty and $\kappa(\mathsf{W}) = m/w$; otherwise the k elements fall in distinct words, giving k dirty words separated by $\Theta(k)$ fill groups, $\kappa(\mathsf{W}) = \Theta(k)$. For the intermediate run counts, choose any composition of k into r parts (the run lengths) and any distribution of the $m-k$ zeros into the $r+1$ gaps with the $r-1$ internal gaps non-empty; this is feasible iff $r \le k$ and $r \le m-k+1$, exactly the claimed range (a run needs at least one element, and r runs need at least $r-1$ separating gaps).

The worked case. $m = 10^6$, $A = [0, 5\times10^5)$, $B = [2.5\times10^5, 7.5\times10^5)$. Both operands sit at density exactly $\frac{1}{2}$ — the worst point of Eq. (2), where cardinality alone leaves 500,001 feasible answers and the null model of Eq. (5) predicts $m/4 = 250{,}000$ runs apiece. Both in fact have one run. $\mathsf{B}\times\mathsf{R}$ with a rank index (Eq. (4)) answers the pair in *two rank lookups*; $\mathsf{B}\times\mathsf{B}$ spends 15,625 word operations to reach the same number. Nothing about the density says which situation a selector is in.

The structure ratio. Define $\sigma_\rho(X) := \mathbb{E}_{\text{null}(d)}[\kappa_\rho]/\kappa_\rho(X)$, against the *fixed-cardinality* null throughout ($\mathbb{E}[r] = k(m-k+1)/m$), which conditions on the observable k and is exact; it differs from the Bernoulli form by $O(1)$ and nothing depends on the choice. $\sigma_\rho \equiv 1$ identically for B and S. At every density,

$$\sigma_{\mathsf{R}}(X) \;\ge\; \frac{\max(k, m-k+1)}{m} \;=\; \max\left(d,\; 1-d+\tfrac{1}{m}\right) \;\ge\; \tfrac{1}{2}, \tag{S8}$$

with equality only for the maximally scattered set, and σ_{R} is unbounded above, reaching $\Theta(m)$ on X_{block}. *Proof.* $\sigma_{\mathsf{R}} = \mathbb{E}[r]/r \ge \mathbb{E}[r]/r_{\max}$ with $r_{\max} = \min(k, m-k+1)$ and $\mathbb{E}[r] = k(m-k+1)/m$; since $xy/\min(x,y) = \max(x,y)$, the ratio is $\max(k, m-k+1)/m$, which exceeds $\frac{1}{2}$ because $k + (m-k+1) = m+1 > m$ forces the larger term above $m/2$. This is stronger than a $d \le \frac{1}{2}$-only bound: the restricted form $1-d+1/m$ degenerates to 0.002 at $d = 0.999$, a spurious $500\times$ worst case, where the true bound is $\sigma_{\mathsf{R}} \ge 0.999$. Read as a sentence: real data can be arbitrarily cheaper than random data at the same density, and at most about twice as expensive.

The impossibility (Corollary B), stated at the strength it is actually proved. Any selector whose input is d (or a, b, or any function of them) evaluates one fixed function $f(d)$ on every row of that density; by the range above, $\kappa(\mathsf{R})$ takes every value in $[1,k]$ at $d = k/m$, so $f(d)$ mis-predicts $\kappa(\mathsf{R})$ by an unbounded factor on some rows at every density, and no such selector can distinguish X_{block} from X_{spread}. Two consequences, and the line between what is proved and what is measured must be drawn precisely. (i) A fixed representation cannot exploit σ, since it commits before σ is knowable. (ii) A density-only *cost model* cannot exploit σ either, since it is a function of d and the impossibility applies to it verbatim — but it does *not* follow that such a model is worse than making no selection at all: whether a mis-prediction costs more than a fixed default depends on which cell the wrong prediction picks and that cell's actual cost, neither of which is modelled here. The regret comparison in Results, "Online tournaments make representation selection negligible" (Fig. 4b), is a measurement of that question, not a derivation of it. What *is* licensed is narrower and still strong: the proved statement rules out one class of selector — those reading only cardinality — and does not choose among the survivors. A run count or a zone-map occupancy count is $O(1)$ metadata that sees σ perfectly and can perfectly well be fed to a model; probe-and-commit is a different survivor. The theory says only: gate selection on a statistic that sees structure rather than on cardinality. Whether that statistic feeds a structural model or a direct measurement of the candidate kernels is settled by measurement, not derived here.

The null model is a floor, and it is the same floor three times (proofs). Independent placement at fixed density is the *extremal* arrangement for every mechanism used here, always in the same direction. For a representation this is Eq. (S8) above. For a zone map of bin width W_z, the occupied-bin fraction under independence is $p = 1-(1-d)^{W_z}$ against $p = d$ when every run spans a bin; $1-(1-d)^{W_z} \ge d$ for all $d \in [0,1]$ and $W_z \ge 1$ (by convexity of $(1-d)^{W_z}$ in d against its tangent at $d = 0$), with equality only at the endpoints. Scattering therefore maximises occupancy, so the summary's null cost is an

upper bound on its cost and a lower bound on its value. For the prefix filter of the narrowed-contract mode, with prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$, the probability that two independently placed prefixes of length p each share an element, drawn from a universe of size $y^{-1}p^2$, is $1 - e^{-y}$ with $y = p^2/m$: the entire mechanism depends on the operands through that one dimensionless quantity, and any ordering that concentrates rare elements into prefixes makes real prefixes collide less often than scattered ones of the same length. In each case the null is what the mechanism is worth on data with no structure to find; real data deviates from it only in the mechanism's favour; the magnitude of that deviation is not a function of density and so is invisible to any cost model whose inputs are cardinalities; and it is exactly the quantity the mechanism converts into avoided work.

Where the headroom vanishes (Proposition 12, full detail). At $s_A = s_B = \frac{1}{2}$: the cardinality view excludes nothing ($U = m/2$, Eq. (2)); the best list-based cell costs $m/2$ probes (Eq. (3)'s proof) against B×B's m/w word operations, so B×B wins by $w/2$ — a factor of 32 at $w = 64$, before vectorisation; and under the null model $\mathbb{E}[r] = m/4$ and $\mathbb{E}\,\kappa(\mathsf{W}) = (m/w)(1 - 2 \cdot 2^{-2w}) \approx m/w$, so neither R nor W offers anything either, and W has degenerated into B with header overhead. So the headroom ratio of Eq. (3) does not merely shrink at $s = \frac{1}{2}$ — it falls below one ($1/(ws) = 1/32$ at $w = 64$), and the correct behaviour of a selector there is to choose B×B. (The ratio is of course positive; "negative headroom" is the wrong description for a quantity that has fallen below unity.) This is a statement about the generic case and is false for structured rows at mid density: X_{block} sits at $d = \frac{1}{2}$ with $r = 1$. The honest form is: given only cardinalities, and absent structure, the mid-band is where nothing can be avoided; a structural summary can still find something there, which is why the zone map earns its place. Call this a *metadata floor* or an *identifiability floor* — never an information-theoretic floor, since nothing here lower-bounds the work of an arbitrary algorithm — and name what it floors: what cardinality metadata determines, not what any kernel must spend.

Proof of Eq. (6) and the three regimes (supporting "why the two mechanisms are selected between rather than stacked"). Partition the universe into bins of width W_z and let each position be set independently with probability d. Let K be a bin's cardinality and $p = \Pr[K \geq 1] = 1 - (1-d)^{W_z}$. Then

$$\mathbb{E}[K \mid K \geq 1] \;=\; \frac{W_z d}{p}, \qquad \mathrm{Var}[K \mid K \geq 1] \;=\; \frac{W_z d(1-d)}{p} - \frac{W_z^2 d^2 (1-p)}{p^2}. \tag{S9}$$

Proof. K is zero on the complement of the conditioning event, so $\mathbb{E}[K] = \mathbb{E}[K \mid K \geq 1] \cdot p$ gives the mean; the second moment follows the same way from $\mathbb{E}[K^2] = W_z d(1-d) + W_z^2 d^2$, and the variance is the difference. As $d \to 0$ the conditional distribution collapses onto the single value 1 (coefficient of variation 0.016 at $d = 10^{-6}$, $W_z = 512$), so in the regime where concentration matters most, "an occupied bin" means "a bin with exactly one bit" almost surely: $d/p \to 1/W_z$ as $d \to 0$ and $d/p \to d$ as $d \to 1$. A zone map cannot hand a kernel anything sparser than one bit per bin — that is the whole phenomenon, and it is why the composition is not a product.

Let $\kappa(d) \in (0, 1]$ be the cheapest-representation envelope of Eq. (5), normalised so $\kappa = 1$ is a plain bitmap, and let the zone-map pass cost a fraction c/W_z of a plain-bitmap scan (c counts how many summary streams the pass reads: $c = 1$ for the AND-and-popcount over the occupancy bitmap alone, $c = 2$ if the pass also charges reading both operands' summary streams separately). For A, B drawn independently at matched density d, $p_A = p_B = p$, the composed cost relative to a plain bitmap is Eq. (6), evaluated at the *concentrated* density d/p of Eq. (S9), not at d. Two consequences are exact. The composition never costs more than the zone map alone, since $\kappa \leq 1$ and zone-map-alone is $c/W_z + p_A p_B$. And the composition does not dominate the representation alone: it is floored at c/W_z, whereas $\kappa(d) \to 0$ as $d \to 0$, so below that floor the summary costs more than a sparse representation pays outright. Hence three regimes, not two. In the sparse band $\kappa(d) = wd$, so the composition beats both the representation alone and the bitmap exactly when

$$x e^{-x} \;>\; c/w, \qquad x = W_z d \text{ (expected set bits per bin)}, \tag{S10}$$

a window in bits per bin that is independent of W_z and of the universe. At $c = 1$, $w = 64$ the window is $x \in [0.0159, 5.94]$, i.e. $d \in [3.1 \times 10^{-5}, 1.2 \times 10^{-2}]$ at $W_z = 512$ and $d \in [3.9 \times 10^{-6}, 1.5 \times 10^{-3}]$ at $W_z = 4096$; at $c = 2$ the window is $x \in [0.0323, 5.09]$. The lower boundary has the exact, memorable form $d_{\text{low}} = c/(wW_z)$ (Eq. (7)), the density at which a one-word-per-element representation's own cost wd falls below the zone map's overhead; below it reading the summary costs more than enumerating the operands outright, e.g. 31× their work at $d = 10^{-6}$ ($c = 1, w = 64, W_z = 512$). The upper boundary is where bins stop being empty: at $x \approx 5$ the occupancy $p = 1 - e^{-x}$ exceeds 0.99, the filter removes almost nothing, and its overhead is pure loss. Both boundaries were confirmed against a direct numerical scan of the full envelope (not the sparse approximation) at $W_z \in \{256, 512, 1024, 4096\}$, agreeing with the closed form to three significant figures. A run- or fill-based representation crosses at its own density, wherever its descriptor length falls below m/W_z; that a crossing exists at all is generic. Prefix filtering has the same shape in $y = p^2/m$: for $y \ll 1$ the surviving fraction is y itself, so pruning improves quadratically as prefixes shorten, while for $y > 1$ prefixes collide by default and the index earns nothing.

Refinement is monotone but the zone-map popcount is an upper bound, not an exact count. Partitioning $[0,m)$ into $n_\beta = m/\beta$ bins with per-bin cardinalities a_k, b_k, $\sum_k \max(0, a_k + b_k - \beta) \le |A \cap B| \le \sum_k \min(a_k, b_k)$, the refined interval is contained in the global one and its width is $\sum_k \min(a_k, b_k, \beta - a_k, \beta - b_k) \le U$ — apply Eq. (1) inside each bin and sum. The relation to the zone-map popcount is an inequality, not an equality:

$$\#\{\text{bins with a non-degenerate refined interval}\} \;\le\; \text{popcount}(\text{occ}_A \wedge \text{occ}_B), \tag{S11}$$

with equality iff no bin is saturated on both sides. A bin in which both operands are full is occupied on both sides, so the popcount counts it, yet its refined interval is the single point β — degenerate. The two operationally useful statements survive untouched: a popcount of zero proves the operands disjoint, and the popcount is the number of bins a kernel consulting only occupancy will visit (a kernel that does not know a bin is saturated must still visit it) — but the popcount does not count only the bins whose contribution is genuinely undetermined.

Why the interval width is not a lower bound on work. Nothing in this note lower-bounds the work any algorithm must perform. Choosing a concrete machine model makes the point sharply: in the pure membership-probe model, where an algorithm knows m, a, b in advance and its only primitive is "is position i in A?" or "in B?", with the requirement that it output $|A \cap B|$ exactly, then whenever $a \notin \{0, m\}$ and $b \notin \{0, m\}$ it makes at least $m/2$ probes in the worst case, *independently of s*. *Proof.* Complementing an operand is a bijection on probe sequences (the answer flips) and determines the answer via $|A \cap B| = b - |A^c \cap B|$, so assume $a \le b \le m/2$. The adversary answers "no" to every probe. Let P_A, P_B be the probed sets, q_A, q_B their sizes, $q = q_A + q_B$. If consistency has already been forced ($m - q_A < a$ or $m - q_B < b$) then $q > \min(m-a, m-b) \ge m/2$ and the claim holds. Otherwise choose $A \subseteq \overline{P_A} \cap \overline{P_B}$ with $|A| = a$, possible when $q \le m - a$. The values of $|A \cap B|$ realisable by b-subsets $B \subseteq \overline{P_B}$ span every integer in $[\max(0, b - (m - q_B - a)), \min(a, b)]$, an interval containing two distinct values whenever $q_B < m - b$. So the algorithm cannot have halted while both $q \le m - a$ and $q_B < m - b$, giving $q \ge \min(m - a + 1, m - b) \ge m/2$ since $a, b \le m/2$. This bound is $\Omega(m)$ at *every* density — at $a = b = 1$ it already forces $m - 1$ probes — so it tracks nothing about the interval width. The reason is that a membership-probe model cannot read a sorted array, which is precisely the capability the proof of Eq. (3) assumes. The interval width is therefore not a lower bound in any model considered here, and the nearest model that does give a lower bound gives one insensitive to s; a matching bound in a representation-aware model is a comparison-complexity question with its own literature and is not attempted here.

Worked numerical examples. Universe $m = 10^6$, word width $w = 64$, so $\kappa(\mathsf{B}) = 15{,}625$ words throughout.

A — sparse, $d_A = d_B = 10^{-4}$, $a = b = 100$. Interval $[0, 100]$, width $100 = ms$; 101 feasible answers; $\mathbb{E}|A \cap B| = 0.01$; $\Pr[\varnothing] \approx 99.0\%$; best list cell 100 probes; headroom $1/(ws) = 156\times$; $\mathbb{E}[r]$ under the null ≈ 100, so R buys nothing over S here. The typical answer is 0, and the whole computation should be a disjointness proof.

B — mid-density, unstructured, $d_A = d_B = \frac{1}{2}$. Interval $[0, 500{,}000]$, width $m/2$, 500,001 feasible answers; best list cell 500,000 probes, $32\times$ worse than B×B; $\mathbb{E}[r]$ under the null 250,000, $16\times$ worse; $\mathbb{E}\,\kappa(\mathsf{W})$ under the null $\approx 15{,}625$, W has become B; headroom $1/(ws) = 1/32 < 1$: none.

C — mid-density, structured (the thesis in one line). $A = [0, 5 \times 10^5)$, $B = [2.5 \times 10^5, 7.5 \times 10^5)$; both $d = \frac{1}{2}$, identical to case B by every cardinality statistic. Interval from cardinality unchanged, $[0, 500{,}000]$; $\mathbb{E}[r]$ under the null 250,000; actual $r_A = r_B = 1$; $\sigma_{\mathsf{R}} = 250{,}000$; B×R with a rank index answers in 2 lookups against B×B's 15,625 words. Cases B and C are indistinguishable to any density-based selector and differ in cost by four orders of magnitude; without Eq. (4) this comparison is a run count against a word count and is meaningless.

D — dense, $d_A = d_B = 0.51$. Pigeonhole floor $(2d - 1)m = 2 \times 10^4$, 2% of the universe (factor two, not one), 3.92% of each operand; ceiling 5.1×10^5; width $m(1 - d) = 4.9 \times 10^5 = ms$, consistent with Eq. (2). At $d = 0.99$ the same algebra gives floor $0.98m$, width $0.01m$, and a complement view of 10^4 elements per side — comparable to $\kappa(\mathsf{B}) = 15{,}625$, right at the crossover $s^* \approx 1/w = 1.6\%$. At $d = 0.999$ the complement side is 10^3 and the dense arm of the U is open.

Full derivation of Eq. (8) (the size bound). Since $|A \cap B| \le \min(a, b)$ and $|A \cup B| = a + b - |A \cap B| \ge \max(a, b)$, $J(A, B) = |A \cap B|/|A \cup B| \le \min(a, b)/\max(a, b)$, so $J(A, B) \ge t$ requires $\min(a, b) \ge t \max(a, b)$, and, since $J \le \min(a, b)/b \le 1$ trivially bounds $|A \cap B| \le \min(a, b)$, $|A \cap B| \ge \tau$ requires $\min(a, b) \ge \tau$. Both conditions are necessary and neither is sufficient, so the test only discards pairs and never reports one; survivors still go to an exact kernel. Hypotheses: $a, b > 0$, $t \in (0, 1]$; nothing is assumed about arrangement, about m, or about how either operand is stored. This is the Swamidass–Baldi pruning bound[12] together with the size (length) filter of[30], in the two-sided form standardised by[32] and applied jointly with prefix filtering to Tanimoto over binary vectors by[37]; the contribution of this note is its placement against the mechanisms above, not the bound itself.

Full derivation of Eq. (9) and its two hypotheses. Model a corpus by drawing cardinalities independently with $\log a$ uniform on $[0, \ln K]$ — the density $\Pr(a) \propto 1/a$, i.e. exactly the $1/i$ cardinality spectrum this paper's benchmark protocol mandates, not an approximation to it. Then $\min(a, b)/\max(a, b) \ge t$ is $|\log a - \log b| \le \gamma$ with $\gamma = \ln(1/t)$. For two independent uniforms on

an interval of length $L = \ln K$, the probability that $|X-Y| \le \gamma$ for $X, Y \sim \mathrm{Unif}[0, L]$ is $1-(1-\gamma/L)^2$ for $\gamma \le L$ (the complement event, $|X-Y| > \gamma$, has probability $(1-\gamma/L)^2$ by direct integration over the two corner triangles), giving Eq. (9), with absolute form $(1-\ln\tau/\ln K)^2$ for the $|A \cap B| \ge \tau$ reading. At $K = 10^4$ the surviving fraction is 0.1449 at $t = \frac{1}{2}$, 0.0479 at $t = 0.8$, 0.0227 at $t = 0.9$ and 0.0111 at $t = 0.95$; at $K = 10^3$, $t = 0.9$ it is 0.0303. Equivalently, examining every pair does 6.9×, 20.9×, 44.0×, 90.0× and 33.0× the work of examining only the survivors. These agree with a 10^5-sample simulation to three decimals at each point. Expanding for small γ gives $2\gamma \int \phi^2$ for any density ϕ of $\log a$, so the governing quantity is the *effective log-spread* $1/\int g^2$ of the size distribution, of which Eq. (9) is the log-uniform case.

Two hypotheses carry real weight. *First*, cardinalities are integers, and $a = b$ passes at every t, so the continuous form is optimistic where sizes are small: drawing integer cardinalities as the generator does leaves 0.0185 rather than 0.0111 surviving at $t = 0.95$, $K = 10^4$. *Second*, Eq. (9) decreases monotonically in K — at $t = 0.9$ the surviving fraction falls from 0.0452 at $K = 10^2$ to 0.0114 at $K = 10^8$ — but that improvement belongs to the log-uniform family, not to heavy tails generally. Under the alternative reading in which *ranked* sizes scale as $1/i$, giving a density $\propto 1/a^2$ whose log is exponential, the surviving fraction is exactly $1-t$, independent of K: the effective log-spread saturates at two nats however far the sizes are spread, and so does the gain. The favourable scaling in K is therefore claimed for the spectrum this paper benchmarks and not for every skewed corpus.

Full derivation of Eq. (10) (why the size bound does not multiply). Acting at a coarser granularity is not sufficient for savings to compose. The size test decides whether a pair is formed at all, while representation choice and the zone map act inside a pair already formed, so the granularity argument of Eq. (6) predicts a product; the product is not achieved, because both mechanisms read the same statistic. The survivors of Eq. (8) are the pairs with $a \approx b$, and every mechanism below is cheapest exactly when a, b are dissimilar, since its cost tracks $\min(a,b)$ against a fixed $\lceil m/w \rceil$. Under the log-uniform model above, conditioning on $\min/\max \ge t$ raises the mean cost of a survivor by the factor of Eq. (10), whose limit as $t \to 1$ is obtained by Taylor-expanding both the conditional and unconditional expectations of $\min(a,b) = e^{\min(X,Y)}$ in $\gamma = \ln(1/t)$ and taking the ratio; the leading term is $\ln K/2$. Numerically, at $K = 10^4, t = 0.9$ the factor is 4.40 against the limit $\ln K/2 = 4.61$. Simulating the composition directly reproduces the shortfall: the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition in fact costs 0.151%, a shortfall of 3.69× once cardinalities are integers; permuting which pairs pass the size test, so the two mechanisms read uncorrelated statistics at the same survival rate, restores the product to within 0.6%, isolating the shared statistic as the cause rather than the granularity. The composition is still strongly worth taking: at these parameters it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does 660× its work. What is refuted is only the product; the general rule is that mechanisms compose multiplicatively when they act on different waste *and* are driven by different operand statistics, and sharing a statistic costs the composition Eq. (10)'s factor even across granularities. All values here are analytic, computed over the log-uniform model at $m = 10^6$, and count work rather than time.

Supplementary Note 9: The representation-choice optimisation problem — full proofs

Methods ("Choosing a representation is a different problem from choosing a size" onward) states this development's results as closed forms with their hypotheses and a pointer here. Chunked setting: the universe is partitioned into chunks of M bits, each stored in one representation; $n = M/w$ words to a chunk, e bits to a stored array element. For an operation ω and a representation pair (ρ_A, ρ_B), the cell cost is $C_\omega(\rho_A, \mu_A; \rho_B, \mu_B) = \sum_j c_j N_j$, where $\mu_X = (k_X, r_X, \ell_X + g_X)$ is the operand's metadata, the N_j are counts of primitive items touched, and the $c_j > 0$ are per-item constants entering only through the ratios $\theta_{pw} = c_p/c_w$, $\theta_{mw} = c_m/c_w$, $\theta_{\rho w} = c_\rho/c_w$ for a membership probe, a merge step and a rank lookup respectively, each measured, not derived. Given a corpus, a distribution π over the chunk pairs actually evaluated, and an operation ω, the decision problem is to choose an assignment ρ : chunks $\to \{\mathsf{S}, \mathsf{B}, \mathsf{R}, \mathsf{W}\}$ minimising $C(\rho) = \sum_{(x,y)\sim\pi} C_\omega(\rho(x), \mu_x; \rho(y), \mu_y)$ subject to a byte budget F.

Proof that the storage objective is separable and the query objective is not. $\sum_x \mathrm{bytes}(\rho(x), \mu_x)$ is a sum of per-container terms, so its minimiser is obtained by minimising each term independently. $C(\rho)$ is a quadratic pseudo-Boolean function of the assignment: exhibiting one pair interaction suffices, and the cost model supplies it directly — $C_{\cap\mathrm{card}}(\mathsf{S}, k_A; \mathsf{B}, \cdot) = c_p k_A$ while $C_{\cap\mathrm{card}}(\mathsf{S}, k_A; \mathsf{S}, k_B) = c_m(k_A + k_B)$, so the cost attributable to chunk x depends on $\rho(y)$ for its partner y. This, not the value of any particular threshold, is the structural difference between the two objectives: a storage-optimal assignment is computed one container at a time with no knowledge of the workload; a query-optimal one cannot be, because there is no such thing as the cost of a container, only the cost of a pair.

Proof that a vendor's fixed storage threshold is M/e, and why it is a compile-time constant. Reading three independent container-promotion rules off a released amalgamation of CRoaring (v4.7.2, `third_party/croaring_amalg`) as an instance of the general result: array against bitset promotes iff $ek > M$; array against run promotes iff $\mathrm{bytes}(\mathsf{R}) < \mathrm{bytes}(\mathsf{S})$; bitset against run promotes iff $\mathrm{bytes}(\mathsf{R}) < \mathrm{bytes}(\mathsf{B})$. Each is a two-way comparison of serialized sizes, and the array-to-bitset

threshold $\tau_{\text{stor}} = M/e$ is exactly the cardinality at which the two containers occupy the same bytes. It contains no machine constant c_j, no property of π, ω or F: it is a function of the format alone, which is exactly why it can be a compile-time constant rather than a tuned one. *An aside on non-confluence.* Because each comparison is two-way and which two depends on the container's present type, the same set can end in different representations depending on how it arrived: from an array container the result is a run iff $2+4r < 2k$; from a bitset it is a run iff $2+4r < 8192$. The two disagree on $(k-1)/2 \le r \le 2047$, $k \le 4096$ — for instance $k = 3000, r = 1600$ gives array 6000 bytes, run 6402 bytes, bitset 8192 bytes: arriving as an array the container stays an array (since $6402 \ge 6000$); arriving as a bitset it becomes a run (since $6402 < 8192$), 402 bytes larger than the three-way minimum for the same set. A three-way minimum over all three serialized sizes is confluent and never larger than the present two-way rule.

Proof of Eq. (11) (the crossover as a formula). For a chunk of cardinality k under count-only intersection, promoting from array to bitset is favourable against a bitset partner iff $c_w n < c_p k$, i.e. $k > \tau_\cap = (c_w/c_p)(M/w) = n/\theta_{pw}$ — the promoted side now costs $c_w n$ (a full word scan) against the alternative $c_p k$ (one probe per element). Against an array partner of cardinality k_Y, promotion turns a two-sided merge into a one-sided probe, $c_m(k+k_Y) \to c_p k_Y$, favourable iff $k > (\theta_{pm}-1)k_Y$ with $\theta_{pm} = c_p/c_m$; in particular if $c_p \le c_m$, promotion helps at every k, since a bitset is the cheapest thing to be probed into. Dividing $\tau_{\text{stor}} = M/e$ by $\tau_\cap$ gives Eq. (11)'s ratio $(w/e)\theta_{pw}$; at $w/e = 4$ (CRoaring on a 64-bit host) the storage threshold exceeds the count-only-AND threshold by exactly $4\theta_{pw}$ and no other factor. Substituting the measured symmetric crossover (the cardinality at which an array–array merge stops beating a bitmap–bitmap popcount, $k \approx 64$–128 on the target host) into the three-way relation $\theta_{pw} = \theta_{mw}\theta_{pm}$ reproduces the reported 30–60× divergence between the two objectives from one measurement and no free parameters, under the assumption $\theta_{pm} \approx 2$ (a probe is an index, load, shift and test against a merge step's pointer advance and compare); that assumption is plausible but unmeasured; and measuring the two crossovers together over-determines the model, since θ_{pm} computed two ways must then agree.

Proof of the price-of-a-byte family (why every threshold between $\tau_\cap$ and τ_{stor} is optimal for some price). Introduce a byte price $\lambda \ge 0$ and minimise $C(\rho) + \lambda \cdot \text{bytes}(\rho)$. For the $\{\mathsf{S},\mathsf{B}\}$ decision on a container of cardinality k participating in ν pairs, write $\Delta w(k) \ge 0$ for the per-pair item saving from promotion and $\Delta b(k) = M/8 - ek/8$ for its byte cost. The rule "promote iff $\nu\,\Delta w(k) > \lambda\,\Delta b(k)$" is a threshold rule in k for every fixed λ, ν: $\Delta w(k)/\Delta b(k)$ is non-decreasing in k on $k < \tau_{\text{stor}}$ (numerator non-decreasing, denominator positive and strictly decreasing), so the promoted set is an up-set in k, and for $k \ge \tau_{\text{stor}}$, $\Delta b \le 0 \le \nu\Delta w$ so those k are promoted at every λ. The threshold is non-decreasing in λ, since the promoted set shrinks pointwise as λ grows; it equals $\tau_\cap$ at $\lambda = 0$ (the rule reduces to $\Delta w(k) > 0$) and tends to τ_{stor} as $\lambda \to \infty$ (only $\Delta b \le 0$ survives); and its range is, up to integer rounding, exactly $[\tau_\cap, \tau_{\text{stor}}]$, since the threshold is the crossing point of two continuous functions of k moving continuously between the two endpoints. So $\tau_\cap$ and τ_{stor} are not a right answer and a wrong answer; they are the two limits $\lambda \to 0$ and $\lambda \to \infty$ of one family, and every threshold between them is optimal at some price of a byte. A vendor's fixed threshold is the unique member of that family at which promotion is never paid for in bytes — the largest threshold at which the compute-favourable move is footprint-neutral, since promoting a chunk of cardinality k multiplies its footprint by τ_{stor}/k. One consequence is worth isolating: the threshold depends on ν, the number of pairs a container participates in, so two containers of identical k and different ν have different optima whenever $\lambda > 0$ — under any byte budget the optimal rule is not a function of the container at all, and no per-container rule attains it.

Proof of Eq. (12) and the full statement of the five hypotheses (the fixed-threshold-is-optimal result, stated first in the direction that weakens it). The claim, at its correct strength. If (H1) the candidate set is $\{\mathsf{S},\mathsf{B}\}$ only, (H2) the operation is count-only intersection, (H3) the per-item constants do not depend on resident footprint, (H4) there is no byte budget, and (H5) every chunk faces the same partner mix, then the optimal assignment *is* the fixed threshold $\tau_\cap$ — a machine constant independent of the corpus and of the pairing distribution. *Proof.* Under H1–H2 the cost of a pair is one of the entries derived above; under H5 the promotion decision is the same functional of k for every container; H4 removes coupling through a budget; H3 makes that functional's constants workload-independent. So the unqualified claim "no fixed constant is optimal" is false; each of the five hypotheses must be dropped explicitly, and each drop has an exact witness.

Dropping H1 (admitting R). No rule reading cardinality alone can rank $\{\mathsf{S},\mathsf{B},\mathsf{R}\}$: at fixed k the run count attains every integer in $[1, \min(k, M-k+1)]$, the $\mathsf{R}\times\mathsf{B}$ cost is strictly increasing in it, and the $\mathsf{S}\times\mathsf{B}$ cost does not depend on it, so two containers of the same cardinality can have different optimal representations and any k-only rule mis-ranks one of them — the impossibility argument above, specialised to the container decision.

Dropping H5 (partner mix). Let a container of cardinality k face bitset partners with probability β and array partners of mean cardinality $\bar{k}$ otherwise. Promotion is favourable iff $\beta c_p k + (1-\beta)c_m(k+\bar{k}) > \beta c_w n + (1-\beta)c_p\bar{k}$, giving the threshold of Eq. (12), which interpolates between $\tau^*(1,\cdot) = \tau_\cap$ and $\tau^*(0,\bar{k}) = (\theta_{pm}-1)\bar{k}$: a statistic of the workload, not of the machine, since two corpora with identical containers and different pairing distributions have different optima. *The fixed point.* β is not exogenous, since partners are themselves subject to the same rule; writing $\beta(\tau)$ for the pair-weighted fraction of partners with $k > \tau$, an optimal fixed threshold satisfies $\tau = \tau^*(\beta(\tau), \bar{k}(\tau))$. Both sides are bounded and τ^* is continuous in β, so a

fixed point exists on $[0,M]$ by the intermediate value theorem whenever $\beta(\cdot)$ is continuous; it need not be unique. A library's container threshold is therefore a fixed point of its own workload, which is exactly why it cannot be a compile-time constant without an objective, such as storage, that removes the dependence.

Dropping H4 (byte budget). The threshold is the dual variable of the footprint constraint above and sweeps the whole interval $[\tau_\cap, \tau_{\text{stor}}]$.

Dropping H2 (the operation). For cardinality-only queries the operation dimension collapses and this is not an approximation: union, difference and symmetric-difference cardinalities are recovered from intersection cardinality by inclusion–exclusion, so all four cardinality queries share one cost matrix. But for *materialising* operations the matrix differs in sign: a materialising union with a bitset operand must produce a bitset-sized result, costing n words whatever the other side is, so promotion helps at every k there and only above $\tau_\cap > 0$ for a count — the two thresholds are 0 and $\tau_\cap$, and no single stored assignment is optimal for a workload mixing the two unless one operation dominates.

Dropping H3 (residency), and this is the one the measurements say dominates. c_w is not a constant: a bitset chunk occupies $M/8$ bytes and an array chunk $ek/8$, so promoting a container multiplies its footprint by $M/(ek) = \tau_{\text{stor}}/k$ — $64\times$ at $k = 64$ and $1\times$ at $k = \tau_{\text{stor}}$. τ_{stor} is therefore also the unique threshold at which promotion never inflates the working set; every lower threshold buys items with bytes at an exchange rate of $\tau_{\text{stor}}/k - 1$, and those bytes are paid in c_w, which rises as the corpus leaves cache. This is the only one of the five hypotheses whose failure is not analytic, and it is treated as a measurement question rather than an analytic one, exactly as Eq. (6)'s divergence between a work model and measured time is.

How far a fixed choice can be from optimal. Bounded exactly under H1–H3 and H5, against the family of the previous paragraph: a container at cardinality k paired with bitsets costs $\min(c_p k, c_w n)$ at the optimum and $c_p k$ or $c_w n$ under a fixed τ, and the worst case sits at the far endpoint, $\max_k C(\tau_{\text{stor}};k)/C(\tau_\cap;k) = c_p\tau_{\text{stor}}/(c_w n) = \tau_{\text{stor}}/\tau_\cap = (w/e)\theta_{pw}$, attained at $k = \tau_{\text{stor}}$; symmetrically, running at $\tau_\cap$ inflates the footprint of a $k = \tau_\cap$ container by the same ratio in both directions.

Proof of Proposition 18/19 (metadata settles the decision wherever the decision matters). Two identifiability questions must not be conflated. Eq. (2) says what cardinality determines about the *answer*, pinned only at the density extremes. What it determines about the *decision* is different and more favourable. If $\mu = (k, r, \ell + g)$, every cost above is a closed-form function of (μ_A, μ_B, n, w) and the constants, so the arg min is too: the decision is determinable exactly, in $O(1)$, touching no operand data — the formal content of "selection must be near-free", a consequence of the cost model rather than an engineering aspiration. If $\mu = (k)$ alone, the decision is *not* determinable whenever R is a candidate (the H1-witness above); one extra integer, the run count, restores determinability, and more cardinality precision does not.

Suppose each constant c_j is known only up to a multiplicative factor $\sqrt{\eta}$ in each direction, so any modelled cost is known within a factor $\eta \geq 1$. Order the candidate cells by modelled cost, $C_1 \leq C_2 \leq \ldots$, and define the margin $\Lambda = C_2/C_1$. *Proof.* If $\Lambda > \eta$, even the most adverse admissible perturbation of the constants leaves $C_1 < C_2$, so the true ordering of the two best cells is determined and the selector's choice is correct. If $\Lambda \leq \eta$, then $C_2 \leq \eta C_1$, so the ordering is not determined by μ but the cost of choosing wrong is at most η, by the definition of Λ. So for every pair, either the metadata settles the decision or the decision does not matter to within the precision of the constants, and the undetermined set is an η-neighbourhood of the boundaries derived above rather than a region of unbounded error. This is a different mid-band from the one in Eq. (2): Proposition 2 (main text) says the *answer* is pinned by cardinality only at the density extremes; this result says the *decision* is pinned everywhere except an η-neighbourhood of a boundary, and the two regions have different consequences and must not be merged — at $d = \frac{1}{2}$ cardinality determines nothing about the answer while determining the decision completely (B×B, Supplementary Note 8). The caveat that makes η interesting rather than cosmetic: η is the uncertainty of the *model*, not of the constants alone, and where the model omits the memory system — the same divergence recorded under Eq. (6) — it is not small. That is the derived motivation for measuring a candidate kernel rather than only modelling it, in a narrower and more defensible form than "the theory implies probe-and-commit": it says only that η is large in a named regime, and that a mechanism which reduces η is worth its cost there.

Proof of Eq. (13) (why an advantage survives repairing the incumbent's kernel). Write the total cost of a batch as $C(\rho) = C_{\text{meta}} + \sum_{(A,B)\in\mathscr{P}_{\text{surv}}} \sum_{z\in Z(A,B)} C_{\text{cell}}(\rho(A|z), \rho(B|z))$, where $\mathscr{P}_{\text{surv}} \subseteq \mathscr{P}$ are the pairs not discarded by metadata and $Z(A,B)$ the regions actually visited. $\mathscr{P}_{\text{surv}}$ and $Z(A,B)$ are functions of the metadata and the data only: the size bound reads (a,b), the zone-map test reads bin occupancy, the narrowed-contract mode reads (a,b,t), and none reads a representation — since every representation here is exact and interconvertible, changing ρ changes no membership. Consequently, for a uniform kernel improvement by a factor $\alpha > 1$ (every C_{cell} divided by α), $G(\alpha) = \dfrac{C_{\text{base}}/\alpha}{C_{\text{meta}} + N_{\text{surv}}\bar{E}_{\text{surv}}/\alpha} = \dfrac{C_{\text{base}}}{\alpha C_{\text{meta}} + N_{\text{surv}}\bar{E}_{\text{surv}}}$ by direct substitution, where $C_{\text{base}} = N_{\text{all}}\bar{E}_{\text{all}}$ is the same batch with no avoidance: invariant in α exactly when $C_{\text{meta}} = 0$, and eroding only through the summary's share of the post-avoidance cost. Improving a kernel therefore cannot subsume avoidance, because avoidance changes which work is asked for and a kernel changes only what the asked-for work costs — but the two are not independent, and which savings multiply is settled by the rule of Eq. (10): a zone map reads bin occupancy while

representation choice reads cardinality and run count, so those multiply; the size bound reads cardinality, which representation choice also reads, so those do not, and a container-threshold gain must not be multiplied into a size-bound gain.

The peak, derived. Working inside a chunk both operands occupy (crediting the incumbent its own chunk-level skip in full), against an incumbent already holding the compute-optimal threshold $\tau_\cap$, with the challenger adding a zone map of bin width W_z at overhead c/W_z, the modelled advantage is $G(d) = \kappa_{\mathrm{Ro}}(d)/\big(c/W_z + p_A p_B\,\kappa(d/p)\big)$ with $\kappa_{\mathrm{Ro}}(d) = \min(1, \theta_{pw} w d)$ the two-cell (array/bitset) envelope and $p = 1-(1-d)^{W_z}$ (Supplementary Note 8, Eq. (S9)). In the sparse band, where $d/p \to 1/W_z$ puts a singly-occupied bin above the compute crossover whenever $W_z \le \theta_{pw} w$, $\kappa(d/p) \equiv 1$ and, substituting $p \approx x = W_z d$ for $x \ll 1$, $G(x) = \theta_{pw} w x/\big(c + W_z(1-e^{-x})^2\big)$. Setting $\mathrm{d}G/\mathrm{d}x = 0$ gives $x^* = \sqrt{c/W_z}$ and the peak value of Eq. (13), valid on $(\theta_{pw} w\sqrt{c})^{2/3} \le W_z \le \theta_{pw} w$: below the lower edge d^* would exceed $1/(\theta_{pw} w)$, where κ_{Ro} saturates at 1 and the peak is capped below the formula's value; above the upper edge $\kappa(d/p) < 1$ and the peak saturates at the W_z-independent value $\sqrt{\theta_{pw} w/c}/2$. The same constant sets a natural bin width: since an occupied bin holds at least one bit in W_z positions, a summary hands the kernel nothing sparser than that, and $W_z^* = \theta_{pw} w$ is the width at which a singly-occupied bin sits exactly at the array–bitset compute crossover, linking the zone-map granularity and the container threshold through one measured ratio. At $\theta_{pw} = 16$, $w = 64$, $c = 2$ the valid window is $128 \le W_z \le 1024$ and the saturated ceiling is $11.3\times$; at $W_z = 512$ (the configuration used throughout the figures) the closed form gives $16.0\times$.

Two consequences are stated rather than smoothed. Eq. (13) is a null-model ceiling on structureless data, and by the one-sidedness of σ (Supplementary Note 8), real corpora can only fall below the null, so a measured advantage exceeding the ceiling is evidence of structure and must be reported as such rather than as agreement with the model — a measured range whose lower and middle values sit inside the ceiling and whose top exceeds it is the expected signature of real structure, not a discrepancy to explain away. And Eq. (13) excludes the narrowed contract entirely: Eq. (9) supplies its own factor, and by Proposition 21's rule the two are not multiplied together.

References

1. Wu, K., Otoo, E. J. & Shoshani, A. Optimizing bitmap indices with efficient compression. *ACM Transactions on Database Syst.* **31**, 1–38, DOI: 10.1145/1132863.1132864 (2006).

2. Chambi, S., Lemire, D., Kaser, O. & Godin, R. Better bitmap performance with Roaring bitmaps. *Software: Pract. Exp.* **46**, 709–719, DOI: 10.1002/spe.2325 (2016). ArXiv:1402.6407.

3. Lemire, D. *et al.* Roaring bitmaps: Implementation of an optimized software library. *Software: Pract. Exp.* **48**, 867–895, DOI: 10.1002/spe.2560 (2018). ArXiv:1709.07821.

4. Zobel, J. & Moffat, A. Inverted files for text search engines. *ACM Comput. Surv.* **38**, 6:1–6:56, DOI: 10.1145/1132956.1132959 (2006).

5. Goodwin, B. *et al.* BitFunnel: Revisiting signatures for search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 605–614, DOI: 10.1145/3077136.3080789 (2017).

6. The Apache Software Foundation. Lucene `SparseFixedBitSet`. https://lucene.apache.org/core/. Accessed 2026-08-06.

7. Aberger, C. R., Tu, S., Olukotun, K. & Ré, C. EmptyHeaded: A relational engine for graph processing. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, 431–446, DOI: 10.1145/2882903.2915213 (2016).

8. Yang, F. *et al.* Druid: A real-time analytical data store. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, 157–168, DOI: 10.1145/2588555.2595631 (2014).

9. Broder, A. Z. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences*, 21–29, DOI: 10.1109/SEQUEN.1997.666900 (1997).

10. Bayardo, R. J., Ma, Y. & Srikant, R. Scaling up all pairs similarity search. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*, 131–140, DOI: 10.1145/1242572.1242591 (2007).

11. Xiao, C., Wang, W., Lin, X. & Yu, J. X. Efficient similarity joins for near duplicate detection. In *Proceedings of the 17th International Conference on World Wide Web (WWW)*, 131–140, DOI: 10.1145/1367497.1367516 (2008).

12. Swamidass, S. J. & Baldi, P. Bounds and algorithms for fast exact searches of chemical fingerprints in linear and sublinear time. *J. Chem. Inf. Model.* **47**, 302–317, DOI: 10.1021/ci600358f (2007).

13. Haque, I. S., Pande, V. S. & Walters, W. P. Anatomy of high-performance 2D similarity calculations. *J. Chem. Inf. Model.* **51**, 2345–2351, DOI: 10.1021/ci200235e (2011).

14. Dalke, A. The chemfp project. *J. Cheminformatics* **11**, 76, DOI: 10.1186/s13321-019-0398-8 (2019).

15. Gustavson, F. G. Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Transactions on Math. Softw.* **4**, 250–269, DOI: 10.1145/355791.355796 (1978).

16. Buluç, A. & Gilbert, J. R. The combinatorial BLAS: Design, implementation, and applications. *Int. J. High Perform. Comput. Appl.* **25**, 496–509, DOI: 10.1177/1094342011403516 (2011).

17. Karp, R. M., Waarts, O. & Zweig, G. The bit vector intersection problem. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science (FOCS)*, 621–630, DOI: 10.1109/SFCS.1995.492663 (1995).

18. Muła, W., Kurz, N. & Lemire, D. Faster population counts using AVX2 instructions. *The Comput. J.* **61**, 111–120, DOI: 10.1093/comjnl/bxx046 (2018). ArXiv:1611.07612.

19. Inoue, H., Ohara, M. & Taura, K. Faster set intersection with SIMD instructions by reducing branch mispredictions. *Proc. VLDB Endow.* **8**, 293–304, DOI: 10.14778/2735508.2735518 (2014).

20. Lemire, D., Boytsov, L. & Kurz, N. SIMD compression and the intersection of sorted integers. *Software: Pract. Exp.* **46**, 723–749, DOI: 10.1002/spe.2326 (2016). ArXiv:1401.6399.

21. Fu, Y.-X. Statistical properties of segregating sites. *Theor. Popul. Biol.* **48**, 172–197, DOI: 10.1006/tpbi.1995.1025 (1995).

22. Demaine, E. D., López-Ortiz, A. & Munro, J. I. Adaptive set intersections, unions, and differences. In *Proceedings of the 11th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 743–752 (2000).

23. Ding, B. & König, A. C. Fast set intersection in memory. *Proc. VLDB Endow.* **4**, 255–266, DOI: 10.14778/1938545.1938550 (2011).

24. Culpepper, J. S. & Moffat, A. Efficient set intersection for inverted indexing. *ACM Transactions on Inf. Syst.* **29**, 1:1–1:25, DOI: 10.1145/1877766.1877767 (2010).

25. Aberger, C. R. *et al.* EmptyHeaded: A relational engine for graph processing. *ACM Transactions on Database Syst.* **42**, 20:1–20:44, DOI: 10.1145/3129246 (2017).

26. An, X., Gabert, K., Fox, J., Green, O. & Bader, D. A. Skip the intersection: Quickly counting common neighbors on shared-memory systems. In *Proceedings of the IEEE High Performance Extreme Computing Conference (HPEC)*, 1–7, DOI: 10.1109/HPEC.2019.8916307 (2019).

27. Intel Corporation. *Intel Intrinsics Guide* (2024). Version 3.6.9, 12 July 2024.

28. Răducanu, B., Boncz, P. & Żukowski, M. Micro adaptivity in Vectorwise. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 1231–1242, DOI: 10.1145/2463676.2465292 (2013).

29. Sarawagi, S. & Kirpal, A. Efficient set joins on similarity predicates. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, 743–754, DOI: 10.1145/1007568.1007652 (2004).

30. Arasu, A., Ganti, V. & Kaushik, R. Efficient exact set-similarity joins. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*, 918–929 (2006).

31. Xiao, C., Wang, W., Lin, X., Yu, J. X. & Wang, G. Efficient similarity joins for near-duplicate detection. *ACM Transactions on Database Syst.* **36**, 15:1–15:41, DOI: 10.1145/2000824.2000825 (2011).

32. Mann, W., Augsten, N. & Bouros, P. An empirical evaluation of set similarity join techniques. *Proc. VLDB Endow.* **9**, 636–647, DOI: 10.14778/2947618.2947620 (2016).

33. Sandes, E. F. d. O., Teodoro, G. & Melo, A. C. M. A. Bitmap filter: Speeding up exact set similarity joins with bitwise operations. *Inf. Syst.* **88**, 101449, DOI: 10.1016/j.is.2019.101449 (2020).

34. Wang, J., Li, G. & Feng, J. Can we beat the prefix filtering? an adaptive framework for similarity join and search. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, 85–96, DOI: 10.1145/2213836.2213847 (2012).

35. Mood, A. M. The distribution theory of runs. *The Annals Math. Stat.* **11**, 367–392, DOI: 10.1214/aoms/1177731825 (1940).

36. Lang, H. *et al.* Tree-encoded bitmaps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 937–967, DOI: 10.1145/3318464.3380588 (2020).

37. Anastasiu, D. C. & Karypis, G. Efficient identification of Tanimoto nearest neighbors. In *Proceedings of the IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 156–165, DOI: 10.1109/DSAA.2016.23 (2016).

38. Jacobson, G. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, 549–554, DOI: 10.1109/SFCS.1989.63533 (1989).

39. Clark, D. R. *Compact Pat Trees*. Ph.D. thesis, University of Waterloo (1996).

40. Vigna, S. Broadword implementation of rank/select queries. In *Experimental Algorithms (WEA 2008)*, vol. 5038 of *Lecture Notes in Computer Science*, 154–168, DOI: 10.1007/978-3-540-68552-4_12 (Springer, 2008).

41. Moerkotte, G. Small materialized aggregates: A light weight index structure for data warehousing. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*, 476–487 (1998).

42. Sidirourgos, L. & Kersten, M. L. Column imprints: A secondary index structure. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 893–904, DOI: 10.1145/2463676.2465306 (2013).

43. Sidirourgos, L. & Mühleisen, H. Scaling column imprints using advanced vectorization. In *Proceedings of the 13th International Workshop on Data Management on New Hardware (DaMoN)*, 4:1–4:8, DOI: 10.1145/3076113.3076120 (2017).

44. The PostgreSQL Global Development Group. Block range index (BRIN) in PostgreSQL. https://www.postgresql.org/docs/current/brin.html. Accessed 2026-08-06.

45. Alexiou, K., Kossmann, D. & Larson, P.-Å. Adaptive range filters for cold data: Avoiding trips to siberia. *Proc. VLDB Endow.* **6**, 1714–1725, DOI: 10.14778/2556549.2556556 (2013).

46. Yu, J. & Sarwat, M. Two birds, one stone: A fast, yet lightweight, indexing scheme for modern database systems. *Proc. VLDB Endow.* **10**, 385–396, DOI: 10.14778/3025111.3025120 (2016).

47. Morzy, M., Morzy, T., Nanopoulos, A. & Manolopoulos, Y. Hierarchical bitmap index: An efficient and scalable indexing technique for set-valued attributes. In *Advances in Databases and Information Systems (ADBIS)*, vol. 2798 of *Lecture Notes in Computer Science*, 236–252, DOI: 10.1007/978-3-540-39403-7_19 (2003).

48. Lemire, D., Kaser, O. & Aouiche, K. Sorting improves word-aligned bitmap indexes. *Data & Knowl. Eng.* **69**, 3–28, DOI: 10.1016/j.datak.2009.08.006 (2010). ArXiv:0901.3751.

49. Lemire, D., Ssi-Yan-Kai, G. & Kaser, O. Consistently faster and smaller compressed bitmaps with Roaring. *Software: Pract. Exp.* **46**, 1547–1569, DOI: 10.1002/spe.2402 (2016). ArXiv:1603.06549.

50. Chaudhuri, S., Ganti, V. & Kaushik, R. A primitive operator for similarity joins in data cleaning. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 5–16, DOI: 10.1109/ICDE.2006.9 (2006).